

D7.4

First demonstration of AI systems security mechanisms and tools

Project number	830892
Project acronym	SPARTA
Project title	Strategic programs for advanced research and technology in Europe
Start date of the project	1st February, 2019
Duration	36 months
Programme	H2020-SU-ICT-2018-2020

Deliverable type	Report
Deliverable reference number	SU-ICT-03-830892 / D7.4/ V1.0
Work package contributing to the deliverable	WP7
Due date	January 2020 – M24
Actual submission date	29th January 2021

Responsible organisation	TCS
Editor	Vincent Thouvenot
Dissemination level	CO
Revision	V1.0

Abstract	This deliverable will describe the first version of the created defensive and reactive mechanisms, explainability enhancing mechanisms and fairness ensuring mechanisms. Here we focus more in details on selected approaches.
Keywords	Explainability, fairness, Adversarial attacks and defense



Editor

THOUVENOT Vincent (TCS)

Contributors (ordered according to beneficiary numbers)

CHIHANI Zakaria (CEA)
KNOCKAERT Manon, VAN GYSEGHEM Jean-Marc (UNamur)
NOROUZIAN Mohammed Reza (TUM)
RIOS VELASCO Erkuden, ITURBE ZAMALLOA Eider, PALACIOS PRADOS M^a Carmen, MARTINEZ MARTINEZ Cristina (TEC)
ECHEBERRIA BARRIO Xabier, GIL LERCHUNDI Amaia, ORDUNA URRUTIA Raul (VICOM)
GRAH Simon, ADDAD Boussad, THOUVENOT Vincent (TCS)
CHORAŚ Michał, PAWLICKI Marek, SZCZEPAŃSKI Mateusz (ITTI)

Reviewers (ordered according to beneficiary numbers)

BIANCHI Giuseppe (CNIT)

Disclaimer

The information in this document is provided "as is", and no guarantee or warranty is given that the information is fit for any particular purpose. The content of this document reflects only the author's view – the European Commission is not responsible for any use that may be made of the information it contains. The users use the information at their sole risk and liability.

Executive Summary

This document is the deliverable D7.4 of SPARTA and presents the work of the SAFAIR program (SPARTA WP7). The program, and the document which follows, seeks to address two major contemporary problems caused by the proliferation of Artificial Intelligence (AI) and Machine Learning (ML). The first objective is to ensure the security and robustness of AI/ML solutions, the second objective deals with the issue of trustworthiness and fairness of AI/ML.

To the first end, the SAFAIR program produces formalisms, methods and tools which make current AI algorithms more robust, ensuring their security, reliability and privacy. While the threat analysis conducted by the SAFAIR program made its way to the D7.1, the D7.2 holds the information regarding the defensive and reactive mechanisms designed to ensure resilience against the new, complex cyber-threats identified in D7.1. The preliminary descriptions of those tools and methods are offered in Chapter 2 of D7.2. The mechanisms and tools proposed by the SAFAIR program seek to optimize resiliency without compromising the advantages provided by AI and aiming not to affect the performance of AI.

Secondly, the SAFAIR program seeks to extend the explainability of AI, which usually comes in the form of a black-box. This provides both the verification of functionality of AI and gives users relying on decisions made by AI the ability to trace back those decision to the inputs, paving the way to better understanding of AI, increasing the transparency of AI and increasing the confidence in AI. In Chapter 3 of D7.2 some explainability mechanisms and tools can be found. The third aspect of the above mentioned objectives deals with the legal and societal aspects of trustworthiness and fairness of AI. The third task of SAFAIR seeks to provide mechanisms to reduce conscious and unconscious bias in AI decisions, ensuring that functionality does not introduce discrimination, and finding ways to grant both credibility and correct compliance with relevant legislative regulations. The current state of that work is contained in chapter 4 of D7.2.

D7.3 is dedicated to test and evaluate the adversarial machine learning solutions. To this end, D7.3 proposes two approaches: i) design an open AI contest in the adversarial environment to facilitate measurable progress towards robust machine learning models and, more generally, applicable adversarial attacks and ii) implement an open-source python tool that provides standardized reference implementations of adversarial example construction techniques and adversarial training.

D7.4 is the report due for Month 24 that presents the first demonstration of AI systems security mechanisms and tools. Compared to D7.2, which presented the state of art of such mechanisms, the objective of D7.4 is to give a detailed description of the approaches chosen in SAFAIR for security, trustworthiness and fairness of AI systems, which will be developed in the final demonstrator and applied in use cases to illustrate them. In Chapter 2 we identify the documents published at international and European level dealing with the issue of Artificial Intelligence and the ethical deployment of these technologies. Chapter 3 presents the AI security methods, Chapter 4 is focused on Explainability of AI and Chapter 5 on Fairness of AI.

Table of Content

Chapter 1	Introduction	1
Chapter 2	Legal issues concerning trustworthy AI	3
2.1	Methodology	3
2.2	Main findings from the deliverable D7.2	3
2.3	Deepening the notion of fairness for artificial intelligence tools: International recommendations	4
2.3.1	Council of Europe	4
2.3.2	G20 principles on AI	7
2.4	European recommendations	8
2.4.1	List of new documents (towards the ones presented in the deliverable D7.2) used in this deliverable	8
2.4.2	Beyond data and artificial intelligence: a human-centric approach in the digital strategy for Europe	8
2.4.3	Recommendations to frame the use of data	10
2.4.4	Recommendations to frame the use of artificial intelligence technologies	11
2.5	Trustworthy AI and GDPR principles	12
2.6	Main characteristics and conclusions	14
Chapter 3	Attack techniques and Defence mechanisms	16
3.1	Evasion attacks	16
3.1.1	Fast Gradient Sign Method (FGSM)	16
3.1.2	Basic Iterative method	17
3.1.3	Carlini and Wagner approach	17
3.1.4	Centered Initial Attack (CIA)	17
3.2	Defence mechanisms under study	17
3.2.1	Adversarial Train	17
3.2.2	Dimensionality Reduction	18
3.2.3	Prediction Similarity	18
3.2.4	Model's Behaviour	18
3.2.5	Feature scattering	18
3.2.6	Double-ended monitoring for adversarial detection	18
3.3	Experimentation	20
3.3.1	PDF Malware detection	20
3.3.2	Defending Healthcare Images against Adversarial Evasion Attacks	27
3.3.3	Evasion Attacks on Graph Anomaly Detection	36
3.3.4	Defending AI algorithms used in Network Intrusion Detection against evasion attacks	50
Chapter 4	Explainability	57
4.1	Introduction	57
4.2	Attribute Importance based on Shapley Values	57
4.2.1	Shapley Values definition	57
4.2.2	Shapley Values as contrastive local attribute importance in Machine Learning	58
4.2.3	Estimation of Shapley Values by Monte Carlo	59
4.2.4	Estimation of Shapley Values using a projected stochastic gradient algorithm	59
4.2.5	Implementation: Python module Shapkit	61
4.2.6	Experimental evaluation: Adult income use case	62
4.3	xAI in intrusion detection	63
4.3.1	Theoretical foundation	64
4.3.2	Implementation	65
4.3.3	Presentation of the web application	70
4.3.4	Evaluation	71
4.3.5	Summary	73
Chapter 5	Fairness	74
5.1	Introduction	74
5.2	Measures	74
5.2.1	Group fairness	74



5.2.2	Individual Fairness	75
5.2.3	The Impossibility theorem of fairness	75
5.3	Algorithms	76
5.3.1	Fair Adversarial Networks	76
5.3.2	Fair Random Forest	77
5.4	Experimental evaluation: Adult income use case	77
5.4.1	Fairness regarding the “Sex” feature	78
5.4.2	Fairness regarding the “Sex” and “Race” features	78
Chapter 6	Summary and Conclusion	81
Chapter 7	List of Abbreviations	82
Chapter 8	Bibliography	84

List of Figures

Figure 1: Requirements of the GDPR	13
Figure 2: Criteria for fairness in data and algorithms used for AI	15
Figure 3: Criteria for fairness in the deployment of AI	15
Figure 4: Adversarial training principle	18
Figure 5: Original network	19
Figure 6: Extension of the original network in Figure 5 with a monitoring second end	19
Figure 7: PDFiD output format	21
Figure 8: Models performance against FGSM attack (FGSM method is used for adversarial example generation).	23
Figure 9: Models performance against iter-FGSM attack (FGSM method is used for adversarial example generation).	23
Figure 10: Models performance against CW attack (FGSM method is used for adversarial example generation).	24
Figure 11: performance against CIA attack (FGSM method is used for adversarial example generation).	24
Figure 12: performance against FGSM attack (CIA method is used for adversarial example generation)	25
Figure 13: performance against iter-FGSM attack (CIA method is used for adversarial example generation)	25
Figure 14: performance against CW attack (CIA method is used for adversarial example generation)	26
Figure 15: performance against CIA attack (CIA method is used for adversarial example generation)	26
Figure 16: different images that form the breast cancer images dataset	27
Figure 17: Architecture of the final model	29
Figure 18: Adversarial training schema.	31
Figure 19: Model's behaviour schema	34
Figure 20: Features examples	35
Figure 21: Classification accuracy for original images, known adversarials data and new adversarials	35
Figure 22: Attacking Graph Anomaly Detection System Description	37
Figure 23: Dataset Class Structure	38
Figure 24: System Description	40
Figure 25: Pickleable Class Structure	41
Figure 26: Attack Class Structure	43
Figure 27: Some features and the class labels of CICIDS2017 after binarisation	51
Figure 28: The IDS ANN classification of the same samples before and after evasion attacks	53
Figure 29: The acquisition of IDS ANN activations for a given test sample	54
Figure 30: The ANN serving as the adversarial attack detector during training	55
Figure 31: Illustration of how are used Attribute Importance on a particular prediction on the COMPAS dataset [17].	58
Figure 32: Attribute Importance	62
Figure 33: Overview of the explanation generation process	64
Figure 34: Sample selection	70
Figure 35: Example of an explanatory tree chart (representativity=0.005, max tree depth=3)	71
Figure 36: Example of an explanatory tree chart (representativity=0.2, max tree depth=5)	72
Figure 37: Overview of adversarial network architecture of [33]	76



List of Tables

Table 1:	Common recommendations at the International level	8
Table 2:	Criteria for trustworthy AI by the High-level Expert Group on Artificial Intelligence	11
Table 3:	Graph Anomaly Evasion Attack Results	50
Table 4:	'IDS ANN' trained on Dataset A and tested on Dataset B	52
Table 5:	Results of Adversarial Attack Detector over the test set activation using the ANN and RF classifiers	55
Table 6:	Results of Nearest Neighbour-based Adversarial Attack Detector over the test set activation	56
Table 7:	<i>shapkit</i> requirements	61
Table 8:	Feature description of the Adult Income data set	63
Table 9:	Main Python modules used in the system	66
Table 10:	Tested Variants of Explainers	71
Table 11:	Explainer Scores with Sample Support for $k=0.2$	71
Table 12:	Explainer Scores with Sample Support for $k=0.005$	73
Table 13:	Baseline metrics from a dummy classifier (always predicts the majority class with max probability)	77
Table 14:	Fair Adversarial Networks - Protected feature "Sex"	78
Table 15:	Fair Forest - Protected feature "Sex"	78
Table 16:	Fair Adversarial Networks - Protected both "Sex" and "Race" features	79
Table 17:	Fair Forest - Protected both "Sex" and "Race" features	79
Table 18:	Fair Adversarial Networks - "Sex" Fairness metrics	79
Table 19:	Fair Adversarial Networks - "Race" Fairness metrics	80
Table 20:	Fair Forest - "Sex" Fairness metrics	80
Table 21:	Fair Forest - "Race" Fairness metrics	80

Chapter 1 Introduction

This document is the deliverable D7.4 of SPARTA and presents the work of the SAFAIR program (SPARTA WP7). The program, and the document which follows, seeks to address two major contemporary problems caused by the proliferation of Artificial Intelligence (AI) and Machine Learning (ML). The first objective is to ensure the security of AI/ML solutions, the second objective deals with the issue of trustworthiness and fairness of AI/ML.

To the first end, the SAFAIR program produces formalisms, methods and tools which make current AI algorithms more robust, ensuring their security, reliability, and privacy. While the threat analysis conducted by the SAFAIR program made its way to the D7.1, the D7.2 work holds the information regarding the defensive and reactive mechanisms designed to ensure resilience against the new, complex cyber-threats identified in D7.1. The preliminary descriptions of those tools and methods are offered in Chapter 2 of D7.2. The mechanisms and tools proposed by the SAFAIR program seek to optimize resiliency without compromising the advantages provided by AI and aiming not to affect the performance of AI. Secondly, the SAFAIR program seeks to extend the explainability of AI, which usually comes in the form of a black-box. This provides both the verification of functionality of AI and gives personnel relying on decisions made by AI the ability to trace back those decisions to the inputs, paving the way to better understanding of AI, increasing the transparency of AI and increasing the confidence in AI. In Chapter 3 of D7.2 some explainability mechanisms and tools can be found. One of the above mentioned objectives deals with the legal and societal aspects of trustworthiness and fairness of AI. The third task of SAFAIR seeks to provide mechanisms to reduce conscious and unconscious bias in AI decisions, ensuring functionality that does not introduce discrimination, and finding ways to grant both credibility and correct compliance with relevant legislative regulations. The current state of that work is contained by Chapter 4 of D7.2. D7.3 is dedicated to test and evaluate the adversarial machine learning solutions.

D7.4 is the report due for Month 24 that presents the first demonstration of AI systems security mechanisms and tools. Compared to D7.2, which presented the state of art of such mechanisms, the objective of D7.4 is to give a detailed description of the approaches chosen in SAFAIR for security, trustworthiness and fairness of AI systems, which will be developed in the final demonstrator and applied in use cases to illustrate them. In Chapter 2 we identify the documents published at international and European level dealing with the issue of artificial intelligence and the ethical deployment of these technologies. It allows to propose some criteria for trustworthy AI around data and algorithms (GDPR compliance, data quality, security and robustness, traceability, reproducibility, predictability, regular audits) and some criteria that encounters the deployment of artificial intelligence (human intervention and human oversight, Transparency/Explainability, right to contest the outcome of AI and effective remedies). In SAFAIR program, we address three of these criteria: security and robustness, explainability and fairness of algorithms. Chapter 3 is dedicated to attack and defence mechanisms in ML. More precisely, we work on evasion attacks and defence mechanisms against them. We propose in the demonstrator classical methods of evasion attack, as well as the method proposed by [3]. Concerning defence, we propose some classical approaches, plus one which will be submitted in a future conference. Four use case are considered: two concern cybersecurity (malware detection in PDF and intrusion detection with AI), one other is about evasion attacks on graph anomaly detection and the last one is medical (healthcare images). Chapter 4 is dedicated to interpretability of machine learning models. We present the use of Shapley Values dedicated to local explanation of machine learning models as well as a hybrid oracle-explainer approach based on comprehensible tree. The Shapley Values, which come from collaborative games theory, have been applied in Machine Learning for local explanation of model, i.e. for the explanation of why a machine learning model predicts a value for a given observation. It allows to fairly distribute a gain, which in Machine Learning is the prediction made by the model, to a set of players, in ML the features, which can collaborate together. By fairly distribute, we mean that the distribution of the gain has to be efficient, i.e. that only and all the gain has to be distributed, symmetric, i.e. that regardless of the order in which the players are paid, they always should have the same pay, null for a player that has not contributed to the game, and additive. Despite their good theoretical properties, Shapley Values have a major issue: the number of mandatory computation grows exponentially with the number of players, which means for ML that it grows exponentially with the number of features. In Chapter 4 we introduce two methods to approximate them that we have published in [50]. The first one is based on the optimization of an existing Monte Carlo estimation and the second one use a projected gradient descent algorithm. Then we present Python module *shapkit*¹ that implements these approaches. Finally we apply this module on the adult income dataset (see Section 4.2.6.1 for a presentation of the use case).

¹<https://thalesgroup.github.io/shapkit/>



Moreover, in Chapter 4, we introduce the use of explainable AI for intrusion detection. Based on work of [55], we use a hybrid oracle-explainer approach based on comprehensible decision trees to explain Artificial Neural Network. It works with a large variety of models, datasets and parameters, expanding its use even beyond Intrusion Detection Systems (IDS), if needed. Here, accompanying the first presentation of the prototype, the theory behind it has been presented, followed by the discussion on the design, implementation and the current performance of the system. Chapter 5 is dedicated to Fairness in ML. Two approaches are proposed in the Chapter: one is based on an adversarial network [33], the other on Fair Random Forest [42]. In this Chapter, we discuss about the metrics dedicated to Fairness. We apply them in the adult income dataset.

Chapter 2 Legal issues concerning trustworthy AI

In its communication “Artificial Intelligence for Europe”, the European Commission insists on three fundamentals pillars: being ahead of technological developments, to be prepared for the socio-economic changes brought about by AI and, finally, to ensure an appropriate ethical and legal framework. The notion of fairness comes into play in this third pillar. The Commission has, moreover, repeatedly stressed the importance of adopting a human-centric approach to AI in order to strengthen the confidence of European citizens in artificial intelligence tools. Fairness thus becomes one of the fundamental requirements. Indeed, the European Commission states that: “Data sets used by AI systems (both for training and operation) may suffer from the inclusion of inadvertent historic bias, incompleteness and bad governance models. The continuation of such biases could lead to (in)direct discrimination. Harm can also result from the intentional exploitation of (consumer) biases or by engaging in unfair competition. Moreover, the way in which AI systems are developed (e.g. the way in which the programming code of an algorithm is written) may also suffer from bias. Such concerns should be tackled from the beginning of the system’ development. Establishing diverse design teams and setting up mechanisms ensuring participation, in particular of citizens, in AI development can also help to address these concerns. It is advisable to consult stakeholders who may directly or indirectly be affected by the system throughout its life cycle. AI systems should consider the whole range of human abilities, skills and requirements, and ensure accessibility through a universal design approach to strive to achieve equal access for persons with disabilities”.

In view of the above, the objective of this chapter is twofold. First, we aim at identifying the documents published at international and European level dealing with the issue of artificial intelligence and the ethical deployment of these technologies. Recently, the European High-level Expert Group released a new version of the guidelines for trustworthy AI. Special attention will be given to this new documentation. Second, the essential components of the notion of fairness will be analysed. After explained the methodology and the main findings from the previous deliverable, the first part of this section is devoted to the deepening of the notion of fairness. To this end, we will begin by analysing the recommendations at the international level. Then we will look at European recommendations both for the use of data (personal or not) and for the development of artificial intelligence tools. Bearing in mind that the guidelines for trustworthy AI released by the European High-level Expert Group on AI is the most recent and most accomplished document on the subject, the second part is entirely devoted to these recommendations. Finally, based on international and European recommendations and doctrinal studies, we present the key criteria for understanding the notion of fairness.

2.1 Methodology

The overall goal of the Task 7.4 is the development of a systematic method to prevent machine learning and the decisions based on AI systems from being used to create discrimination. This section aims particularly to continue the analysis of the notion of fairness in artificial intelligence we started in deliverable D7.2. Indeed, one of the main challenges of the deliverable 7.2 was to collect major political statements and scientific papers regarding the notion of fairness in order to provide a basis for evaluating the performance of an AI system. Artificial intelligence and the use of data continued to generate debate, leading to the positioning of international and European bodies. For the analysis of the notion of fairness, we identified and analysed the reports and recommendations released since the previous deliverable.

2.2 Main findings from the deliverable D7.2

Based on the most recent documents existing at the European level, the previous deliverable was an opportunity to analyse the European strategy in front of the increasing development of artificial intelligence.

- European Parliament, “Recommendations to the Commission on Civil Law Rules on Robotics”¹
- EPRS, “EU guidelines on ethics in artificial intelligence: Context and implementation”²

¹https://www.europarl.europa.eu/doceo/document/A-8-2017-0005_EN.html

²[https://www.europarl.europa.eu/thinktank/en/document.html?reference=EPRS_BRI\(2019\)640163](https://www.europarl.europa.eu/thinktank/en/document.html?reference=EPRS_BRI(2019)640163)

- EPRS, “A governance framework for algorithmic accountability and transparency”³
- European Commission, “Artificial Intelligence for Europe”⁴
- European Commission, “Building Trust in Human-Centric Artificial Intelligence”⁵
- High-Level Expert Group on Artificial Intelligence, “A definition of AI: main capabilities and disciplines”⁶
- High-level Expert Group on AI, “Ethics Guidelines for Trustworthy AI”⁷
- High-level Expert Group on AI, “Policy and Investment Recommendations for Trustworthy AI”⁸
- European Consumer Organization, “AI Rights for Consumers”⁹

The European Parliamentary Research Service¹⁰ observes that fairness is a multi-faceted notion: “Fairness reflects the appreciation of a situation based on a set of social values, such as promoting equality in society. The assessment of fairness depends on facts, events, and goals, and therefore has to be understood as situation or task-specific and necessarily addressed within the scope of a practice (. . .). The concept of fairness in the context of algorithmic implementations appears as a balance between the mutual interests, need and values of different stakeholders affected by the algorithmic decisions”¹¹. On the basis of this observation, we then tried to provide initial solutions. See “Sources of unfairness and solutions” and the summary of the guidelines from the independent High-level Expert Group on AI in the D7.2 document. Finally, we have also adopted an approach based on the notion of fairness in the GDPR (e.g. transparency and quality of the information, integrity and confidentiality of the data, misuse of data, data quality and the right to obtain a human intervention and the right to object). A particular attention has been paid to the right not to be subject to an automated-decision.

2.3 Deepening the notion of fairness for artificial intelligence tools: International recommendations

The aim of this section is to, based on the recent reports from international bodies, analyse the proposals to strengthen the reliability and ethical validity of artificial intelligence tools.

2.3.1 Council of Europe

2.3.1.1 Algorithms and Human Rights

In March 2020, the Council of Europe releases a document listing the Council of Europe’s work and recent initiatives on Artificial Intelligence¹². There are 42 links to several studies, reports and recommendations. In this section, we only develop the reports relevant for fairness in AI.

In its report on “Algorithms and Human Rights”¹³, the Council of Europe lists several human rights that might be impacted by the massive development of algorithms and artificial intelligence tools. Mainly, they care about the preservation of rights to:

- A fair trial and due process
- Privacy and data protection
- Freedom of expression
- Freedom of Assembly
- Effective remedy
- Prohibition of discrimination
- Social rights and access to public services

³[https://www.europarl.europa.eu/thinktank/en/document.html?reference=EPRS_STU\(2019\)624262](https://www.europarl.europa.eu/thinktank/en/document.html?reference=EPRS_STU(2019)624262)

⁴<https://ec.europa.eu/digital-single-market/en/news/communication-artificial-intelligence-europe>

⁵Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions - Building Trust in Human Centric Artificial Intelligence (COM(2019)168)

⁶European Commission - A definition of Artificial Intelligence: main capabilities and scientific disciplines - 2019

⁷<https://ec.europa.eu/digital-single-market/en/news/ethics-guidelines-trustworthy-ai>

⁸European Commission - Policy and investment recommendations for trustworthy Artificial Intelligence - 2019

⁹<https://www.beuc.eu/publications/ai-rights-consumers/html>

¹⁰[https://www.europarl.europa.eu/thinktank/en/document.html?reference=EPRS_STU\(2019\)624262](https://www.europarl.europa.eu/thinktank/en/document.html?reference=EPRS_STU(2019)624262)

¹¹EPRS, “A governance framework for algorithmic accountability and transparency”, 2019, p. 10.

¹²CAHAI, “Council of Europe’s work and recent initiatives on artificial intelligence”, CAHAI(2019) INF2

¹³<https://rm.coe.int/algorithms-and-human-rights-en-rev/16807956b5>

- Free elections

Then, the Council of Europe proposes regulatory solutions in order to preserve and guarantee these human rights¹⁴. As it concerns human rights and democratic values, we decide to encompass those solutions into the criteria to ensure a fair development of AI tools:

- **Transparency.** The Council of Europe notes a lack of understanding of how artificial intelligence works and how a certain result is obtained. This lack of understanding leads to the difficulty for citizens to be able to use and interpret the results¹⁵. The Council of Europe calls for effective transparency by informing in particular “which variables are in use, which goals the algorithms are being optimised for, the training data and average values and standard deviations of the results produced, or the amount and type of data being processed by the algorithms”¹⁶. Furthermore, a review of the tool by an independent auditor should be put in place¹⁷. Finally, according to the FATEN principle (see below), information must also be given when a user is in a relation in which he/she interacts with an AI[40]. Interestingly, the doctrine calls for an appropriate balance between the information to be given to users to ensure full understanding of the system and its outcome (with a view to allowing contestation) and to avoid drowning users with information, which would then be unproductive [60].
- **Accountability.** The Council of Europe is aware of the difficulty of determining who has some degree of control over artificial intelligence/algorithms, but nevertheless encourages further reflection¹⁸. The Council of Europe dedicated a special report on “Responsibility and AI”¹⁹ and offers several models for allocating responsibility which are intention/culpability-based models, risk/negligence-based model and strict responsibility.
- **Ethical Frameworks.** The Council of Europe promotes the use of emerging standards such as EI-IIP7000, IEEE7001, IEEE7002 and IEEE7003²⁰.

In addition to fundamental rights issues, interestingly, the authors of [60] map ethical problems that could be raised from the use of algorithms:

- inconclusive evidence that leads to unjustified actions
- inscrutable evidence that leads to opacity
- misguided evidence leading to unwanted bias
- unfair outcomes leading to discrimination
- transformative effects that could impact autonomy and informational privacy and traceability .

For each identified problem, the authors propose several solutions. It is interesting to note that some of them meet the latest recommendations from the High-level Expert Group on AI. The authors raise the issues of fairness and identifies up to 21 different definitions based on the literature [60]. Four main definitions emerge:

- **Anti-classification.** According to the authors, it refers to “protected categories, such as race and gender, and their proxies not being explicitly used in decision making”²¹;
- **Classification parity.** According to the authors, it concerns “a model as being fair if common measures of predictive performance, including false positive and negative rates, are equal across protected groups”²¹;
- **Calibration.** This definition considers fairness as “a measure of how well-calibrated an algorithm is between protection groups”²¹;
- **Statistical parity.** Fairness is defined as “an equal average probability estimates over all members of protected groups”²¹.

In the study “Discrimination, artificial intelligence and algorithmic decision-making”²², the Council of Europe lists several components of AI tools that could lead to discrimination such as the use of some training data, the feature selection, the use of proxies and the discrimination on purpose. Then, the Council of Europe maps fields in which AI brings or could bring discrimination such as police and crime prevention, selection of

¹⁴Council of Europe, “Algorithms and Human Rights”, DGI(2017) 12, p. 34 and following.

¹⁵Council of Europe, “Algorithms and Human Rights”, DGI(2017) 12, pp. 37-38.

¹⁶Council of Europe, “Algorithms and Human Rights”, DGI(2017) 12, p. 38

¹⁷Council of Europe, “Algorithms and Human Rights”, DGI(2017) 12, p. 39.

¹⁸<https://rm.coe.int/algorithms-and-human-rights-en-rev/16807956b5>

¹⁹<https://rm.coe.int/responsability-and-ai-en/168097d9c5>

²⁰Council of Europe, “Algorithms and Human Rights”, DGI(2017) 12, p. 41. Lately, the Committee of Ministers highlights the risks on the “manipulative capabilities of algorithmic processes” in AI tools. In this regard, fairness seems to becoming a bulwark against the risks encountered by data-driven technologies and systems designed to achieve optimum solutions with a mathematical perspective. Unfortunately, the Committee of Ministers does not give more precise criteria to evaluate an artificial intelligence system; Committee of Ministers, “Declaration on the manipulative capabilities of algorithmic processes”, Decl(13/02/2019)1

²¹Tsamados, Andreas and Aggarwal, Nikita and Cows, Josh and Morley, Jessica and Roberts, Huw and Taddeo, Mariarosaria and Floridi, Luciano, The Ethics of Algorithms: Key Problems and Solutions (July 28, 2020), p. 13.

²²<https://rm.coe.int/discrimination-artificial-intelligence-and-algorithmic-decision-making/1680925d73>

employees and students, advertising, price discrimination and image search and analysis.

Then, interestingly, the study reminds the current legal and regulatory safeguards that are part of the solution and are key components of the notion of fairness:

- Non-discrimination law
- Data protection law
- Consumer law
- Competition law
- Freedom of information
- The proposal for the new E-privacy Regulation
- Regulation (EU) 2017/589 of 19 July 2016 supplementing Directive 2014/65/EU of the European Parliament and of the Council with regard to regulatory technical standards specifying the organisational requirements of investment firms engaged in algorithmic trading.

2.3.1.2 Human rights impacts of algorithmic systems

Very recently, the Committee of Ministers from the Council of Europe published a Recommendation on the human rights impacts of algorithmic systems²³. As a fundamental remark, the Committee of Ministers reaffirms the importance in maintaining principles of **legality, transparency, predictability, accountability and human oversight** in algorithmic systems²⁴. Then, the paper is divided in two main sections. One is dedicated to recommendations for governments of Member States and the other one is devoted to the responsibilities of private sector actors. In this deliverable, we will focus on this second part of the Recommendation in which the Committee of Ministers from the Council of Europe recommends:

- **Adequate measures** to effectively respect human rights (in particular direct and indirect discrimination), regardless of the size, sector or ownership structure of the private actors. However, the measures could be graduated according to the resources and the severity of the potential impact that he services and system could have on human rights²⁵.
- **Data management.** First, the Council of Europe highlights the importance for data subject to be informed of their rights accorded by the GDPR. Secondly, the data subject shall be informed of the datasets used, what the impact (real and potential) of the algorithm system, how to object to the processing of their personal data and how to contest and challenge specific outputs. Thirdly, the private sectors should reinforce privacy settings that are in line with the GDPR requirements²⁵. Indeed, the report states that “The possibility of choosing from a set of privacy setting options should be presented in an easily, visible, neutral and intelligible manner, and facilitate the use of privacy enhancing technologies. Default options should lead only to the collection of data that are necessary for, and proportionate to, the specific legitimate purpose of the data processing, while tracking settings should be set as default in opt-out mode. Any application of mechanisms to block, erase or quarantine user data, for example for security purposes, should be accompanied by due process guarantees and rapid remedies in the event of the erroneous or disproportionate use of data”²⁵.
- **Data and model quality.** The private sectors actors should be aware of the quality of their data sets and the origin of the data. Particular attention should be paid to sample that are representative of the entire populations²⁶. Moreover, according to the authors of “The Ethics of Algorithms: Key Problems and Solutions”, the data quality requirement should also cover the timeliness of the data, the completeness and correctness²⁷.
- **Transparency.** The information given to the users or data subjects and their updates has to be clear and in a simple language. Adequate information encompasses the nature and functionality of the algorithmic system, information about available options to change the features of the system, complaint mechanisms, the various stages of the procedure, the exact competencies of the contact point, indicative time frames

²³https://search.coe.int/cm/pages/result_details.aspx?objectid=09000016809e1154

²⁴Council of Europe’s Committee of Ministers, “Recommendation on the human rights impacts of algorithmic system”, CM/Rec(2020) 1, p. 1.

²⁵Council of Europe’s Committee of Ministers, “Recommendation on the human rights impacts of algorithmic system”, CM/Rec(2020) 1, p. 12.

²⁶Council of Europe’s Committee of Ministers, “Recommendation on the human rights impacts of algorithmic system”, CM/Rec(2020) 1, pp. 12-13.

²⁷Tsamados, Andreas and Aggarwal, Nikita and Cowls, Josh and Morley, Jessica and Roberts, Huw and Taddeo, Mariarosaria and Floridi, Luciano, The Ethics of Algorithms: Key Problems and Solutions (July 28, 2020), pp. 5-6.

and expected outcomes²⁸.

- **Contestability.** The Committee of Ministers insists on the importance to have a human in the loop with a direct contact for the users²⁸.
- **Effective remedies.** The Committee of Ministers states that “All complaints should allow for an impartial and independent review, should be handled without unwarranted delays and should be conducted in good faith, with respect for due process guarantees”²⁸.
- **Precautionary measures.** The Committee of Ministers is in favour of a continuous evaluation, not only to detect technical problems but also to make a regular update of the potential ethical, legal and societal impacts²⁹.
- Regular and continued quality assurances **checks and auditing**²⁹.

2.3.2 G20 principles on AI

From its side, the G20 adopted the G20 AI principles³⁰ based on the Recommendation on Artificial intelligence from the OECD³¹. The key principles are:

- Inclusive growth, sustainable development and well-being: “Stakeholders should proactively engage in responsible stewardship of trustworthy AI in pursuit of beneficial outcomes for people and the planet, such as augmenting human capabilities and enhancing creativity, advancing inclusion of underrepresented populations, reducing economic, social, gender and other inequalities, and protecting natural environments, thus invigorating inclusive growth, sustainable development and well-being”³².
- Human-centred values and fairness: “AI actors should respect the rule of law, human rights and democratic values, throughout the AI system lifecycle. These include freedom, dignity and autonomy, privacy and data protection, non-discrimination and equality, diversity, fairness, social justice, and internationally recognized labor rights. To this end, AI actors should implement mechanisms and safeguards, such as capacity for human determination, that are appropriate to the context and consistent with the state of art”³².
- Transparency and explainability: “AI Actors should commit to transparency and responsible disclosure regarding AI systems. To this end, they should provide meaningful information, appropriate to the context, and consistent with the state of art : i. to foster a general understanding of AI systems; ii. to make stakeholders aware of their interactions with AI systems, including in the workplace; iii. to enable those affected by an AI system to understand the outcome; and, iv. to enable those adversely affected by an AI system to challenge its outcome based on plain and easy-to-understand information on the factors, and the logic that served as the basis for the prediction, recommendation or decision”³².
- Robustness, security and safety
 - “AI systems should be robust, secure and safe throughout their entire lifecycle so that, in conditions of normal use, foreseeable use or misuse, or other adverse conditions, they function appropriately and do not pose unreasonable safety risk.”
 - “To this end, AI actors should ensure traceability, including in relation to datasets, processes and decisions made during the AI system lifecycle, to enable analysis of the AI system’s outcomes and responses to inquiry, appropriate to the context and consistent with the state of art.”
 - “AI actors should, based on their roles, the context, and their ability to act, apply a systematic risk management approach to each phase of the AI system lifecycle on a continuous basis to address risks related to AI systems, including privacy, digital security, safety and bias”³³.
- Accountability: “AI actors should be accountable for the proper functioning of AI systems and for the respect of the above principles, based on their roles, the context, and consistent with the state of art”³³.

These recommendations are consistent with the assessment list from the High-Level Expert Group on AI. On the specific topic of fairness, the OECD recommends that AI actors implement mechanisms and safeguards to respect the law, the human rights and democratic values. The objectives of transparency and explainability

²⁸Council of Europe’s Committee of Ministers, “Recommendation on the human rights impacts of algorithmic system”, CM/Rec(2020) 1, p. 13.

²⁹Council of Europe’s Committee of Ministers, “Recommendation on the human rights impacts of algorithmic system”, CM/Rec(2020) 1, p. 14.

³⁰<https://www.g20-insights.org/wp-content/uploads/2019/07/G20-Japan-AI-Principles.pdf>

³¹OECD, Recommendation of the Council on Artificial Intelligence, OECD/LEGAL/0449.

³²G20 AI Principles, 2019, p. 1.

³³G20 AI Principles, 2019, p. 2.

are of utmost importance to enable those affected by an AI to challenge its outcome³⁴.

The Table 1 lists the various elements of the international documents mentioned to building trust in artificial intelligence. Transparency and accountability are common to all recommendations.

Council of Europe	Committee of Ministers	G20
Transparency/Explainability	Transparency/Explainability	Transparency/Explainability
Accountability	Accountability	Accountability
Ethical Framework		Human-centred
	Non-discrimination	
	GDPR framework and privacy settings	
	Security (Data quality, Predictability)	Robustness, safety and security
	Human intervention	
	Regular auditing	

Table 1: Common recommendations at the International level

2.4 European recommendations

Since the appointment of Ursula von der Leyen - who declared that one of her priorities is the human and ethical implications of AI- the European Commission has issued numerous reports on artificial intelligence, its challenges and opportunities for the European Union. The aim of this section is to present these European recommendations.

2.4.1 List of new documents (towards the ones presented in the deliverable D7.2) used in this deliverable

- European Commission, "Shaping Europe's digital future" ³⁵
- European Commission, "White Paper on Artificial Intelligence- A European approach to excellence and trust"³⁶
- Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, "A European strategy for data" ³⁷
- European Commission, "Towards a European strategy on business-to-government data sharing for the public interest" ³⁸
- High-level Expert Group on AI, "Assessment list for Trustworthy AI"³⁹

2.4.2 Beyond data and artificial intelligence: a human-centric approach in the digital strategy for Europe

In its "EU guidelines on ethics in artificial intelligence: Context and implementation"⁴⁰, the European Parliament insists on the fact that the European Union adopts a human-centric approach to AI in order to build trust⁴¹. From this key principle, the European Parliament identified three main requirements, namely that artificial intelligence tools must respect the legal framework of the European Union, be ethical in order to maintain European values, and be technically robust.

³⁴OECD, Recommendation of the Council on Artificial Intelligence, OECD/LEGAL/0449.

³⁵https://ec.europa.eu/info/publications/communication-shaping-europes-digital-future_en

³⁶European Commission - White Paper on Artificial Intelligence: a European approach to excellence and trust - 2020

³⁷<https://ec.europa.eu/digital-single-market/en/policies/building-european-data-economy>

³⁸European Commission - Experts say privately held data available in the European Union should be used better and more - 2020

³⁹European Commission - Assessment List for Trustworthy Artificial Intelligence (ALTAI) for self-assessment - 2020

⁴⁰[https://www.europarl.europa.eu/RegData/etudes/BRIE/2019/640163/EPRS_BRI\(2019\)640163_EN.pdf](https://www.europarl.europa.eu/RegData/etudes/BRIE/2019/640163/EPRS_BRI(2019)640163_EN.pdf).

⁴¹EPRS, "EU guidelines on ethics in artificial intelligence: Context and implementation", 2019, p. 3.

The human-centric approach is defined as follows: “The human-centric approach to AI strives to ensure that human values are central to the way in which AI systems are developed, deployed, used and monitored, by ensuring respect for fundamental rights, including those set out in the Treaties of the European Union and Charter of Fundamental Rights of the European Union, all of which are united by reference to a common foundation rooted in respect for human dignity, in which the human being enjoys a unique and inalienable moral status. This also entails consideration of the natural environment and of other living beings that are part of the human ecosystem, as well as a sustainable approach enabling the flourishing of future generations to come”⁴¹.

Then, the European Commission extended the reflection with its strategic document “Shaping Europe’s digital future”⁴², the European Commission emphasizes certain fundamental principles that must be at the centre of all technological developments. The European Commission considers that, without the maintenance of these principles, there can be no trust in new technologies. These principles are:

- Technology that works for people ⁴³
- A fair and competitive economy ⁴⁴
- An open, democratic and sustainable society⁴⁵. The European Commission reminds us that the European digital society must be and remain “inclusive, fair and accessible to all”⁴⁶. Furthermore, a fair society seems to be intrinsically linked with the transparency and accountability requirements⁴⁷.

In the same vein, in its White Paper on Artificial Intelligence⁴⁸, the European Commission insists on the user confidence and a user-centric approach that must be at the heart of the development of artificial intelligence in the Union⁴⁹. It also insists on the need to further promote transparency, traceability, human control and quality data⁵⁰.

The European Commission paid particular attention to the high-risk AI application. To be considering as high-risk, an artificial intelligence tool that cumulatively presents two criteria⁴⁸:

1. Artificial intelligence is used in sectors of activity likely to be subject to significant risks. (e.g. healthcare; transport; energy and parts of the public sector).
2. Second, the AI application in the sector above is used in a way that significant risks could be expected. According to the European Commission: “This second criterion reflects the acknowledgment that not every use of AI in the selected sectors necessarily involves significant risks. For example, whilst healthcare generally may well be a relevant sector, a flaw in the appointment scheduling system in a hospital will normally not pose risks of such significance as to justify legislative intervention. The assessment of the level of risk of a given use could be based on the impact on the affected parties. For instance, uses of AI applications that produce legal or similarly significant effects for the rights of an individual or a company; that pose risk of injury, death or significant material or immaterial damage; that produce effects that cannot reasonably be avoided by individuals or legal entities”⁵¹.

A examples of high risk AI is surveillance technologies. Regarding a future regulatory framework, the requirements for this type of AI could concerns the training data⁵², data and record-keeping⁵³, transparency⁵⁴, robustness, accuracy⁵⁵ and human oversight⁵⁶. From a proactive perspective, the artificial intelligence sector should focus on its aspects.

⁴²EuropeanCommission, \OT1\textquotedblleftShapingEurope\OT1\textquoterightsdigitalfuture\OT1\textquotedblright, COM(2020) 67final.

⁴³European Commission, “Shaping Europe’s digital future”, COM(2020) 67 final, pp. 3-6.

⁴⁴European Commission, “Shaping Europe’s digital future”, COM(2020) 67 final, pp. 7-10.

⁴⁵European Commission, “Shaping Europe’s digital future”, COM(2020) 67 final, pp. 10-13.

⁴⁶European Commission, “Shaping Europe’s digital future”, COM(2020) 67 final, p. 10.

⁴⁷European Commission, “Shaping Europe’s digital future”, COM(2020) 67 final, p. 11.

⁴⁸European Commission, White Paper On Artificial Intelligence- A European approach to excellence and trust, 19.02.2020; COM(2020) 65 final.

⁴⁹European Commission, White Paper On Artificial Intelligence- A European approach to excellence and trust, 19.02.2020; COM(2020) 65 final, p. 3.

⁵⁰European Commission, White Paper On Artificial Intelligence- A European approach to excellence and trust, 19.02.2020; COM(2020) 65 final, pp. 18 and following.

⁵¹European Commission, White Paper On Artificial Intelligence- A European approach to excellence and trust, 19.02.2020; COM(2020) 65 final, p. 17.

⁵²For the list of requirements concerning the training data, please see White paper, p. 19.

⁵³For the list of requirements concerning the record of data, please see White paper, pp. 19-20.

⁵⁴For the list of requirements concerning the transparency, please see White paper, p. 20.

⁵⁵For the list of requirements concerning the robustness of the system and accuracy of the data, please see White Paper, pp. 20-21.

⁵⁶Concerning the human oversight requirement, please see White paper, p. 21.

2.4.3 Recommendations to frame the use of data

In parallel with the issue of artificial intelligence, the European Commission is continuing its reflections on data and their uses. In 2020, the European Commission published another document focusing on data sharing between the private sector and the public sector⁵⁷. While it is true that the document does not focus on artificial intelligence tools as such, requirements are set in terms of fairness in data sharing, thus enriching the study of the notion of fairness we analyse. Indeed, the European Commission applies the definition of transparency in data sharing given by the World Economic Forum stating that it “involves giving stakeholders meaningful ways to understand relationships, intent and outcomes. To achieve this, individuals need accessible and understandable information on how relationships are structured on how data is being used”⁵⁸. Further, the European Commission reminds us the importance of the FATEN principles to frame both personal and non-personal data⁵⁹. The ethical principles included in the FATEN principles are notably:

- **Fairness.** According to the European Commission “B2G data-sharing frameworks might need to address potential biases in the data, or provide information about such biases to the public-sector bodies”⁵⁹. Moreover, the report revised the principles on B2G data sharing for the public interest elaborated by the European Commission on April 2018. The principle of fair and ethical data use is added. According to it: “Data should be shared and used in an ethical, legitimate, fair and inclusive manner, with full respect for the choices made by individuals on how their data can be used. Public bodies should ensure that data coming from different sources is processed in such a way as to avoid possible biases, including ‘selection bias’”⁶⁰. Even consideration concerns the public sector, it seems of greatest importance for the data used in AI for reinforce the trust and to achieve the ecosystem of trust wanted by the European Commission⁶¹.
- **Accountability and autonomy.**⁵⁹
- **Trust and transparency.** According to the European Commission- which refers to the FATEN principles-, “trust emerges when three conditions are met: (1) competence regarding the specific task that the trust is placed onto; (2) reliability, that is, sustained competence over time; and (3) honesty and transparency”⁵⁹.
- **Education, beneficence and equality**
- **Non-maleficence** to ensure the security, reliability and robustness of the data and the processes⁵⁹.

Reflection on the use of data, both personal and non-personal, has also led to the creation of a European data strategy⁶². After having established an overview of the legislation applicable to the use of data⁶³, the Commission is taking up the challenges to achieve, in the coming years, a true data economy in the European Union. First, we can retain the challenge of data interoperability and quality. Indeed, the European Commission states that “Data interoperability and quality, as well as their structure, authenticity and integrity are key for the exploitation of the data value, especially in the context of AI deployment”⁶⁴.

Second, the European Commission pointed out the need for data infrastructures. The European Commission notes that EU-based cloud providers only have a small share of the global market. However, we believe that this may have an impact in terms of fairness. Indeed, it is not certain that non-EU cloud providers fully comply with data protection legislation, leading to a lack of effectiveness of a fundamental right for European citizens⁶⁵.

Third, the European Commission insist in the possibilities for individuals to exercise their rights, in particular those provided by the GDPR. It therefore appears necessary to always be able to guarantee the respect of fundamental rights when developing data-based tools, such as artificial intelligence tools⁶⁶.

Fourth, the European Commission insists on cybersecurity and points out that: “The new data paradigm where

⁵⁷ European Commission - Experts say privately held data available in the European Union should be used better and more - 2020

⁵⁸ European Commission, “Towards a European strategy on business-to-government data sharing for the public interest”, 2020, p. 49.

⁵⁹ European Commission, “Towards a European strategy on business-to-government data sharing for the public interest”, 2020, p. 57.

⁶⁰ European Commission, “Towards a European strategy on business-to-government data sharing for the public interest”, 2020, p. 86.

⁶¹ European Commission, “White Paper on Artificial Intelligence – A European approach to excellence and trust”, p. 3.

⁶² European Commission, Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, “A European strategy for data”, COM (2020) 66 final

⁶³ General Data Protection Regulation, Regulation on the free flow of non-personal data, the Cybersecurity Act, the Open Data Directive and several sector-specific legislations such as intelligent transport system, smart metering information, automotive. European Commission, Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, “A European strategy for data”, COM (2020) 66 final, p. 8.

⁶⁴ European Commission, Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, “A European strategy for data”, COM (2020) 66 final, p. 8.

⁶⁵ European Commission, Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, “A European strategy for data”, COM (2020) 66 final, p. 9.

⁶⁶ European Commission, Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, “A European strategy for data”, COM (2020) 66 final, p. 10.

less data will be stored in data centres, and more data will be spread in a pervasive way closer to the user ‘at the edge’, brings new challenges for cybersecurity. It will be essential to preserve data security when data are being exchanged. Ensuring the continuity of access control (...) across data value chains will be a key, but demanding, pre-requisite to foster data sharing and ensure trust among different actors of the European data ecosystems⁶⁷.

2.4.4 Recommendations to frame the use of artificial intelligence technologies

In deliverable D7.2, we presented the first report of the High-level Expert Group on Artificial Intelligence, mandated by the European Commission to draft guidelines to increase confidence in the development of artificial intelligence. In this first report, issued in April 2019, the expert group identified seven fundamental criteria that must be taken into consideration in the development of artificial intelligence algorithms. As a reminder, the criteria are human agency and oversight, technical robustness and safety, privacy and data governance, transparency, diversity, non-discrimination and fairness, societal and environmental well-being and, finally, accountability⁶⁸.

In July 2020, after a consultation period, the guidelines were reviewed and updated. As a result, the list of questions that people working in the sector must answer to ensure they meet the various criteria was modified. The Expert Panel calls for a coordinated response that brings together people from different backgrounds: AI designers and developers, data scientists; procurement officers or specialists, front-end staff that will use or work with the AI system, legal/compliance officers and management⁶⁹.

The Table 2 shows the seven main criteria identified by the Expert Group that will enable the development of trustworthy AI. Each of these criteria is divided into sub-criteria in order to refine the analysis.

Human Agency and Oversight	Technical Robustness and Safety	Privacy and Data Governance	Transparency	Diversity, Non-discrimination and Fairness	Societal and Environmental Well-being	Accountability
Human Agency	Resilience to Attack and Security	Privacy	Traceability	Avoidance of Unfair Bias	Environmental Well-being	Auditability
Human Oversight	General Safety	Data Governance	Explainability	Accessibility and Universal Design	Impact on Work and Skills	Risk Management
	Accuracy		Communication	Stakeholder Participation	Impact on society or Democracy	
	Reliability and Reproducibility					

Table 2: Criteria for trustworthy AI by the High-level Expert Group on Artificial Intelligence

For each of the most relevant criteria to the Sparta project, we use the full list of questions developed by the expert group. The ALTAI is also available in a web-based tool version⁷⁰. These guidelines appear to be the most achieved ones at this time. For more information, the assessment list is available here: <https://ec.europa.eu/digital-single-market/en/news/assessment-list-trustworthy-artificial-intelligence-altai-self-assessment>.

⁶⁷ European Commission, Communication from the Commission to the European Parliament, the Council, the European Economic and Social Committee and the Committee of the Regions, “A European strategy for data”, COM (2020) 66 final, p. 11.

⁶⁸ <https://ec.europa.eu/futurium/en/ai-alliance-consultation/guidelines#Top>

⁶⁹ European Commission - Assessment List for Trustworthy Artificial Intelligence (ALTAI) for self-assessment - 2020

⁷⁰ European Commission - ALTAI - The Assessment List on Trustworthy Artificial Intelligence - 2020

2.5 Trustworthy AI and GDPR principles

The Centre for Information Policy Leadership (CIPL) released a very interest paper entitled “Artificial Intelligence and Data Protection: How the GDPR Regulates AI”⁷¹.

First, CIPL listed the provisions of the GDPR that are general and should applied to artificial intelligence systems. Then, some provisions that are particularly relevant for the context of AI are also highlighted. In this matter, we refer to the work done in the deliverable D7.2.

What is particularly interesting and innovative is the table showing the overlap between the ethical guidelines from the High-level Expert Group on Ai and the requirements of the GDPR (see Figure 1).

⁷¹https://www.informationpolicycentre.com/uploads/5/7/1/0/57104281/cipl-hunton_andrews_kurth_legal_note_-_how_gdpr_regulates_ai__12_march_2020_.pdf

Key requirements of Trustworthy AI	Overlap with GDPR provisions
Human Agency and Oversight	- Legitimate interest balancing test (Art. 6(1)(f)) - Transparency (Art. 13 & 14) - ADM (Art. 22) and Right to obtain human intervention (Art. 22(3)) - Risk assessment and DPIA (Art. 35)
Technical Robustness and Safety	- Security (Art. 32) - Risk assessment and DPIA (Art. 35) - Data accuracy (Art. 5(1)(d))
Privacy and Data Governance	- Data protection principles (Art. 5) - Legal grounds for processing (Art. 6) - Legal grounds for sensitive data (Art. 9) - Rights of the data subject (Chapter III) and in particular Transparency (Art. 13 & 14); Right to information on ADM and logic involved (Art. 15(1)(h)); Right not to be subject to an ADM decision (Art. 22) and Right to human intervention (Art. 22(3)) - Accountability (Art. 5(2) & Art. 24(3)) - Data protection by design (Art. 25) - Processor due diligence (Art. 28(1)) - Security (Art. 32) - DPO (Art. 37 & 38)
Transparency	- Transparency (Art. 13 & 14) - ADM (Art. 22)

Key requirements of Trustworthy AI	Overlap with GDPR provisions
Diversity, Non-Discrimination and Fairness	- Fairness data protection principle (Art. 5.1(a)) - Risk assessment and DPIA (Art. 35) - Right to information on ADM and logic involved (Art. 15(1)(h))
Societal and Environmental Wellbeing	- Risk assessment and DPIA (Art. 35) - Transparency (Art. 13 & 14)
Accountability	- Accountability (Art. 5(2) & 24(3)) - Risk assessment and DPIA (Art. 35) - Processor due diligence (Art. 28(1)) - DPO (Art. 37 & 38)

Figure 1: Requirements of the GDPR

2.6 Main characteristics and conclusions

At the international level, the major concerns about artificial intelligence are the requirement of transparency and explainability of the algorithms and the AI mechanism, accountability and a secured AI system. While the European Commission also takes up these three criteria, it adds to them the need to place people at the heart of technological development. This mainly includes compliance with the entire European regulatory framework (in particular compliance with the European framework on the protection of personal data at all stages of AI deployment and during the processing of this information by the machine itself) and safeguarding the fundamental rights of individuals.

In parallel, the European Commission is also focusing on the use of data, putting in place a framework to promote data sharing while protecting personal data against illegal processing.

It seems undeniable that the latest tool made available to stakeholders by the European Commission, namely the ALTAI guidelines, will become increasingly important in the future. In essence, in order to strengthen trust in AI, the European Commission focuses on three main aspects, namely a transparent governance and the respect of personal data, a high level of security and the possibility of human control in artificial intelligence activities. A prototype web-based tool to practically guide developers and deployers of AI through an accessible and dynamic checklist has been developed: <https://futurium.ec.europa.eu/en/european-ai-alliance/pages/altai-assessment-list-trustworthy-artificial-intelligence>. The objective of this approach is twofold. First, the High-level Expert Group on AI and the European Commission want to provide a visualisation of the self-assessed level of adherence of the AI system. Second, some recommendations to enhance the system will be provided. Additionally, having the same basis for evaluating the system also allows for a harmonised approach to the notion of fairness. If the assessment list is quite exhaustive, it seems to us that the notion of fairness comprises two main components:

- The first part includes the quality of the algorithm itself, which mainly allows to avoid bias and the risks of discrimination or marginalization. The criteria around the data and the algorithms used in Artificial Intelligence are: GDPR compliance, Data quality, Security and robustness, Traceability, Reliability, Reproducibility, Predictability and Regular audits.
- The second aspect encompasses everything that surrounds the technical development of artificial intelligence, such as the right to information and the right to a remedy. The criteria that encounters the deployment of artificial intelligence tools are mainly: A Human intervention and a human oversight, Transparency/Explainability, the right to contest the outcome of AI and effective remedies.

It should also be noted that the ALTAI guidelines (although they address the issue of privacy) must be accompanied by a personal data governance policy that complies with all the requirements of the GDPR and its key protection principles. These principles must be respected in the collection of the data necessary for the deployment of the algorithms (e.g. the data minimisation principle prevents the collection of more personal data than what is necessary to fulfil the purpose announced at the outset) and also in the configuration of the system (privacy by design and by default). On the question of GDPR compliance in AI system, we refer to the D7.1 deliverable.

The Figures 2 and 3 below summarize the different criteria to ensure the development of artificial intelligence that respects fundamental values at international and European level. For both figures, we recalled the results of deliverable D7.2 concerning the requirements of the GDPR to have a fair processing of personal data (for more information, we refer to the deliverable D7.2).



Figure 2: Criteria for fairness in data and algorithms used for AI

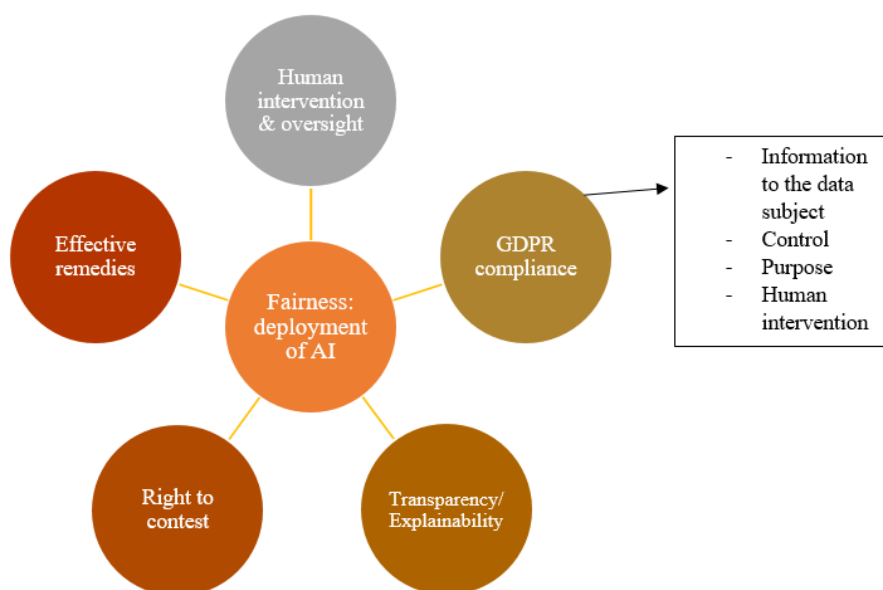


Figure 3: Criteria for fairness in the deployment of AI

Chapter 3 Attack techniques and Defence mechanisms

Machine Learning (ML) models have proved to be successful for many tasks that range from object detection on images to natural language question answering task to even cybersecurity tasks such as intrusion detection on hosts or networks. Unfortunately, as described in deliverable *D7.1 AI systems threat analysis mechanisms and tools* of SPARTA [51], ML models are also vulnerable and subject to some inherent attacks, either during the training process (data poisoning) or after the algorithm deployment (model evasion). In order to fully understand the basics of the concepts and the spectrum of adversarial attacks against ML, we refer the reader to deliverable *D7.1* [51] which provides a comprehensive knowledge base of available attack techniques against ML models and applicable countermeasures.

The current report addresses evasion attacks against ML and above all it studies how to counter them. To assess the robustness of different defence approaches that may be adopted, we have built a demonstrator of attacker-defender game according to four scenarios:

- Cybersecurity scenario: malware detection in PDF files use case
- Evasion Attacks on Graph Anomaly Detection
- Medical scenario: defending healthcare images against evasion attacks
- Cybersecurity scenario: defending AI algorithms used in Network Intrusion Detection against evasion attacks

In the following we explain first in Section 2.1 the evasion attack techniques under study in SAFAIR, then in Section 2.2 the defence mechanisms studied, and in Section 2.3 we describe the experimentation performed in each of the four scenarios thanks to the demonstrator built.

3.1 Evasion attacks

Suppose X is an input sample, and F is a trained ML classification model. The objective of an adversary is to craft a malicious example $X' = (X + \delta)$ by adding a perturbation δ to the sample, so that $F(X')$ is different from the true output. In case of targeted attack, the goal is to get $F(X') = Y'$ where Y' is the target output sought by the attacker. We can therefore formulate the problem as an optimization problem where we look for the perturbation δ that minimizes the following probability:

$$P[F(X') = F(x)].$$

In case of a targeted attack, the perturbation δ is the one that maximizes the probability:

$$P[F(X') = Y'].$$

3.1.1 Fast Gradient Sign Method (FGSM)

The above formulations are quite similar to the optimization of loss functions while training neural networks. The only difference is that we optimize the input X , not the weights and biases of the network. Gradient descent approaches can therefore be easily applied to minimize loss:

$$L(X) = P[F(X') = F(x)].$$

A simple solution to minimize $L(X)$ was introduced by [24]. They proposed a fast gradient sign methodology which calculates the gradient of the loss function with respect to the input of the neural network. The adversarial examples are generated using the following equation:

$$X' = X - \varepsilon \times \text{sign}[\Delta_x L(X)],$$

where Δ_x denotes the gradient of the model with respect to sample X , and ε is a small step.

3.1.2 Basic Iterative method

This is a straightforward extension of the basic FGSM method [30]. This method generates adversarial samples iteratively using small step size.

$$X_{N+1} = X_N - \alpha \times \text{sign}[\Delta_x L(X_N)],$$

where α is the step used to update the input at each step.

3.1.3 Carlini and Wagner approach

Carlini and Wagner work in [12] introduced a reinforced attack by minimizing the loss function $L(X)$ and the perturbation δ at the same time as follows:

$$L'(X) = c \times L(X) + \|\delta\|_p,$$

where $\|\delta\|_p$ is the p norm of the perturbation δ .

3.1.4 Centered Initial Attack (CIA)

As a matter of fact, the adversarial attacks presented previously need a supplementary step of clipping, either at each step or at the end of the gradient descent. Clipping is made to respect two constraints:

- The input X must often be in a constrained to a limited domain, often $[0, 1]$;
- The norm of the perturbation must not exceed a given threshold Δ (strict constraint).

The problem with clipping is the degradation of the attack. The clipped X is likely to not be an adversarial example anymore. To avoid clipping, a method named Centered Initial Attack was developed by [3]. This method respects the above constraints by design. The CIA method introduces a projection function $g \rightarrow [-1, 1]$, which changes the variable X to another r without constraints (a real number) and recalculates the maximum threshold for each component of X , and hence, the maximum threshold becomes elementwise:

$$X' = X_* + \Delta_* \odot g(r),$$

where X_* is the center of the allowed domain of variation of X (the above constraints being taken into account), and operator $\odot$ is the elementwise product.

The loss to minimize with respect to variable r is therefore:

$$L'(r) = F[X_* + \Delta_* \odot g(r)].$$

3.2 Defence mechanisms under study

In this section we describe the defence strategies that will be assessed in SPARTA project as part of WP7 SAFAIR work. The goal is to evaluate the effectiveness of relevant defence mechanisms with regards to all the attacks described in previous section.

3.2.1 Adversarial Train

The primary objective of the adversarial training is to increase model robustness by injecting adversarial examples into the training dataset [56], [24]. This defence retrains targeted model with a new training data, which is generated adding known adversarial examples to the original training data. More information about this defence can be found in the deliverable D7.2 [52]. Moreover, a proof of concept for this defence has been developed in the SPARTA project. The code developed is accessible in [16]. For a more extensive study of this defence, [20] compare this defence with dimensionality reduction (Section 3.2.2) and prediction similarity (Section 3.2.3).

As shown in Figure 4, adversarial training [47] is a standard brute force approach where the defender simply generates a lot of adversarial examples, using one or more crafting strategies, and use them for retraining the target model (whitebox attack) or the proxy model (blackbox or nobox attack). The data augmentation can be

made either by feeding the model with both the original data and the crafted data, or by replacing the original data (or a portion of it) with their adversarial counterpart.

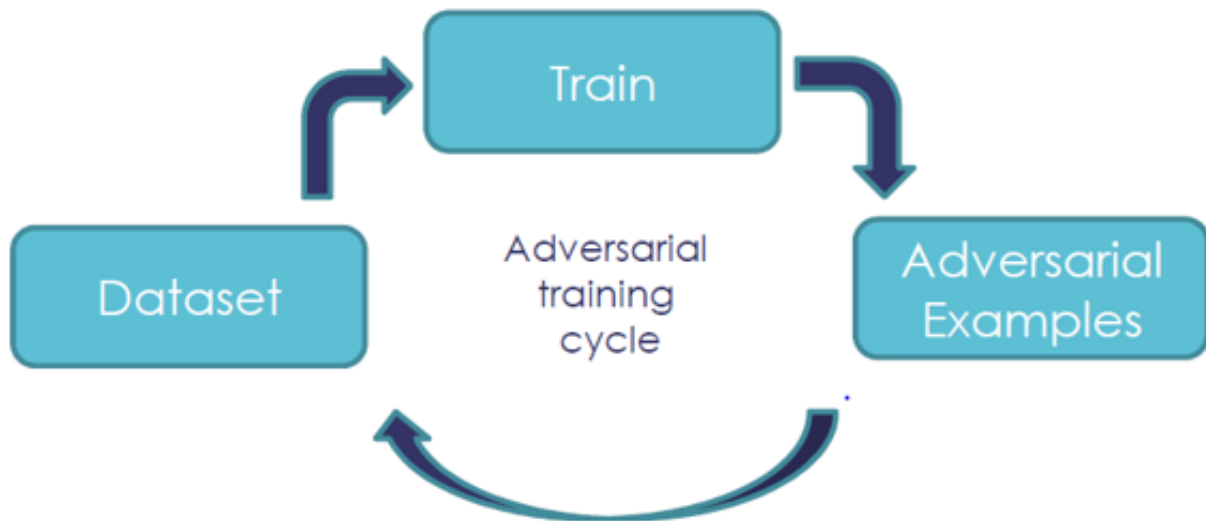


Figure 4: Adversarial training principle

3.2.2 Dimensionality Reduction

This defence method passes data through a dimensionality reduction layer to remove as much noise as possible from the input dataset. Thus, the model is able to generalize, avoiding adversarial examples. This defence has been explained in the deliverable D7.2 [52]. For a more extensive study of this defence, [20] compare this defence with adversarial train (Section 3.2.1) and prediction similarity (Section 3.2.3).

3.2.3 Prediction Similarity

This defence adds an external layer to the model, which saves the history of parameters obtained through the input images, trying to detect the process for obtaining adversarial examples. Therefore, this layer can return an adversarial probability. More details about this defence can be found in the deliverable D7.2 [52]. For a more extensive study of this defence, [20] compare this defence with dimensionality reduction (subsection 3.2.2) and adversarial train (Section 3.2.1).

3.2.4 Model's Behaviour

This defence is an adversarial detector which implements aggregated functions in each layer, obtaining the features of the input that are used in the defence to detect if it is an adversarial examples. This defence is more detailed in deliverable D7.2 [52].

3.2.5 Feature scattering

The authors in [64] present an interesting approach for neural networks robustification. It will be added to deliverable D7.5.

3.2.6 Double-ended monitoring for adversarial detection

The mere detection rate of perturbations is not the only metric that is considered when comparing defenses to adversarial attacks. One also needs to take into account, for example, the complexity of setting up the defense and the degradation that it causes to the accuracy, compared to the undefended model. With this in mind, we are developing the Chimera method (for Common-Headed Input Monitoring to Enhance Robustness to Adversaries) to robustify any network by plugging a second, monitoring, end. Related to both the methods

seen in Sections 3.2.4 and 3.2.1, this two-pronged methodology has the goal of being easy to implement and to retain a maximum of accuracy on the unattacked datasets.

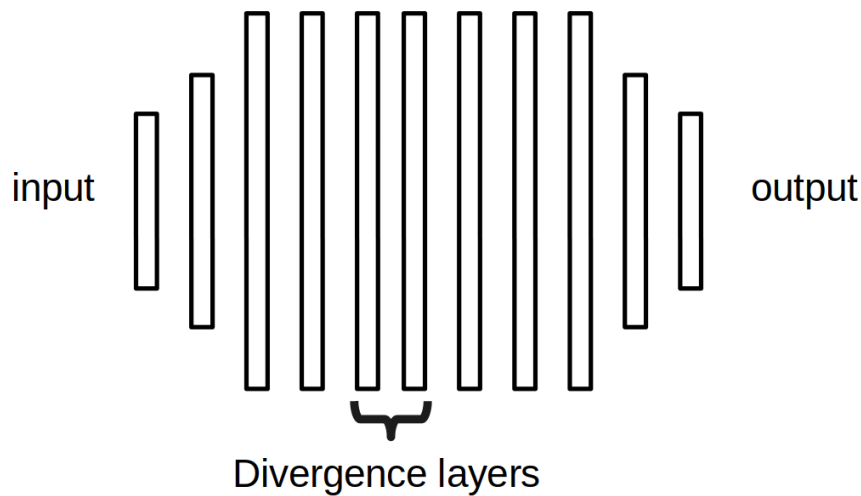


Figure 5: Original network

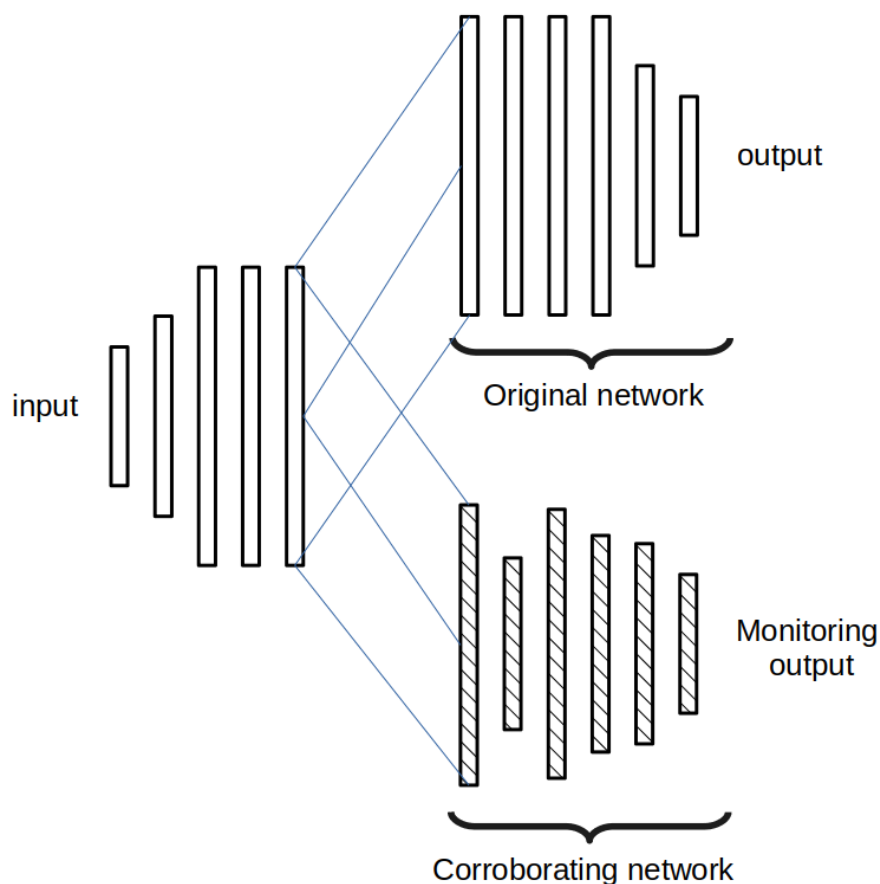


Figure 6: Extension of the original network in Figure 5 with a monitoring second end

The key insight is to rely on a class by class reference behaviour of the network to locate the layers where the class-wise divergence takes place (see Figure 5). Starting after the divergence point, a second half-network is plugged into the first to monitor and corroborate the results (see Figure 6). Our initial experiments show that this two-pronged procedure gives promising results on many datasets, and is not limited to networks of a particular architecture. It is still being investigated and will be the subject of a submission to a future conference.

Our main contributions are: (i) a method for generating a class by class reference behaviour of a network tasked with classification to locate the layers where the class-wise divergence takes place, (ii) a monitoring

expansion that can be plugged to said network and its training to corroborate the classification, (iii) experiments with several attacks on several datasets and comparison to state-of-the-art methods. The next step is of paramount importance when assessing the efficiency of a defense: to design a white-box attack tailored against our defense to show its usefulness and inherent limitations. In our case, we explore attacks that push the divergence towards the entry layers of the network.

3.3 Experimentation

3.3.1 PDF Malware detection

To demonstrate machine learning models vulnerability against evasion attacks, we consider a cybersecurity use case where a ML-based malware detector in PDF files is the target. Subsequently, we show how the robustness of the detector can be improved. It should be noted that most of the results given here are exposed for the first time and not published anywhere. It's a new content compared to D7.2.

3.3.1.1 PDF Malware detection dataset

Billions of Portable Document Format (PDF) files are available on the web. Not all of them are as harmless as one may think. In fact, PDF files may contain various objects, such as JavaScript code or binary code. Sometimes, these objects may be malicious. A malware may try to exploit a flaw in a PDF reader in order to infect the machine [18]. In 2017, sixty-eight vulnerabilities were discovered in Adobe Acrobat Reader¹. More than fifty of them may be exploited to run arbitrary code. Every reader has its own vulnerabilities, and a malicious PDF file may find ways to make use of them. In this context, several works propose the use machine learning to detect malicious PDF files, (e.g. ([27]; [7]; [9]; [59])). These works rely on the same main idea: select discriminating features (i.e. features that are more likely to appear in malicious PDF files) and build what is called a classifier. For a given PDF file, this classifier will take as input the selected features and, considering the number of occurrences of each of them, will try to determine if the PDF file contains malware or not. Many approaches may be considered, both for the choice of the features and for the design of the classifier. In fact there are many classification algorithms that can be used: Naive Bayes, Decision Tree, Random Forest, SVM, etc. [7] first describe their own way to select features, and use a Random Forest algorithm in their classifier, while the author [9] relies on the choice of features proposed by Didier Steven², and uses a SVM (Support Vector Machine) classifier. Both approaches seem to provide accurate results. However, it is still possible to bypass such detection algorithms, as always the case with machine learning. Several evasion attacks have been proposed in literature (e.g. ([5]; [7])). In [7], the authors propose to evade SVM and Neural Network classifiers using gradient descent algorithms.

3.3.1.1.1 Dataset description

A PDF file is composed of objects which are identified by one or several tags. These tags stand for features that characterise the file. There are several tools made to analyze PDF files. In this work we used the PDFiD Python script designed by Didier Stevens. Stevens also selected a list of 21 features that are commonly found in malicious files. For instance the feature /JS indicates that a PDF file contains JavaScript and /OpenAction indicates that an automatic action is to be performed. It is quite suspicious to find these features in a file, and sometimes, it can be a sign of malicious behavior. PDFiD essentially scans through a PDF file, and counts the number of occurrences of each of these 21 features. It can also be used to count the number of occurrences of every features (not only the 21) that characterize a file. Figure 7 shows an output example of PDFiD. For each feature, the corresponding tag is given on the first column, and the number of occurrences on the second one. So, the demonstrator we designed gives the possibility to limit the experimentation to these 21 features or to extend the feature vectors to 221 components as designed in Slayer framework [7].

We trained three types of classifiers (see in following section) with a dataset of 10 000 clean and 10 000 malicious PDF files from the Contagio database³.

¹<https://www.cvedetails.com/product/497/Adobe-Acrobat-Reader.html>

²<https://blog.didierstevens.com/>

³<http://contagiodump.blogspot.fr/2013/03/16800-clean-and-11960-malicious-files>

```
PDFiD 0.2.1 CLEAN_PDF_9000_files/rr-07-58.pdf
PDF Header: %PDF-1.4
obj                23
endobj             23
stream             6
endstream          6
xref               2
trailer            2
startxref          2
/Page              4
/Encrypt           0
/ObjStm            0
/JS                0
/JavaScript         0
/AA                0
/OpenAction        0
/AcroForm          0
/JBIG2Decode       0
/RichMedia         0
/Launch           0
/EmbeddedFile      0
/XFA               0
/Colors > 2^24     0
```

Figure 7: PDFiD output format

To be in a scenario as realistic as possible, we suppose that the attacker dataset is different from the one used by the defender. We also suppose that the adversarial examples were not in the dataset used by the attacker to train the proxy model. So, we split the Contagio dataset into three sub-datasets:

- A dataset used to train the defense model (45%)
- A dataset used to train the proxy model (45%)
- A dataset used to craft adversarial examples (10%)

3.3.1.2 Malware detection algorithms

To assess the malware detection models robustness, even in cross-technique settings, we implemented three different algorithms:

- RBF SVM (Support Vector Machine)
- Random Forest (50 estimators, *max_depth* = 4)
- Neural network (4 layers network 221 32 32 2)

3.3.1.3 Evasion attack modus operandi

As stated above, the feature vector is constituted of the number of occurrences of each tag in the PDF file. The vector is then standardized before feeding the ML model.

Positive Constraint: adversarial examples are crafted by modifying the input, i.e. the features. The problem with PDF files is the fact that we cannot remove tags (decrease a feature). So, the change is to be made only in the positive direction. As consequence, a clipping is applied to all generated examples to respect this constraint (except for Centered Initial Attack that takes into account the constraint by design).

3.3.1.4 Transferability of attacks

All the experiments we carried out were made in no-box setting. In other words, the attacker does not have access to the targeted model (not whitebox) and cannot request it to get feedback (not blackbox). The attacker uses a neural network as proxy to generate adversarial examples. Then, they are sent to the three different target models (SVM, Random Forest, Neural Network). From literature, we know that transferability property holds in such a scenario and we will verify it on the current PDF malware detection use case.

3.3.1.5 Experimental results

3.3.1.5.1 Experimentation platforms

All the code implementing both the PDF malware detection system is written in python while relying upon popular libraries and frameworks like Tensorflow, scikit-learn, etc. All the attacks and defences were implemented from scratch to have more flexibility and to take into account the specific constraints of the considered use case (e.g. positive constraint).

All the experiments were carried out on a computer with a NVIDIA GeoForce RTX 2080 graphical unit and an Intel Core i9 3.0 GHz CPU. Given the relatively small volume of the Contagio dataset ⁴ and the size of the ML models, each run is finished within seconds.

3.3.1.5.2 Results

Without any attack, all the models display a good accuracy between 98% and 99%.

Four attacks (FGSM, Iter-FGSM, CW, and CIA) are implemented to generate the adversarial examples to be added in the adversarial training robustification technique. Eight models are evaluated in the demonstrator: the proxy model (the model used to generate the adversarial examples, a neural network or NN), three non robustified models (NN, SVM, FOREST), their ensemble model (vote), and the the robustified versions of the three models (NN, SVM, FOREST).

To run the demonstrator there are some parameters to provide

- The attack method to use for adversarial examples generation (from set "FGSM", "iter-FGSM", "CW", "CIA")
- The list of attacks to use to evaluate the robustified models robustness. Ex : ["FGSM", "CW"]
- The list of maximum thresholds to use for adversarial examples generation. Ex : [0.1, 1.0, 2.0, 10]
- The list of maximum thresholds to use for adversarial examples generation. Ex : [0.1, 1.0, 2.0, 10]
- The number of iterations of adversarial retraining of models. Ex : 2.

The output of the demonstrator are curves displaying the transferability success rate of the selected attacks against the eight mentioned models. This success rate is given as a function of the logarithm of the maximum perturbations given when running the demonstrator.

⁴<http://contagiodump.blogspot.fr/2013/03/16800-clean-and-11960-malicious-files>

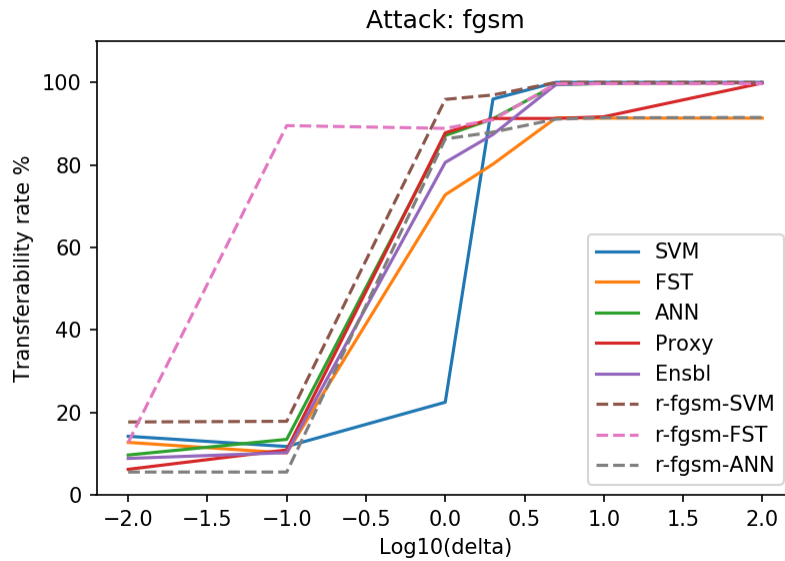


Figure 8: Models performance against FGSM attack (FGSM method is used for adversarial example generation).

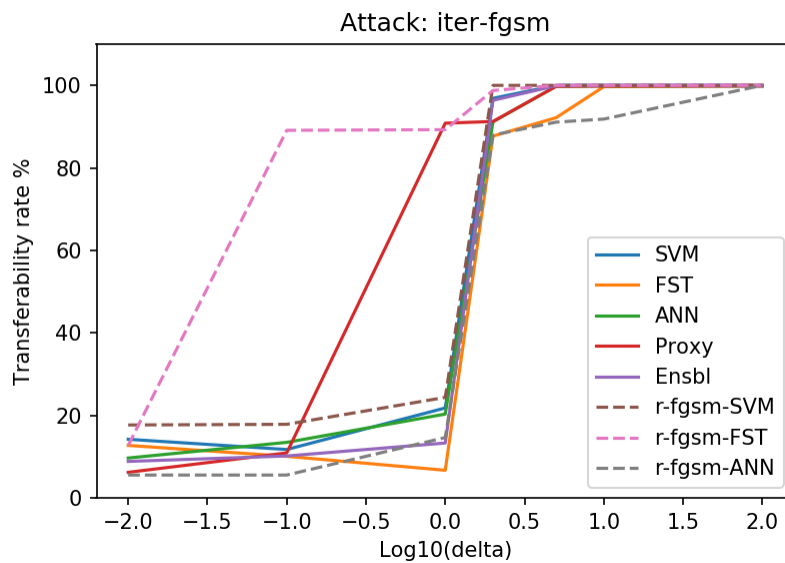


Figure 9: Models performance against iter-FGSM attack (FGSM method is used for adversarial example generation).

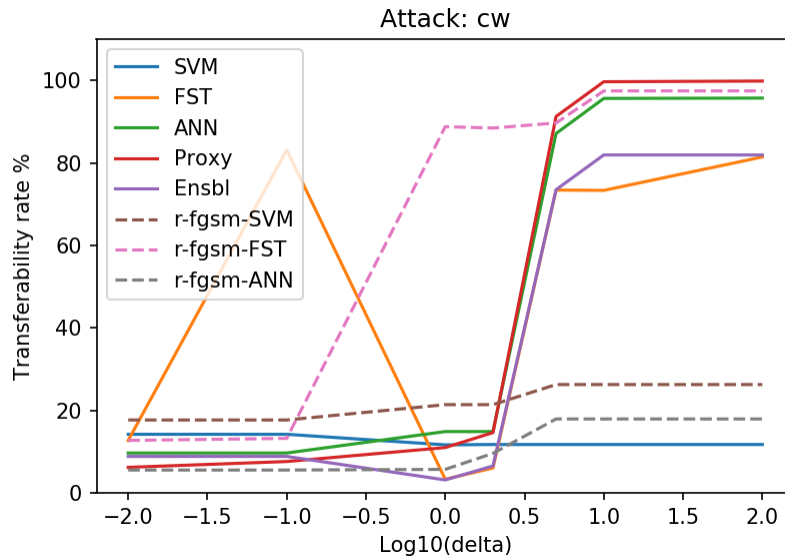


Figure 10: Models performance against CW attack (FGSM method is used for adversarial example generation).

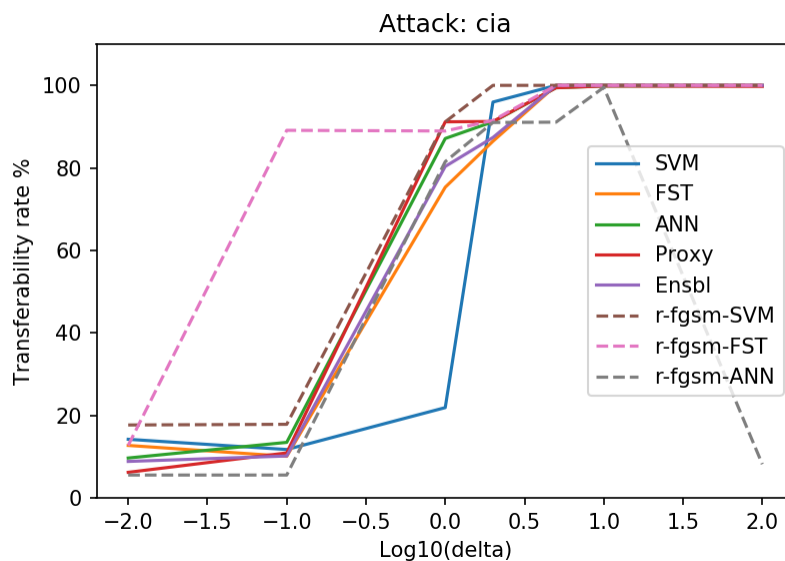


Figure 11: performance against CIA attack (FGSM method is used for adversarial example generation).

The Figures 8, 9, 10 and 11 show more or less expected results:

- The success rate is most of the case growing as the maximum perturbation allowed is bigger.
- In all the cases, the proxy model is the least robust. This is an expected result since the attack is a whitebox one with regard to this model, which is not the case for the other models.
- The success rate depends on the technique used to attack. We can see the FGSM for instance is less successful than CIA but surprisingly better than iter-FGSM or even CW.
- Adversarial training is not a robust method even when the technique of attack is the same as the one used for robustification.
- The ensembling method is better than each non robustified model taken separately but not with regard to the adversarially trained ones in most the cases.
- Some curves are somehow irregular, increasing then decreasing. This could be explained by one fact related to the current use case (PDF Malware). The features are actually positive natural numbers (counters) and therefore we have to round the features accordingly after generating an adversarial example. This process similar to clipping can therefore degrade the attack success rate.

We also ran the demonstrator considering the CIA technique as the attack to generate the adversarial examples. The results are given by Figures 12, 13, 14, 15.

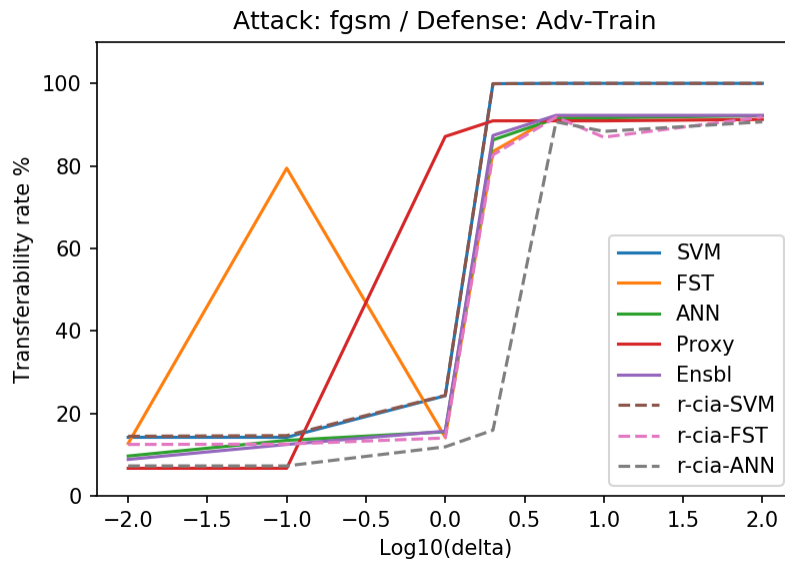


Figure 12: performance against FGSM attack (CIA method is used for adversarial example generation)

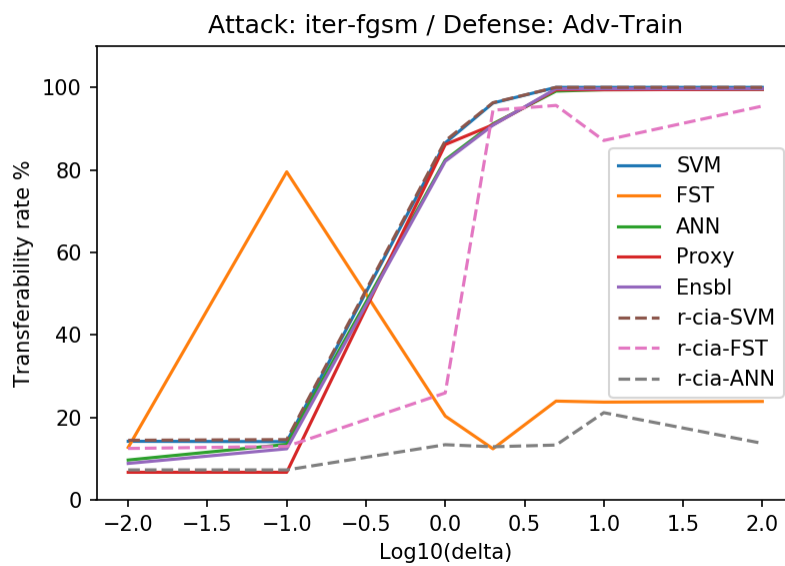


Figure 13: performance against iter-FGSM attack (CIA method is used for adversarial example generation)

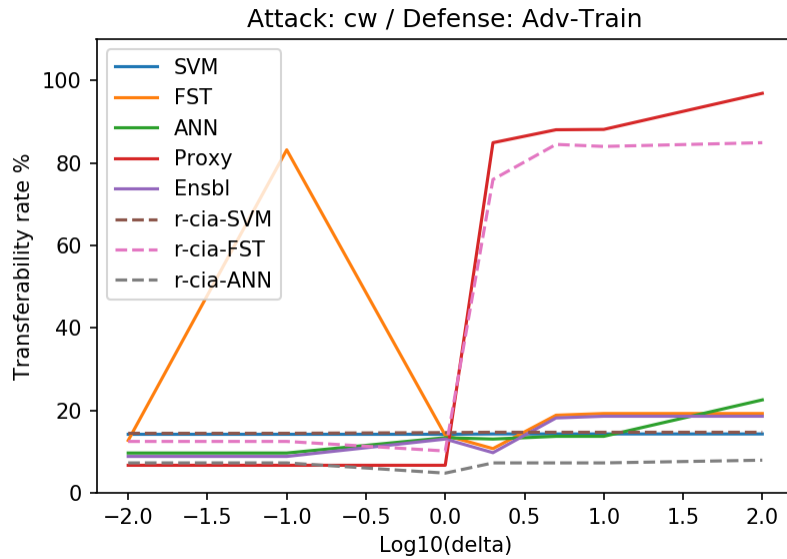


Figure 14: performance against CW attack (CIA method is used for adversarial example generation)

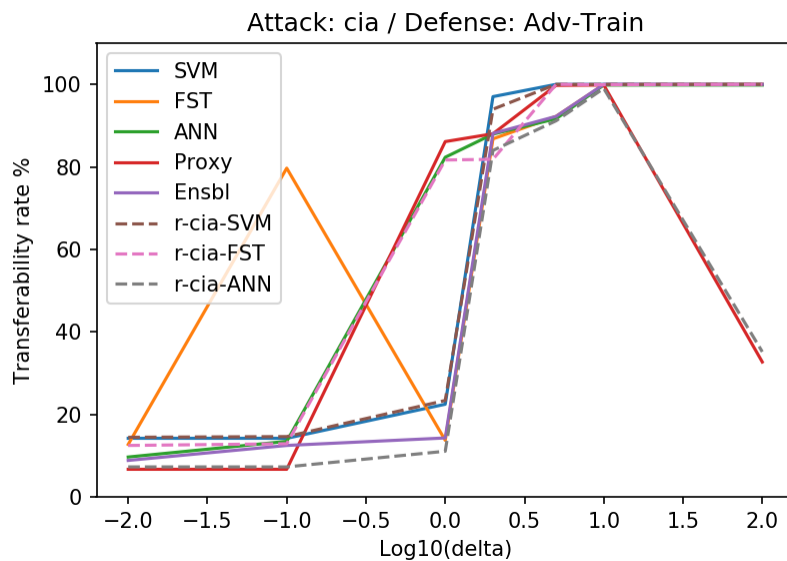


Figure 15: performance against CIA attack (CIA method is used for adversarial example generation)

Some remarks about these new results:

- Robustification using CIA technique to generate the adversarial examples seem to give better results since the transferability rate of attacks is lower.
- When the maximum threshold is enough big, all the methods fail to counter the adversarial examples, especially those generated using CIA that performs better than the other techniques (this is due to the fact that this method does not introduce clipping, except the rounding one that is carried out whatever the used method).

3.3.1.6 Conclusion and future work

Adversarial training is an interesting approach for models robustification given its simplicity and the fact that it does not introduce any overhead on the deployed model. The major contribution of this work is the possibility to assess the approach of PDF Malware use case while the evasion attack is not necessarily the same as the one used for generating the adversarial example of the defence. A 4×4 setting (16 possibilities) can then be considered since we implemented four evasion techniques (FGSM, iter-FGSM, CW, and CIA).

As we can see from the previous experimentations, Adversarial training can be considered for models robustification since the evasion attacks are less successful when it is applied. Nonetheless, one must preferably consider an aggressive technique for generating the adversarial examples. Unfortunately, it remains insufficient when big perturbations can be applied for original data. Other methods must therefore be investigated.

3.3.2 Defending Healthcare Images against Adversarial Evasion Attacks

Deep Neural Networks are progressively being applied and have undergone far-reaching advancements, even getting a spot in healthcare treatments. This is very critical matters, and having a proper risk management is pressing. Some errors in the diagnostics or decision-making in these fields could potentially lead to major incidents that put people's lives at risk.

The adversarial attack has been widely studied in deep learning. Taking advantage of the sensitivity of the models, the attacker adds noise to a specific input sample, modifying the image imperceptibly to change the original output prediction of the sample. Thus, these attacks must be studied to be able to assess their risk and integrate it in risk analysis procedures, and defenses need to be developed to make models more robust against them.

The evolution respect to D7.2 focused on improving the code of each defence, adding some new utilities to the libraries and making the code more manageable. These utilities are detailed in the corresponding defence.

The proof of concept for these defences have been developed in the SPARTA project within WP7 SAFAIR. The code developed to implement the defences is accessible in [16]. The results of the experiments can be found in detail at [20] and in the deliverable D7.2 [52].

3.3.2.1 Healthcare image dataset

The breast cancer images dataset⁵ used is publicly available. This dataset is composed of 277,524 images of size 50X50 of scan patches for Invasive Ductal Carcinoma (IDC) detection, which is the most common subtype of all breast cancer types.

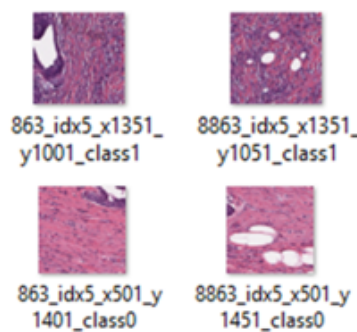


Figure 16: different images that form the breast cancer images dataset

The objective of the model is to classify adequately the images in types non-cancerous (class 0) and cancerous (class 1). This dataset is already divided in three subgroups for model generation: training data, validation data and test data.

3.3.2.2 Neural network model

The model used is formed by a VGG16 (pre-trained CNN) and DNN. It is implemented in Python using keras library. This implementation's code could be found in [16] github repository, specifically in the script called *Cancer_VGG16*.

```
CNN = VGG16(weights='imagenet', include_top=False, input_shape=(50, 50, 3))
```

VGG16 function generates the VGG16 trained convolutional neural network. Moreover, a function that generates a dense neural network has been developed.

⁵<https://www.kaggle.com/paultimothymooney/breast-histopathology-images>

```
def Dense(input_shape, Layer, Dropout):
    """
    This function generates DNN model.
    Parameters:
        - input_shape(int): Dense's input shape.
        - Layer(int): Neuron number of hidden layer.
        - Dropout(float): Dropout layer.
    Output: Generated DNN model.
    """
    # Model
    print('Generating a new model...')
    model = models.Sequential()
    model.add(layers.Dense(
        Layer, activation='relu', kernel_regularizer=regularizers.l2(0.01),
        input_shape=(input_shape, )))
    model.add(layers.Dropout(Dropout))
    model.add(layers.Dense(2, activation='softmax'))

    model.compile(Adam(lr=0.00001), loss='binary_crossentropy', metrics=['accuracy'])
    return model
```

Finally, both convolutional neural network and dense neural network are combined, generating the desired model.

```
def Full_model(CNN, DNN):
    """
    This fuction joins both models CNN and DNN.
    Parameters:
        - CNN: Convolutional neural network.
        - DNN: Dense neural network.
    Output: Generated deep learning model.
    """
    print('Generating full model...')
    Model = models.Sequential()
    Model.add(CNN)
    Model.add(layers.Flatten())
    Model.add(DNN)

    Model.compile(Adam(lr=0.00001), loss='binary_crossentropy', metrics=['accuracy'])
    return Model
```

In Figure 17 the dimensions and types of each layer of the model is shown.

3.3.2.3 Adversarial attack algorithms

As was mentioned in the deliverable D7.2 [52], three adversarial attack algorithms were implemented, using foolbox library in Python. Those implemented adversarial attacks were:

- Fast Gradient Sign Method (FGM).
- Basic Iterative Method (BIM).
- Projected Gradient Descent (PGD).

This implementation can be found in the github repository [16], specifically in the script called *Cancer_VGG16*. With the function shown next, several adversarial examples for each adversarial attack algorithm can be generated. Moreover, it can load or save these adversarial examples for their later use.

```
def Adversarial_examples(Path, Model, X_test, Y_test, Set_size, save_adversarials,
                        load_adversarials):
    """
    Through foolbox library, this fuction generates adversarial examples of
    a test data's image. It uses different attack methods.
    Parameters:
```

```

DNN:
Layer (type)                Output Shape                Param #
=====
dense_1 (Dense)              (None, 256)                 131328
dropout_1 (Dropout)          (None, 256)                 0
dense_2 (Dense)              (None, 2)                   514
=====
Total params: 131,842
Trainable params: 131,842
Non-trainable params: 0

None
CNN:
Layer (type)                Output Shape                Param #
=====
input_1 (InputLayer)         (None, 50, 50, 3)          0
block1_conv1 (Conv2D)         (None, 50, 50, 64)         1792
block1_conv2 (Conv2D)         (None, 50, 50, 64)         36928
block1_pool (MaxPooling2D)    (None, 25, 25, 64)         0
block2_conv1 (Conv2D)         (None, 25, 25, 128)        73856
block2_conv2 (Conv2D)         (None, 25, 25, 128)        147584
block2_pool (MaxPooling2D)    (None, 12, 12, 128)        0
block3_conv1 (Conv2D)         (None, 12, 12, 256)        295168
block3_conv2 (Conv2D)         (None, 12, 12, 256)        590080
block3_conv3 (Conv2D)         (None, 12, 12, 256)        590080
block3_pool (MaxPooling2D)    (None, 6, 6, 256)          0
block4_conv1 (Conv2D)         (None, 6, 6, 512)          1180160
block4_conv2 (Conv2D)         (None, 6, 6, 512)          2359808
block4_conv3 (Conv2D)         (None, 6, 6, 512)          2359808
block4_pool (MaxPooling2D)    (None, 3, 3, 512)          0
block5_conv1 (Conv2D)         (None, 3, 3, 512)          2359808
block5_conv2 (Conv2D)         (None, 3, 3, 512)          2359808
block5_conv3 (Conv2D)         (None, 3, 3, 512)          2359808
block5_pool (MaxPooling2D)    (None, 1, 1, 512)          0
=====
Total params: 14,714,688
Trainable params: 14,714,688
Non-trainable params: 0

```

Figure 17: Architecture of the final model

```

- Path(str): Dataset directory.
- Model: CNN+DNN deep neural network.
- X_test(array): Input data from test data.
- Y_test(array): Labels from test data.
- Set_size(int): Number of adversarials.
- save_adversarials(bool): True to save generated adversarials.
- load_adversarials(bool): True to load adversarials.
Output: Different adversarial example types.
"""

```



```
print('Generating the adversarial examples...')
Set = None
if load_adversarials:

    L1 = np.load(str(Path)+'/Adversarials'+'/L1.npy', allow_pickle = True)
    L2 = np.load(str(Path)+'/Adversarials'+'/L2.npy', allow_pickle = True)
    L3 = np.load(str(Path)+'/Adversarials'+'/L3.npy', allow_pickle = True)
    Set = np.load(str(Path)+'/Adversarials'+'/Positions.npy', allow_pickle = True)
    return Set, L1, L2, L3

backend.set_learning_phase(False)
fmodel = fb.models.KerasModel(Model, bounds=(0, 255), channel_axis=3)

if np.array(Set).any() != None:
    # Attack method: FGSM.
    attack = fb.attacks.FGSM(fmodel, criterion=Misclassification())
    L1 = [np.array(attack(X_test[i], np.argmax(Y_test[i]))) for i in Set]

    # Attack method: BIM.
    attack = fb.attacks.BIM(fmodel, criterion=Misclassification())
    L2 = [np.array(attack(X_test[i], np.argmax(Y_test[i]))) for i in Set]

    # Attack method: ProjectedGradientDescent.
    attack = fb.attacks.ProjectedGradientDescent(fmodel,
                                                  criterion=Misclassification())
    L3 = [np.array(attack(X_test[i], np.argmax(Y_test[i]))) for i in Set]

    return Set, L1, L2, L3
else:

    Set = random.sample(list(range(X_test.shape[0])), Set_size)

    # Attack method: FGSM.

    attack = fb.attacks.FGSM(fmodel, criterion=Misclassification())
    L1 = [np.array(attack(X_test[i], np.argmax(Y_test[i]))) for i in Set]

    # Attack method: BIM.
    attack = fb.attacks.BIM(fmodel, criterion=Misclassification())
    L2 = [np.array(attack(X_test[i], np.argmax(Y_test[i]))) for i in Set]

    # Attack method: ProjectedGradientDescent.
    attack = fb.attacks.ProjectedGradientDescent(fmodel,
                                                  criterion=Misclassification())
    L3 = [np.array(attack(X_test[i], np.argmax(Y_test[i]))) for i in Set]

    if save_adversarials:
        if 'Adversarials' not in os.listdir(Path):
            os.makedirs(str(Path)+'/Adversarials')

        np.save(str(Path)+'/Adversarials'+'/L1', np.array(L1))
        np.save(str(Path)+'/Adversarials'+'/L2', np.array(L2))
        np.save(str(Path)+'/Adversarials'+'/L3', np.array(L3))
        np.save(str(Path)+'/Adversarials'+'/Positions', np.array(Set))

    return Set, L1, L2, L3
```

3.3.2.4 Defence Methods

All the code was implemented in Python using libraries such as keras, pandas and scikit-learn. The defences shown in Sections 3.2.1, 3.2.2, 3.2.3 and 3.2.4 are available in [16] with examples explaining how to use the libraries and the setup needed to run the code.

Adversarial train

The defence retrains the original model with new training data, formed by original training data and adversarial examples. In Figure 18 the schema of the defence is shown.

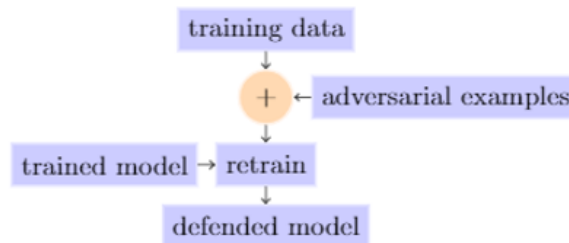


Figure 18: Adversarial training schema.

For that, a first step is to generate the new training data, adding the generated adversarial examples to the original training data.

```

def add_adversarials(X_train, Y_train, X_adversarials, Y_adversarials):
    for j in range(len(X_adversarials)):
        if X_adversarials[j].any() != None:
            X_train=np.append(X_train, X_adversarials[j])
            Y_train=np.append(Y_train, Y_adversarials[j])

    return (X_train, Y_train)

```

Finally, the model is retrained with new training data generated on the previous step, obtaining the defended model.

Dimensionality reduction

In this section, three variants of dimensionality reduction ((1) middle autoencoder, (2) encoder and (3) initial autoencoder) were covered, which were based on the same idea, but the returned defended model was different.

The middle autoencoder variant was obtained by training an autoencoder using CNN features. The CNN features are the outputs obtained when the data goes through CNN (VGG16 in this study). An autoencoder was trained using these features and this trained autoencoder is inserted before the DNN.

```

class AutoencoderDefense():
    def __init__(self, final_part, initial_part = None, dropout = 0.3,
                 difference_threshold = 0.2, distance_threshold = 0.3):
        self.initial = initial_part
        self.final = final_part
        self.model = self.full_model()
        self.dropout = dropout
        self.autoencoder = self.autoencoder_part()
        self.defended_model = None
        self.history = []

```

The function starts by initializing the autoencoder. In this case, it takes the CNN as the *initial_part* and the DNN as the *final_part*. Once the defended model is initialized, it is trained, obtaining the desired *model*.

The initial autoencoder variant trains an autoencoder using the provided dataset and after trained is inserted before the CNN. In this case, the same function is used, but in the inputs provided to it, it takes the full model as the *initial_part* and the *final_part* is taken as *None*. Once the defended model is initialized, it is trained using functions previous functions, obtaining the desired defended *model*.

Finally, the encoder variant is implemented generating a new model with an encoder called *detector*.

```
class EncoderDetector():
    def __init__(self, initial_part, final_part, dropout = 0.4 ,
                  difference_threshold = 0.1, distance_threshold = 0.3,
                  history_size = 20, distance_parameter = False,
                  action = Block(), similarity = EuclideanSimilarity()):
        self.initial = initial_part
        self.final = final_part
        self.model = self.full_model()
        self.dropout = dropout
        self.difference_threshold = difference_threshold
        self.distance_threshold = distance_threshold
        self.autoencoder = self.autoencoder_part()
        self.encoder = self.encoder_part()
        self.detector = None
        self.distance = distance_parameter
        self.action = action
        self.similarity = similarity
        self.history_size = history_size
        self.history = []
        self.detector_accuracy = {}
        self.threshold_history = {}
```

Once the encoder is initialized, it is trained obtaining the *detector*, which is the model formed by the encoder. Thus, this defence consists of the defended model and its possible use as a detector. For using this models as a detector we need both the defended model and the original model, i.e., the *model* and the *detector* are needed.

```
def comparation_predict(self, input_data, n):
    predicts = []
    for sample in input_data:
        predict1 = self.model.predict(np.expand_dims(sample, axis=0))[0]
        predict2 = self.detector.predict(np.expand_dims(sample, axis=0))[0]
        if np.argmax(predict1) == np.argmax(predict2):
            difference = np.abs(predict1[np.argmax(predict1)]
                               -predict2[np.argmax(predict1)])
            if difference >= n:
                if len(predict1) <= 2:
                    predicts.append(True)
                elif self.distance:
                    distance = self.similarity.distance(predict1, predict2)
                    if distance-2*(difference**2) >= self.distance_threshold:
                        predicts.append(True)
                    else:
                        predicts.append(False)
                else:
                    predicts.append(True)
            else:
                predicts.append(False)
        else:
            predicts.append(True)
    return predicts
```

Between the added utilities, save and load options for each defended model and for all the generated adversarial data can be found. Moreover, the detection method of the encoder defence have been implemented and now the encoder part consists of two possible defences: the generation of a defended model or the generation of a detector to use together with the original model. **Prediction similarity**

In this experiment, the pieces of information saved were user, image, prediction value (the class and the probability of this class), minimum distance (to all previous images), prediction alarm (number of times the percentage of the class is smaller) and distance alarm (number of images with distance less than threshold).

```
class Model():
    def __init__(self, model, compared_images = 20, history_size = 50,
                  distance_threshold = 50, total_alarm = 20,
```

```

        distance_alarm_threshold = 50, prediction_alarm_threshold = 50,
        similarity = EuclideanSimilarity(), action = InversePrediction()):
self.model = model
self.compared_images = compared_images
self.history_size = history_size
self.distance_threshold = distance_threshold
self.total_alarm = total_alarm
self.distance_alarm_threshold = distance_alarm_threshold
self.prediction_alarm_threshold = prediction_alarm_threshold
self.similarity = similarity
self.action = action
self.history = []

```

This detector is initialized with certain part of the history of saved data. Different thresholds, similarity functions and the action which will be executed are the inputs of this defence. Once the detector is initialized, the *predict* function compares the input with the other previous images through *record* function and it computes a risk value.

```

def predict(self, images):
    predicts = []
    for image in images:
        predict = self.model.predict(np.expand_dims(image, axis=0))
        prediction = [float(np.amax(predict[0])), float(np.argmax(predict[0]))]
        self.record(image, prediction)
        distance_alarm = self.history[-1]['Distance_alarm']
        prediction_alarm = self.history[-1]['Prediction_alarm']
        if float(distance_alarm[-1]) > self.distance_alarm_threshold:
            if float(prediction_alarm[-1]) > self.prediction_alarm_threshold:
                predicts.append((self.action).take_action(predict, image))
            else:
                predicts.append(predict[0])
        else:
            predicts.append(predict[0])
    return np.array(predicts)

```

Moreover, *record* function computes different parameters and it saves them with the rest data available in history dataset. Through this history, the probability that an image being an adversarial example is obtained.

```

def record(self, image, prediction):
    timestamp = datetime.timestamp(datetime.now())
    if len(self.history)==0:
        (self.history).append({'Date': timestamp, 'User': 'user',
                                'Image': image, 'Prediction': [0,0],
                                'Distance': 0, 'Prediction_alarm': '0%',
                                'Distance_alarm': '0%'})
    else:
        distances = []
        if self.compared_images>=len(self.history):
            for line in self.history:
                saved_image = line['Image']
                distances.append((self.similarity).distance(image, saved_image))
            distance = min(distances)
            distance_alarm = 0
            prediction_alarm = 0
            if distance<self.distance_threshold and distance!=0:
                for line in self.history:
                    if line['Distance']<self.distance_threshold
                        and line['Distance']!=0:
                        distance_alarm = distance_alarm+1
                        saved_prediction = line['Prediction']
                        if saved_prediction[0]>prediction[0]:
                            prediction_alarm = prediction_alarm+1
            else:

```

```

for line in self.history[-self.compared_images:]:
    saved_image = line['Image']
    distances.append((self.similarity).distance(image, saved_image))
distance = min(distances)
distance_alarm = 0
prediction_alarm = 0
if distance < self.distance_threshold and distance != 0:
    for line in self.history[-self.compared_images:]:
        if line['Distance'] < self.distance_threshold
            and line['Distance'] != 0:
            distance_alarm = distance_alarm + 1
            saved_prediction = line['Prediction']
            if saved_prediction[0] > prediction[0]:
                prediction_alarm = prediction_alarm + 1
distance_alarm = (distance_alarm / self.total_alarm) * 100
prediction_alarm = (prediction_alarm / self.total_alarm) * 100
(self.history).append({'Date': timestamp, 'User': 'user', 'Image': image,
                        'Prediction': prediction, 'Distance': float(distance),
                        'Prediction_alarm': str(float(prediction_alarm)) + '%',
                        'Distance_alarm': str(float(distance_alarm)) + '%'})

if len(self.history) >= self.history_size:
    self.history.remove(self.history[0])

```

For this defence, more possible image similarity metrics have been added. Furthermore, different final actions implementations are available, with them once the adversarial attack has been detected the model manager can decide to use actions such as block the user or send a random results.

Model's behaviour

In Figure 19 the schema used for creating the defence is shown. First, from adversarial and original images features used by the detector are generated. Then, those features are used to generate the detector that will be used as defence.

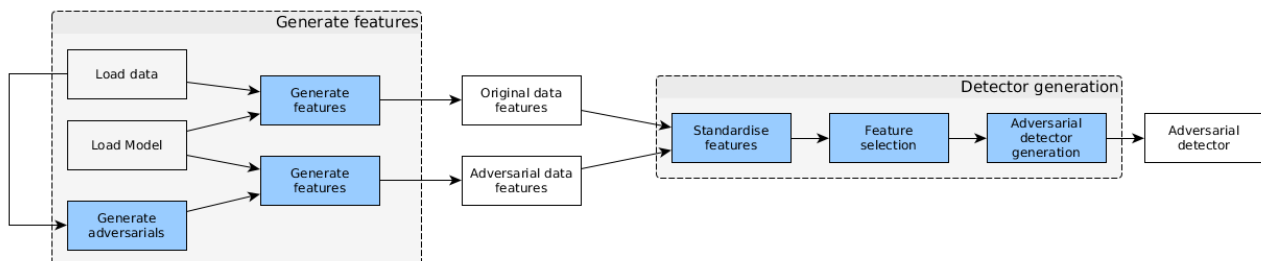


Figure 19: Model's behaviour schema

The features are generated by the use of statistical aggregation functions such as mean, standard deviation and skewness of activation values of all the neurons in each layer of the model. In Figure 20 some of the features for a sample of data inputs are shown.

layer_0_coul	layer_0_mel	layer_0_max	layer_0_min	layer_0_std	layer_0_ske	layer_0_kurt	layer_0_mec	layer_0_iqr	layer_0_rang	layer_1_coul	layer_1_mel	layer_1_max	layer_1_min	layer_1_std	layer_1_ske	layer_1_kurt	layer_1_mec	layer_1_iqr	layer_1_rang
77503	97.2308749	716.986938	0.0001086	90.6943207	2.12072659	5.72714878	81.8779221	91.2111855	716.986816	106535	360.395325	2210.58643	0.0004465	423.951385	1.96096027	3.42			
74917	96.1568669	885.956604	0.00085545	127.751717	3.12272549	10.6433242	59.7298775	90.9158306	885.95575	106671	381.017671	2881.89526	0.00978485	404.70636	1.71164179	2.73			
77377	93.8239852	864.023071	0.00305974	102.400597	2.78509259	9.16216155	71.1604691	90.281126	864.02002	105493	363.948508	2164.62646	0.00453904	411.496124	1.74650776	2.4			
75048	83.0961851	916.10083	0.00055993	135.039261	3.27283812	11.2663857	37.981144	78.8671694	916.100281	110020	341.280604	2719.82593	0.0090049	386.96347	1.78244591	3.02			
76928	88.3778338	838.629272	0.002893	98.2779617	2.77133846	9.0851577	62.7601166	84.3248444	838.626404	101280	361.717852	2280.48389	0.00376248	357.214874	1.4977808	1.94			
76782	78.1170066	901.260254	0.00023431	132.357651	3.41761732	11.9003712	35.9693985	62.4771793	901.26001	110003	325.820314	2516.50098	0.00023084	387.134308	1.84368753	3.14			
77620	90.3238212	792.169495	0.0011047	102.908684	3.03461814	10.2264159	68.0726929	79.6731911	792.168396	104474	356.430882	2277.49487	0.00159007	389.598389	1.68680239	2.25			
77888	88.2584673	880.001953	4.89E-05	106.02095	3.07272625	11.6136573	63.4660759	89.9363089	880.001892	105716	351.768247	2663.5542	0.00047034	382.034485	1.67910659	2.44			
77529	94.7827716	893.272766	0.00198942	105.860298	2.88535166	9.82917139	70.051384	87.7678127	893.270752	103253	381.212401	2154.96655	0.00971654	395.533386	1.55708349	1.89			
76910	77.9724483	578.811218	0.0003348	85.2908325	2.75128078	8.95086492	57.1078529	76.5254545	578.810913	103951	309.813778	1756.45142	0.00418454	335.988739	1.74993038	2.74			
77326	87.3974472	740.50769	0.0002901	90.7445145	2.63861465	8.86967669	64.885643	81.456183	740.507385	103221	343.859137	2425.89087	0.00386435	357.095947	1.64680457	2.40			
77652	95.5477837	871.191895	0.00171745	97.5447235	2.70445943	8.869894	77.9139099	88.0032959	871.190186	106101	361.134542	2420.9834	0.00143247	408.423523	1.83084273	2.81			
76446	78.469763	713.588074	0.00073197	88.7469711	2.78735638	8.8236816	53.2930145	72.6093569	713.587341	103756	315.285921	2359.58618	0.00016338	335.381012	1.6519388	2.22			
74922	86.6513174	935.02002	0.00031611	134.769272	3.27249861	11.2408567	44.2401581	79.6545644	935.019714	109182	350.780825	2792.93677	0.00663486	397.211548	1.76951373	2.84			
78001	91.6618569	747.592163	0.00015211	107.848595	3.02330327	10.1320727	67.7937698	82.6723804	747.592041	105379	360.241225	2476.85034	0.00741234	409.001892	1.76961243	2.58			
77743	86.4054899	712.727722	0.00015026	101.499527	3.02935314	10.2225631	65.3152161	80.7134895	712.7276	105742	337.889769	2099.59302	0.00433387	389.407196	1.77859604	2.54			
77485	83.2698974	746.256348	0.00138429	103.479004	3.08956861	10.4473752	55.2763138	72.5110798	746.254944	101339	349.243115	2192.57642	0.00532697	344.336426	1.42729175	1.64			
77140	81.1713897	700.879333	0.00404292	87.4683762	2.62713051	8.19732106	56.9584427	78.5545955	700.875305	101330	332.045278	2008.79053	0.00704628	324.790894	1.44484949	1.62			
77401	81.0929122	892.504211	0.00103123	108.425957	3.1328516	10.9244788	50.1290703	73.8612099	892.503174	103886	337.217026	2403.10107	0.00436436	353.571075	1.56285894	2.10			
77132	83.2881683	867.11731	0.00020516	109.239708	3.24406934	11.5535448	54.6288757	71.3618598	867.117126	104341	338.822055	2784.64429	0.00309016	360.27066	1.6838944	2.70			
76064	80.8303008	677.736511	0.00325096	89.9523773	2.86574197	9.73069823	59.3775635	79.2874107	677.733276	104094	319.110323	1929.21265	0.00123339	346.157562	1.79982483	3.06			

Figure 20: Features examples

Due to different scales between the features, before starting to train a detector, it is necessary to standardise the features. As the number of features is high (190), feature selection algorithms are applied first. Once the most meaningful features are selected for the problem, a detector is generated. Finally, the detector obtained is validated by cross-validation.

With the last code updates, the user is able to select different machine learning models to generate the detectors as the feature generator from the original model and the detector generator tools are independent. Multiple feature selection algorithms have been added to help the user generate detectors in an easy way.

3.3.2.5 Experimental results

There are two essential characteristics of each defended model to take in account: the classification accuracy and the success avoiding adversarial examples (robustness). A balance between them is necessary, because a machine learning model must be robust without losing applicability. We have studied the accuracy of each of the defended models compared to the original model as well as the robustness of these models. As expected, in general defended models accuracy for the original data has decreased compared with original model's classification accuracy. Moreover, we have obtained result of each defense through two types of adversarial examples: the original model's adversarial examples (known adversarials) and the defended model's adversarial examples (new adversarials). All the results for each defence and for the three type of data used (original data, known adversarials and new adversarials) are shown in Figure 21.

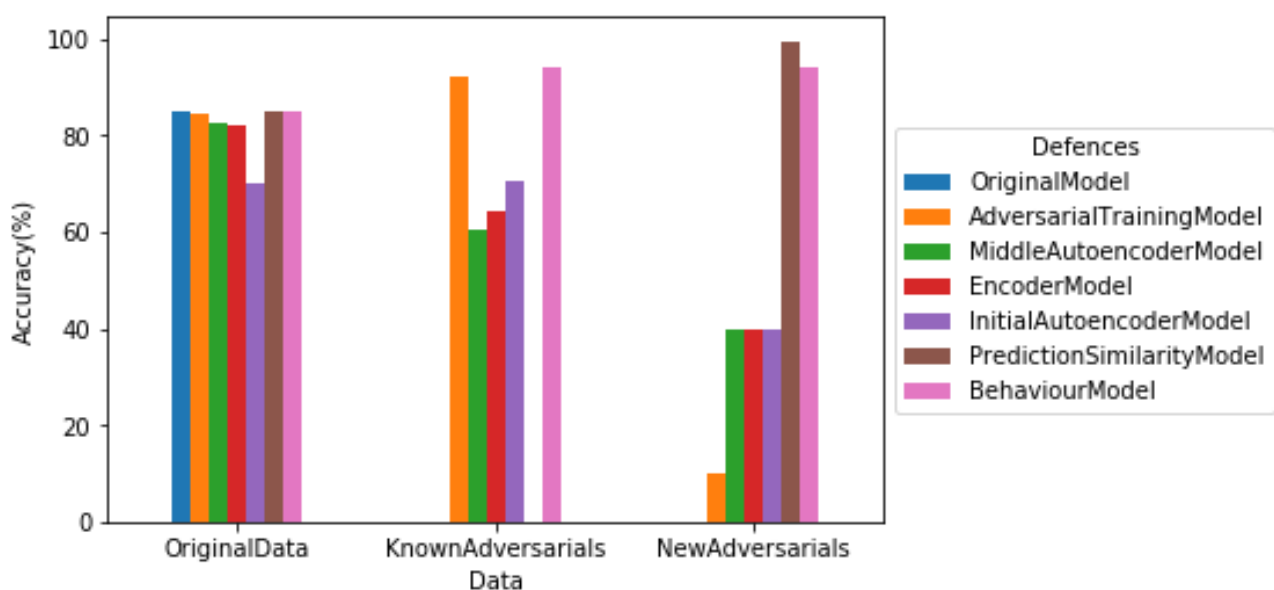


Figure 21: Classification accuracy for original images, known adversarials data and new adversarials

These values are representative of the robustness level of the defended models. Both robustness results are explained next.

Known adversarial examples

All four defenses have been tried on this case. Each defense was tested to calculate how many known adversarial examples are no longer misclassified. Adversarial training and behaviour model are the best options compared to the rest in order to avoid known adversarial examples. In the adversarial training case, the model is retrained with the known adversarial examples directly, ensuring a good classification of those images. For the behaviour model case, as is an external layer of the original model, the features used by the detector are generated directly from Known images, helping the model to detect them. Dimensionality reduction defended models versions are capable to classify certain amount of them (between 60% and 70%). However, the prediction similarity does not (0% value is shown in Figure 21), since the process of generating known adversarials has already been carried out and there is no history available needed by the defense. This defence, it merely detects when an adversarial attack attempt is happening, which is useful as a parameter for risk assessment, but does not help if the attack already has been generated.

New adversarial examples

Once the defences had been tested with known adversarial examples, new ones were generated to attack the defended model. The similarity of the predictions defence is the one that detects the greatest number of generation processes of these new adversarials (99.5% detection success). The difficulty of this defense is selecting the adequate parameters and thresholds, as these values change are dependent of the dataset, the adversarial attack generation algorithms and the chosen metric to compare images. In our case, it has been implemented with the parameters shown in [52] and the structural similarity index measure (SSIM) is used as metric.

Finally, the defence of the model's behaviour is more regular, in the sense of accuracy with respect to both types of adversarial, among the defences studied.

3.3.3 Evasion Attacks on Graph Anomaly Detection

Anomaly detection is an established method where patterns in data that indicate unusual behavior compared to the usual functioning of a system are detected. Unsupervised machine learning approaches are appropriate and often used for detecting anomalies in data, as they help us to capture the patterns of normal functionality during the training process. The data that deviate from these patterns are detected as anomalies. In graph data, the anomalies are manifested in unusual connectivity structure in the graph. We consider anomaly detection at the level of nodes, where anomalous nodes are the ones that do not comply with the connectivity patterns in the graph. Detection of anomalous nodes is used in multiple applications, such as financial fraud detection or detection of attacks in computer security. In security-related scenarios, one additional problem for graph anomaly detection is that the attackers could be able to change the connectivity structure of nodes in order to circumvent anomaly-based detectors. We investigated such possibilities and executed different types of evasion attacks on benchmark dataset. The results showed that it is possible to make small changes in the connectivity data that result in high changes in the node anomaly score. With this, we showed vulnerabilities of graph anomaly detection methods and demonstrated their severity which was published in D7.2. The goal of this work was to test the robustness of methods for graph anomaly detection in the face of adversarial attacks with the goal of evading detection.

Additionally, a modular and extensible framework for attacking graph anomaly detection systems was created to allow for easier implementation and comparison of different attacks and different detectors. We were able to successfully evade detection for several nodes initially flagged as anomalies and significantly lower the anomaly score for others, making detection harder. We tried three main different attacks (Random, Heuristic, and Gradient Heuristic) and all the attacks were able to successfully decrease the anomaly score in only a few steps to evade detection. In the following sections, we are going to describe the system architecture and implementation.

3.3.3.1 System Description

In order to implement the attacks and models, we introduce the following system. This is aimed to be a framework for attacking graph anomaly detection.

It provides a way to both run detection and attacks on given models, attacks, and dataset. The key idea here is extensibility. Each model instance is provided with an instance of a dataset (which contains a graph) to run detection on. Then, the detection is run iteratively, and optionally each step is saved as a checkpoint. The final

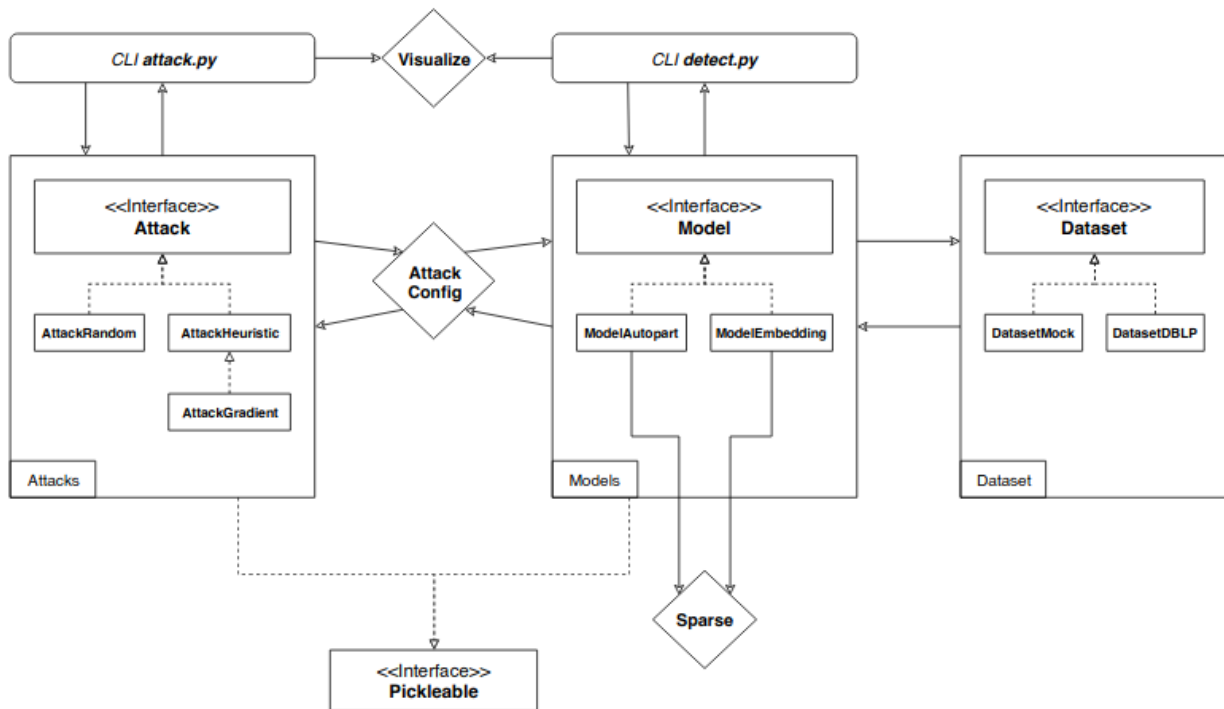


Figure 22: Attacking Graph Anomaly Detection System Description

results are then evaluated in order to identify anomalous nodes within the graph.

For attacks, each attack instance is provided with a model instance and the corresponding dataset. The model is then run for a specified number of iterations, according to the process specified above. Afterward, the attack modifies the underlying graph by inserting an edge with the goal of minimizing the likelihood of detection. At this point, it might also remove the edge inserted in the previous step, should it not have decreased the anomaly score. The next edge is chosen based on the information derived from the model iterations such as loss, gradient, etc.

Then, the model iteration is run again. This process is repeated until the likelihood of detection is sufficiently low. Optionally, each step is saved as a checkpoint. Finally, the difference in the anomaly score distributions is computed and displayed, as well as the change in the underlying graph structure.

Checkpoint saving for models and attacks is implemented through the use of the *Pickleable* interface. This interface provides functionality for storing and loading dictionaries using the Python serialization framework pickle. Both detection models rely on the external *Sparse* component for implementing their desired behavior. This component provides two classes, *SparseColIndexer* and *SparseCol*. The first maintains a numpy array of the latter. *SparseColIndexer* can be used as a fast sparse matrix implementation for slicing arrays into direct references, instead of newly created objects. The detailed implementation will be discussed later on.

3.3.3.2 Implementation

Our system provides an implementation of both introduced models using different dataset as well as a mechanism for attacking those models. This mechanism provides access to different scopes of information about the model, based on the chosen attack type (*whitebox*, *graybox*, *blackbox*). All previously introduced attacks have been implemented. The entire software model is modular and extensible, providing room for additional attacks, models, and dataset. Both models and attacks provide automatic checkpoint support in order to avoid data loss when the scripts are interrupted and supports longer experiments. The framework is implemented using *Python3*.

In order to enable quick and easy user interactions, a command-line interface (CLI) is provided for both detection and attack. Beyond that, a visualization class provides a clear and informative overview of the distribution of anomaly scores in the dataset based on the model result. This can also be used to display the

change in anomaly score for a given node after running an attack.

Furthermore, the initial setup is simplified through a setup script, which installs all necessary libraries and dataset and sets all configurations. Alternatively, a docker script is provided which will install all necessary requirements inside of a docker container and launch it. This enables cross-platform support and avoids overwriting any existing configurations and setups.

Dataset

This framework supplies multiple dataset, which can be specified when running attacks or detection. The provided dataset are:

- mock0-mock4 are small mock dataset used for testing the proper functionality of detection and attack algorithms. The first three examples are handwritten, while the latter two have been taken from the AutoPart [13] paper implementation.
- dblp⁶ refers to the dblp dataset provided by the University of Trier, which contains a real-world citation graph based on major computer science journals and proceedings.

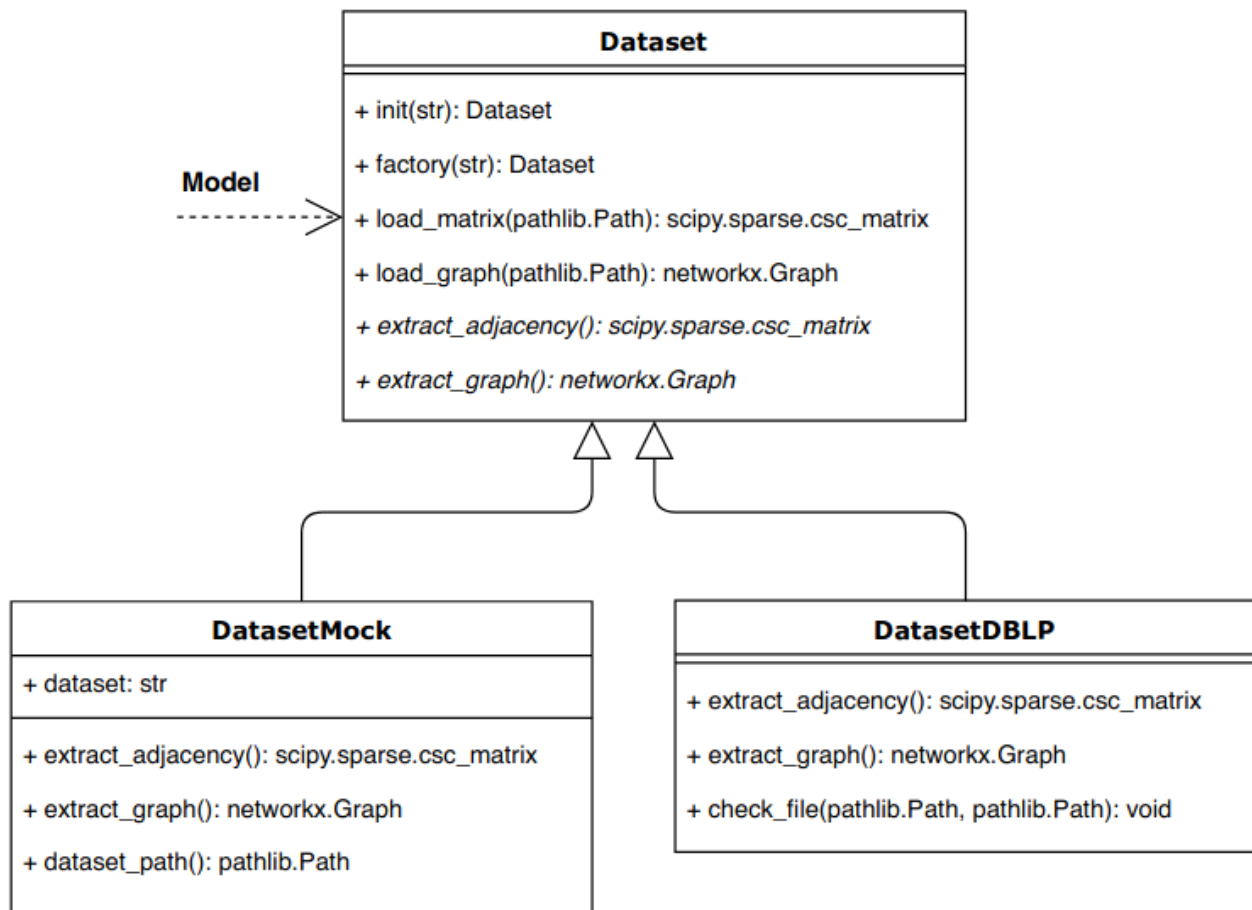


Figure 23: Dataset Class Structure

Interface

Dataset support is enabled through a class structured displayed in Figure 23. Each dataset is wrapped in a similarly named subclass of the abstract *Dataset* base class. Access to the dataset is provided in two ways:

- *extract_adjacency* takes no arguments and returns a the adjacency matrix of the dataset as a sparse column-sliced matrix of type *scipy.sparse.csc_matrix*.
- *extract_graph* takes no arguments and returns the full graph of the dataset in the form of a *networkx.Graph*.

⁶<https://snap.stanford.edu/data/com-DBLP.html>

Implementation

Both previously introduced abstract methods are implemented by all subclasses of `Dataset`. However, the core functionality of extracting the adjacency matrix or graph from a `Dataset` is abstracted in the parent class `dataset` through the methods `load_matrix` and `load_graph`. They both take a `pathlib.Path` and extract the required information from the dataset stored at the supplied path.

All graph datasets are stored as `.txt` files, with each line containing two natural numbers i and j separated by a space. This represents an edge between nodes i and j in the graph. The numbers start at 1.

`DatasetMock` implements both methods by calling `load_matrix` and `load_graph` with the corresponding `pathlib.Path` of the dataset. This path is determined by calling its internal `dataset_path` method, which constructs the path based on `self.dataset`. This variable of type `str` indicates the name of the dataset to be used and is specified when `DatasetMock` is constructed. `DatasetDBLP` also relies on `load_matrix` and `load_graph` in order to implement its abstract parent methods. However, it first uses its internal `check_file` method in order to see if the dataset file already exists in the right format. If it does, the file is passed directly to the methods and the information is extracted. If the correct file does not exist yet but the base file for `dblp` does, that file is parsed for the relevant information and the correct file is created. If no file exists, an exception is thrown.

The `load_matrix` function utilizes the `numpy.loadtxt` function to load the list of edges from a dataset file into a matrix of dimension $2 \times m$. Then, it modifies the node naming from $[1, \dots, n]$ to $[0, \dots, n - 1]$. This simplifies a lot of later operations since Python indexing starts at 0. The matrix is then split into two columns of length m . These can be thought of as the from and to columns. Finally, these columns are passed into the `scipy.sparse.csc_matrix` constructor and the resulting sparse matrix is returned. The `load_graph` function starts by creating a new `networkx.Graph` instance. Then, it opens the supplied file and iterates over all lines. In each line, it extracts both numbers i and j and subtracts 1 to change the node naming to $[0, \dots, n - 1]$. Then it inserts (i, j) into the Graph by calling `add_edge(int, int)` on the instance of `networkx.Graph`. Finally, the `networkx.Graph` is returned.

The functions `extract_adjacency` and `extract_graph` are called directly by the model or attack once the graph data is needed. They are called on the `Dataset` object reference passed to each model or attack object.

Models

Models are the central pieces of this framework for attacking graph anomaly detection and interact with various pieces in the overall system, such as dataset, attack configurations, and provided components like the Sparse module. Both the general model class structure in relation to the whole framework as well as the detailed implementation will be discussed in the following.

Interface

The required model functionality is achieved through the class architecture showcased in Figure 24. Each model is implemented in its own class as a subclass of the abstract `Model` base class. This base class specifies several functions to be implemented and directly provides some other functionality. Each `Model` subclass needs to implement the functions `detect` and `detect_iter` which are called by the `detect.py` CLI and the attacks respectively. Beyond that, the required `get_threshold` returns the default threshold for anomaly detection for each model. The base class itself extends the `Pickleable` interface in order to enable checkpoint saving and loading. Models are created through the use of `ModelFactory`. This factory provides one method, `init`. That method takes a string and multiple pass-through parameters as input. The string determines which model is created based on a lookup table. The corresponding model is then initialized with the pass-through parameters and the constructed object is returned.

Parameterization

Each model is configurable through the use of multiple parameters that are set through the CLI. While the specific CLI options will be introduced later, the modifiable parameters are introduced explained here.

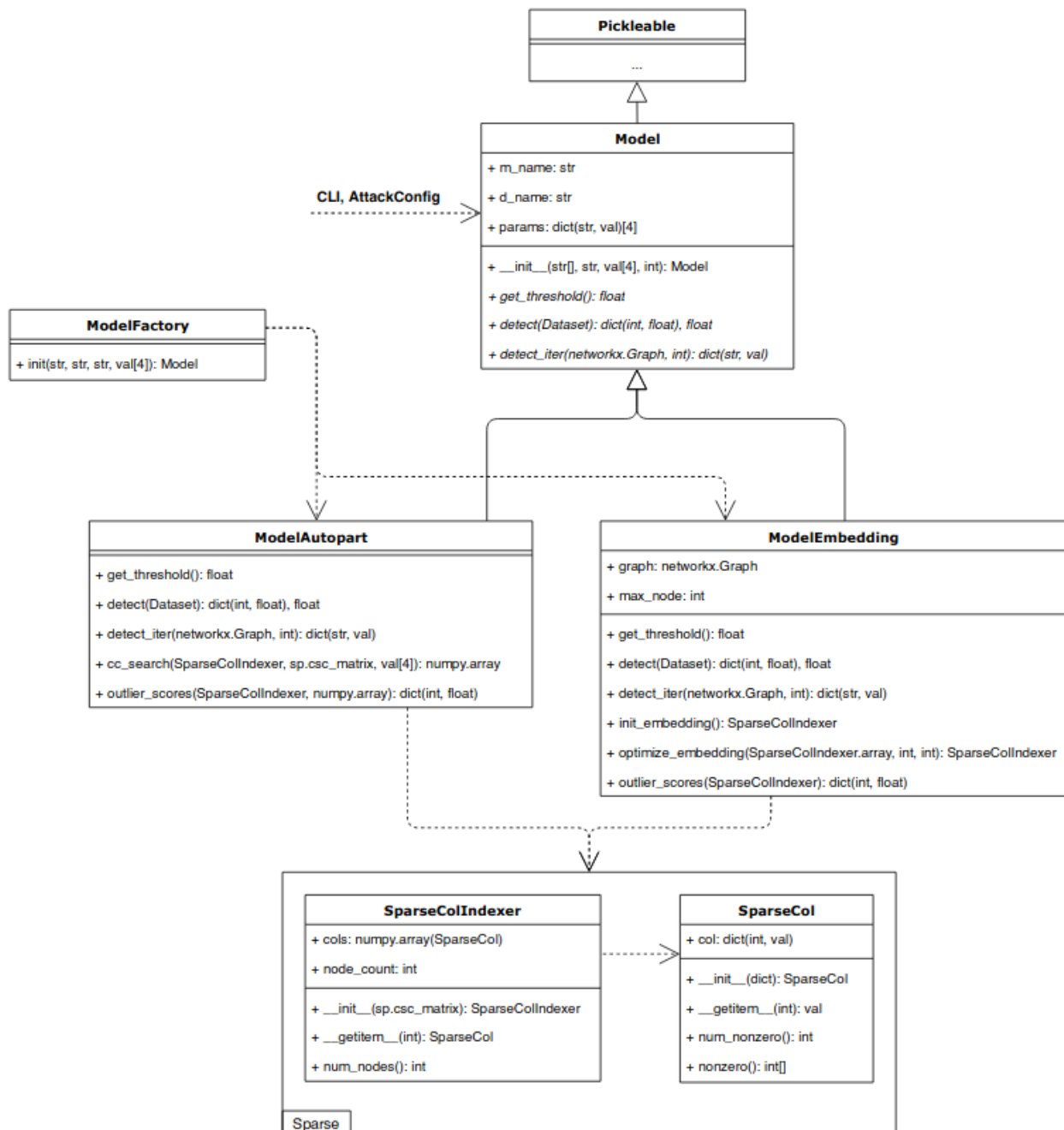


Figure 24: System Description

chkp [bool] Whether to store checkpoints when running the attack or not.
 verb [bool] Whether to enable verbose printing or not.
 final [bool] Whether to directly load previous final results.
 max_k [int] Maximum number of iterations to run before terminating.

In order to avoid misconfigured models causing unexpected behavior, all parameters and their values are printed when the script starts execution. This way, the user can manually verify that everything is as it is intended to be.

Checkpoints

Checkpoint saving and loading is enabled by extending the *Pickleable* base class. The class structure for *Pickleable* is displayed in Figure 25.

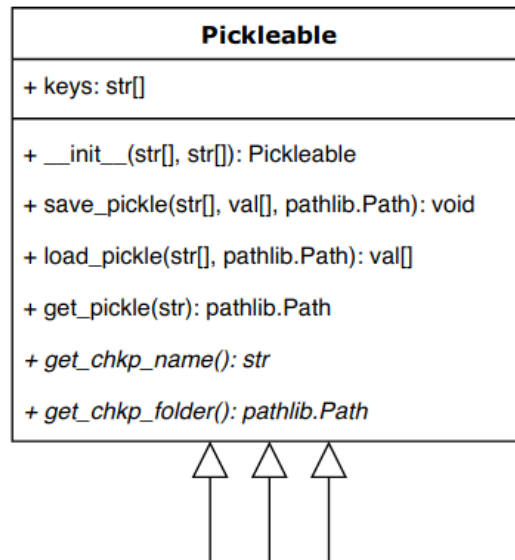


Figure 25: Pickleable Class Structure

The *save_pickle* and *load_pickle* functions implemented by *Pickleable* allow for the saving and loading of Pickle files. It is generally used for saving checkpoints, initialization configurations as well as final results. The *save_pickle* method takes a list of string keys and a list of values params and stores a dictionary that maps keys to params at a given location. Meanwhile, *load_pickle* retrieves params given keys and a location to search. The params list is returned as a tuple to allow for easier value assignment when called by a function.

To make loading from the correct path easier, *get_pickle* returns a path of the correct *.picklefile* to load, given it exists. It takes a string that specifies the type of file to look for (such as *chkp* for checkpoint, *init* for initialization, et cetera). Then, it relies on the corresponding abstract name and folder functions implemented by the subclasses, such as *get_chkp_name* and *get_chkp_folder* to construct the file path to look for.

It then looks for the existence of such a file or any variation thereof in the file system. If found, it will return the path. Otherwise, it will throw an exception.

Implementation of AutoPart

The AutoPart model is implemented through the *ModelAutopart* class, which extends the *abstractModelbase* class. The *ModelAutopartimplementation* uses the adjacency matrix version of the given graph (as opposed to directly using the underlying graph) for detection. This choice was made since the algorithm relies on clusters in the adjacency matrix and uses many operations related to the adjacency matrix, such as computing the number of nonzero elements per cluster. The main *detect* method is implemented in three steps:

- **Initialization.** First, the adjacency matrix of type *scipy.sparse.csc_matrix* is extracted from the *dataset* class and is immediately converted into an instance of *SparseColIndexer* to speed up array slicing. The *Sparsemodule* will be discussed later on.

Using the *Pickleable* behaviors introduced previously, *detect* then looks for the existence of any checkpoints or final files to load. If checkpoints are found, the cluster mapping is loaded from there. If final files are found, the result is returned directly. Otherwise, the clustering is initialized as a single cluster containing all nodes.

In the implementation, the clustering is stored as a numpy array *cluster*, where *cluster[i]* is the cluster to which node *i* belongs.

- **Cluster Search.** Then, using the previously introduced AutoPart algorithm, the best clustering is determined. This iterative process continues until the cost can no longer be decreased. At each step when *k* is incremented, the current clustering is stored as a checkpoint.
- **Anomaly Detection.** Once the final clustering has been determined, it is used to infer the anomalousness of the individual nodes by calculating their percentage of outgoing edges. The final results are stored as a *.pickle* file for future reference. Finally, the computed anomaly scores are returned.

The *detect_iter* method of *ModelAutopart* works similarly, however, it does not support checkpoint saving and stops iterating after a given step size k_{max} . Additionally, it returns not only the scores but all internal values to be filtered through the later introduced attack configurations.

Implementation of Embedding

The Embedding model is implemented through the *ModelEmbedding* class, which inherits from the *Model* base class. The *ModelEmbedding* model uses the *networkx.Graph* representation of the dataset, since it does not make use of the adjacency matrix.

The *detect* method of *ModelEmbedding* is implemented in three steps:

- **Embedding Initialization.** The embedding is initialized as described previously by using the *METIS*⁷ clustering algorithm. To ensure reproducibility, the initialized embedding is stored as a *pickle* and loaded for any following model calls instead of rerunning *METIS*.
To implement the $(k + \beta)$ -dimensionality reduction, the embeddings are stored as a *SparseColIndexer*, where only nonzero embedding elements are stored to reduce space. The embedding for node *i* is stored as the *i*-th *SparseCol* in the matrix.
- **Embedding Optimization.** This loaded embedding is then optimized based on the procedure outlined before, through the use of gradient descent optimization. Here, the gradient w.r.t. the loss is also stored as a *SparseColIndexer*, however with $2(k + \beta)$ values for each *SparseColvector*. The optimization is an iterative process and after each step, the current state is stored as a checkpoint.
- **Anomaly Detection.** Once the gradient optimization converges, the computed embeddings are then used to assign each node *i* its *AScore(i)* based on the neighborhood of *i*. The final results are stored as a *pickle* file for future reference. Finally, the computed anomaly scores are returned.

The *detect_iter* method of *ModelEmbedding* works analog to the *detect_iter* method of *ModelAutopart*.

Optimization

Both previously introduced methods for anomaly detection rely on storing data as a sparse matrix. *AutoPart* requires the adjacency matrix in a sparse format of $n \times n$. Meanwhile, the embedding approach stores all embeddings as a matrix of sparse vectors in order to implement the $(k + \beta)$ -dimensionality reduction. While the former requires many fast slicing operations with only read access, the latter demands for individual columns in the sparse matrices to be modifiable by reference. Both of these operations are not supported very well by the default *scipy.sparse.csc_matrix*, which generates new objects when slicing, generating enormous overhead in the process.

Therefore, this framework includes a sparse module, which provides fast implementations for sparse matrices and their vectors for this specific use case. The module contains two classes:

- **SparseColIndexer** consists of a *numpy.array* of references to the *SparseCol*. The *__getitem__* method returns the direct reference to the *SparseCol* object, avoiding any new object generation. Additionally, its constructor takes a *scipy.sparse.csc_matrix* and directly converts it into the needed format.
- **SparseCol** consists of a wrapper around a dictionary which provides additional methods. These methods allow *SparseCol* to be used like a column of a *scipy.sparse.csc_matrix*. Beyond that, its wrapper around *__getitem__* returns the value, if the key exists and zero otherwise. This way, a *KeyError* can be avoided.

When using the custom sparse module, the *AutoPart* implementation was able to speed up step duration by about 3 orders of magnitude. For the embedding implementation, a $3\times$ speedup was reached.

Attacks

Attacks are key to testing the robustness of graph anomaly detection models. This framework provides support for multiple attacks. This support includes information access specification, command-line access, attack parameterization and automatically loading and storing of checkpoints to simplify experimentation. This functionality is realized through the class architecture displayed in Figure 26. The next section will elaborate on the implemented constellation.

⁷<http://glaros.dtc.umn.edu/gkhome/metis/metis/overview>

Interface

Each attack is created through the use of the *AttackFactory*. The *AttackFactory* takes a string and multiple pass-through parameters as input and uses a map to look up the corresponding attack to use. Based on that, it creates a new instance of this attack, initializes it with the pass-through parameters, and returns that instance. All attacks extend the *Attack* base class, which extends the abstract *Pickleable* class in order to support checkpoint saving and loading. The main function provided by each attack is the attack function, which performs an attack on a given model and dataset.

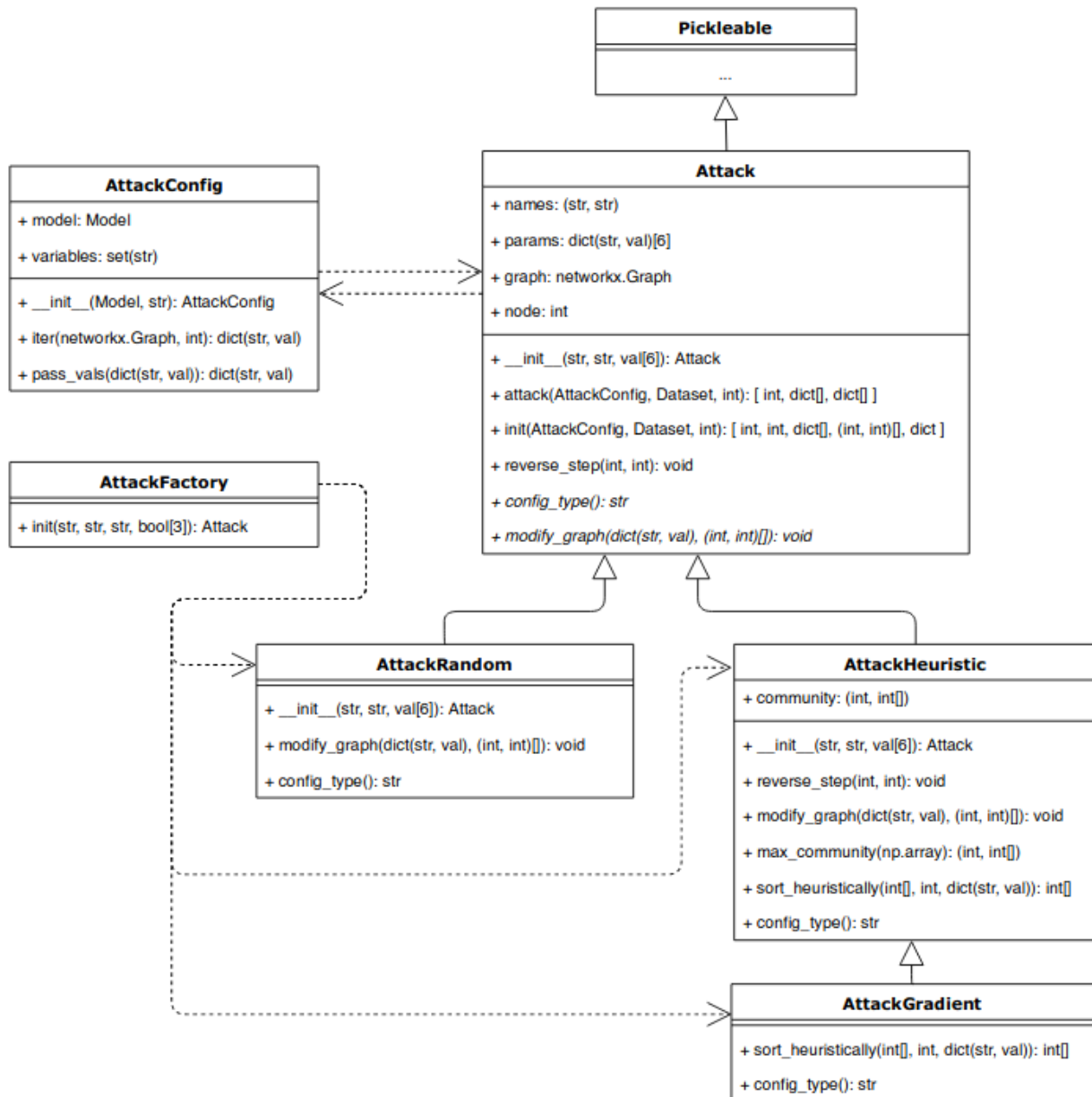


Figure 26: Attack Class Structure

Configuration

The sole way for attacks to interface with their model is to use an *AttackConfig* to handle access to the information within the model. Specifically, the *iter* method of *AttackConfig* will run the model with the attack-modified dataset for the through *step_size* specified number of iterations. Then, it will use its internal *pass_vals* function to filter the appropriate information to use for the specific attack privileges and returns a dictionary of select values to the attack. If the results indicate successful evasion, the attack will terminate. Otherwise, the attack uses the received information to modify the graph. Then, the whole process repeats.

Each attack specifies which configuration it uses through the implementation of the abstract *config_type* method declared in *Attack*.

The detailed implementation of the *AttackConfig* module will be explained below. The constructor of *AttackConfig* takes two-parameter. The first is an object of one of the subclasses of *Model*. This model is stored as the *AttackConfig* instance attribute *self.model*. The second parameter *config_type* is of type *str* and indicates which configuration to use for the attack. The supported options for *config_type* are *whitebox*, *blackbox* and *graybox*.

The constructor then uses a map to infer the path of the YAML (YAML Ain't Markup Language) file to load based on the value of *config_type*. YAML is a data serialization language used commonly for configuration files. Our attack configuration YAML file consists of two layers. The first layer is string representing the model used. Each model supported by this framework needs to be present on this layer. Each string in the first layer maps to a set of strings on the second layer. The set of strings represents the variables of the model that should be passed through to the attack.

The constructor extracts the set of variable names from the YAML file by first using the standard Python YAML library to extract the complete layered dictionary and then looking up the name of the model supplied to it for finding the specific set within this dictionary. The set of variables to pass through is stored under the instance attribute *self.variables* of the *AttackConfig*.

The *iter* method introduced previously represents the interface for attacks to interact with the model. It takes two parameters, *graph* and *step_size*. The *graph* parameter is of type *networkx.Graph* and represents the current version of the dataset the attack has produced. Meanwhile, *step_size* specifies how many steps the model should iterate before returning the results.

```
def iter(self, graph, step_size):
    results = self.model.detect_iter(graph, step_size)
    return self.pass_vals(results)

def pass_vals(self, params):
    filtered = {}
    for i in params.keys() | self.variables:
        filtered[i] = params[i]
    return filtered
```

The *iter* method works by directly calling the *detect_iter* method on *self.model* and passing through the parameters. Then, *AttackConfig* relies on its own *pass_vals* method to filter which of the results to pass on to the attack and which to hide.

The *pass_vals* method takes a dictionary params of key-type strand filters which values to pass. This is done by constructing a new dictionary. Only keys in the set intersection between params and *self.variables* are copied to the new dictionary, which is then returned to the attack.

Parameterization

Each attack has multiple parameters that can be configured through the CLI. This process works the same way as for models. However, the actual parameters are not identical between models and attacks. Each model parameter can be used with an attack, as well. Beyond that, there are two attack-exclusive parameters.

step_size [int] Number of steps to run model for each iteration.

thresh [float] Detection threshold to use to determine when to stop the attack.

Just as with models, all parameters and values are printed during initialization to avoid misconfigured attacks. This way, the user can manually verify that everything is as it is intended to be.

Checkpoints

Checkpoint saving is supported for attacks. The functionality is implemented in the same way as for models, by extending the *Pickleable* superclass and implementing the desired behaviors. The naming scheme for

attack checkpoints consists of attack name, model name, dataset name, evasive node, and checkpoint index.

For example, *chkp_random – embedding – dblp_1098 – 5.pickle* refers to checkpoint 5 of a random attack on the embedding model using dblp, with 1098 being the node for which evasion is attempted.

Each in-progress attack checkpoint contains 4 key-value pairs, holding the information necessary to restart the attack at the given checkpoint.

k	[int]	Iteration index. Also, the checkpoint counter.
node	[int]	Node index, for which evasion was attempted.
scores	[(dict(int, float))]	Ordered list of score maps for each step.
edges	[(int, int)]	Ordered list of inserted edges.

Final attack checkpoints are structured similarly, with the exception of not storing the iteration index *k*, which loses relevance for complete attacks.

Implementation of Base Attack

The core method provided by the base attack class is *attack*. It takes an *AttackConfig(config)*, a subclass of the *Dataset* base class (data), and a node index (node). It then proceeds to run the evasive attack for node against the model referenced by config. Showcased below is a simplified version of the attack method, highlighting only the core functionality for a better understanding.

```
def attack(self, config, data, node):
    k, node, scores, edges, results = self.init(config, data, node)

    while k <= self.params['max_k']:
        k += 1
        results = config.iter(self.graph, self.params['step_size'])
        scores.append(results['scores'])
        if self.params['chkp']:
            params = [k, node, scores, edges]
            self.save_checkpoint(params, str(k).zfill(2))

        if scores[-1][self.node] <= self.params['thresh']:
            # Achieved evasion
            self.save_final([node, scores, edges])
            return node, scores, edges

        if scores[-1][self.node] > min_score and edges:
            self.reverse_step(edges.pop(-1))
        else:
            min_edges = edges
            min_score = scores[-1][self.node]
            edges = self.modify_graph(results, edges)

        if edges[-1] == (node, node):
            return node, scores, edges
    return node, scores, min_edges
```

The method first initializes the core variables by calling *self.init*. This method automatically checks for existing checkpoints and loads them. Then, it starts the core loop, which runs for *max_k* iterations, or until the anomaly score of node drops below the threshold *thresh*. Each loop iteration consists of multiple steps:

- The *AttackConfig.iter* method is called with the current graph and constant *step_size*. This will evaluate the performance of the detection model at the current state of the attack. Accessing the model through the *AttackConfig* hides the actual model while still allowing interactions with the model and passing back only access-appropriate information in the form of the results dictionary.
- The current score map is then extracted from the results and appended to the list of score maps at each step of the attack. This information is later used to observe the change in score maps over the course of the experiment.

- If the *chkp* option is set, the checkpoint of the current step is saved. The specific information stored are iteration index *k*, node index *node*, list of score maps *scores*, and list of inserted connections *edges*. This information suffices to reconstruct all relevant values about this state of the attack.
- If the score of node has dropped below *thresh*, the attack has been successful. In that case, the final checkpoint is saved and the results are returned
- If the score of node is larger than the lowest previously achieved value, the previous iteration is considered as unsuccessful. In that case, the last inserted edge is removed and the last step is reversed using *reverse_step*.
However, If the score decreased compared with the lowest previous score, a new minimum has been achieved. Therefore the values *min_edges* and *min_scores*, storing the best combination of edges and the best score map respectively, will be updated.
- Finally, the graph is updated based on results. This is the key step of each attack and is realized through the *modify_graph* method, which is an abstract method of *Attack* that is implemented by all subclasses. The *modify_graph* method returns the updated list of all edges that have been added to the graph so far.
- However, it is possible for attacks to stop completely if all possible edges have been inserted. In order to detect such a result, the attack inserts the self-edge(*node*, *node*) as a modification. If the attack detects this, it will terminate the attack.

Once the loop is completed, the attack terminates and returns all required values.

Implementation of Random Attack

This section will elaborate on the implementation of the previously introduced random attack. The random attack is implemented through the *AttackRandom* class which is a child of *Attack*. The *config_type* method for *AttackRandom* returns 'black box' to specify black-box access privileges.

The methodology relating to random attacks has been introduced previously. Below is a simplified version of the implementation of the *modify_graph* method for a random attack.

```
def modify_graph(self, params, edges):
    pool = self.graph.nodes - self.graph[self.node] - {self.node}
    if not pool:
        edges.append((self.node, self.node))
        return edges

    next = random.sample(pool, 1)[0]
    edge = (self.node, next)

    self.graph.add_edge(*edge)
    edges.append(edge)
    return edges
```

First, the pool of available nodes to connect to is determined. If this pool is empty, (*node*, *node*) is appended to the list *edges* and returned to signal attack termination. Otherwise, a random node *next* is sampled from this pool, and a new edge from *node* to *next* is inserted into both the graph and the list of edges. Finally, the method returns the edges.

Implementation of Heuristic Attack

The generic attack is implemented through the use of the *AttackHeuristic* class which inherits from the *Attackbase* class. The *config_type* method for *AttackHeuristic* returns 'graybox' to specify gray box access privileges. This is since it requires both scores and current embedding but no other information about the internals of the model. A simplified version of the *modify_graph* method for *AttackHeuristic* is displayed below.

```
def modify_graph(self, params, edges):
    if self.community is None:
        self.community = self.max_community(params['embedding'])
        self.sort_heuristically(nodes, comm, params)

    neighbors = self.graph[self.node]
```

```
pool = [i for i in self.community[1] if i not in neighbors]

if not pool:
    edges.append((self.node, self.node))
    return edges

edge = (self.node, pool[0])

edges.append(edge)
self.graph.add_edge(*edge)
return edges
```

As discussed previously, our heuristic attack works by selecting a community to connect to and then determining an ordering of the nodes within that community by preference. Then, it connects to those nodes one by one. In our implementation, this functionality is enabled through the *self.community* attribute. This attribute is a tuple of the community index of type *int* and the nodes within that community of type *int*.

The community is determined through the *max_community* method, which simply finds the community with the largest number of edges. Meanwhile, the preferential ranking of the nodes is created through the *sort_heuristically* method, a simplified version of which is showcased below.

```
def sort_heuristically(self, nodes, community, params):
    li_1 = sorted(nodes, key=lambda a: params['scores'][a])
    li_2 = sorted(nodes, key=lambda a: self.comm_edges(a, nodes), reverse=True)
    nodes.sort(key=lambda a: li_1.index(a) + li_2.index(a))
```

The method takes a list of nodes, the community index (which is not used in this base implementation), and the filtered dictionary of parameters created by the model. It then sorts the nodes based on the two factors introduced previously, embedding x_j^k and the number of intracommunity edges $c(j, k)$.

Selecting the next node to connect to by cycling through a constant list with only existing connections filtered out, has consequences for reversing steps. Specifically, in order to prevent re-connecting to nodes that only increased the anomaly score previously, the *reverse_step* method is overloaded to remove such nodes from the community.

Implementation of Heuristic Gradient Attack

As previously mentioned, the heuristic gradient attack is technically a type of heuristic attack. Therefore, it has been implemented through the *AttackGradient* class which is a subclass of the base *AttackHeuristic*. The *config_type* method for *AttackGradient* returns 'whitebox' to specify white-box access privileges. This is since it requires the internal gradient information beyond the scores which is deep enough knowledge that it can be considered a white box attack.

AttackGradient overwrites its parent *sort_heuristically* method, as it is the only point of divergence between the standard heuristic attack and this gradient-based one. The implementation of *sort_heuristically* is fairly simple:

```
def sort_heuristically(nodes, community, params):
    nodes.sort(key=lambda a: abs(params['gradient'][a][community]))
```

As is shown, the ordering of nodes within the community is implemented by sorting the nodes based on the absolute value of ∇x_j^k , which is an indicator of stability in the community assignment.

User Interface

Multiple measures are taken to improve the usability of the implemented system and make experimentation as effortless as possible. Having a convenient interface can save significant time when experimenting with different models and attacks and encourages the adoption of the framework for attacking graph anomaly detection.

There are two main aspects of usability that were taken into consideration for this framework. Firstly, having a quick and easy setup makes it easier to get started. Secondly, making sure the interactions with the framework

itself are intuitive and convenient facilitates frequent usage. To improve the interactions, a CLI is provided as well as a visualization plugin.

Setup

This framework depends on multiple factors. The *DBLP* dataset needs to be downloaded and at the correct place. The *METIS* library needs to be downloaded, built, and linked. Furthermore, all Python requirements need to be installed using pip. Finally, all required environment variables need to be set.

There are two ways to properly set up this framework. The generic solution available on every platform is to use Docker. Docker is a container platform that enables packaging up code with its dependencies to then be run on any platform. It can be used with this framework by simply running the *docker.sh* script included in the repository. Should the Docker image not exist yet, this will build it, including running the scripts to download and install all requirements and setting the environment variables.

Alternatively, on UNIX-based operating systems the setup can be done directly on the machine by running *setup/setup.sh*. This will install all requirements locally. Afterward, the environment variables have to be set manually. First, *PATH_REPO_AGAD* points to the location to which the repository has been cloned. Then, *METIS_DLL* points to the executable of the *METIS* library.

The setup process is also described in the repository's *README* file.

CLI

A command-line interface is a way of communicating with the user through the command line. This framework provides two CLIs, *detect.py* for running detection models and *attack.py* for attacking these models. Both accept a number of options to determine how to run their task. The options for *detect.py* are as follows:

-m	--model	TEXT	Model to use. The accepted options are auto part and embedding(default).
-d	--data	TEXT	Dataset to use. The accepted options are mock (default), mock[0-4], and dblp.
-i	--iterations	INT	Number of iterations to run the model for.-1 (default) uses the model default.
-t	--thr	FLOAT	Threshold to apply for the anomaly score.-1 (default) uses the model default.
-c	--check	BOOL	Enable checkpoint saving
-v	--verb	BOOL	Enable verbose CLI output
-p	--plot	BOOL	Enable plotting the distribution of anomaly scores.
-f	--final	BOOL	Enable loading final results from previous runs.

The options for *attack.py* are as follows:

-a	--attack	TEXT	Attack method to deploy. The accepted options are gradient, heuristic, random (default)
-m	--model	TEXT	Same as for <i>detect.py</i>
-d	--data	TEXT	Same as for <i>detect.py</i>
-i	--iterations	INT	Number of iterations to run the attack
-t	--thr	FLOAT	Same as for <i>detect.py</i>
-n	--node	INT	Node to evade. -1 (default) picks most anomalous.
-s	--step-size	INT	Number of model iterations per attack iteration.
-c	--check	BOOL	Same as for <i>detect.py</i>
-v	--verb	BOOL	Same as for <i>detect.py</i>
-p	--plot	BOOL	Same as for <i>detect.py</i>
-f	--final	BOOL	Same as for <i>detect.py</i>

Visualization

In order to help provide a better understanding of the experiments results, a visualization component is included in the framework. It supports visualizing distribution of anomaly scores for the results of a given detection model. Additionally, it supports overlaying two distributions, while highlighting the position of a given node in both. This can be used in combination with an attack, to see how the overall distribution of scores changed. Also, it showcases how much the anomaly score of the evasive node decreased when compared to the overall distribution.

Internally, the visualization component relies on the *matplotlib* library to draw the diagrams. Specifically, it utilizes *matplotlib.pyplot* to produce the plot. Plotting the distribution of anomaly scores for all nodes in a graph is a multi-step process.

- The visualization component receives a *dict*{*int* : *float*} which maps each *node_id* to an anomaly score. Here, higher anomaly scores indicate a higher likelihood of maliciousness.
- The dictionary is converted into a *list*[(*int*, *float*)] and sorted by descending anomaly score of the nodes.
- The list is plotted as a distribution (in blue), with x-values being the index in the sorted list and y-values corresponding to the anomaly scores.
- (If plotting detection) Using the threshold passed to the component, the list is split into two parts. The first contains all nodes with anomaly scores above the threshold, therefore being classified as anomalies. These nodes are plotted again in red, to indicate anomalousness. This overrides the initial blue plots for said nodes.
- (If plotting attack) Another dictionary is supplied for the initial scores before the attack. These are plotted in gray using the procedure outline in step 1-2
- (If plotting attack) The evasive node is specified and highlighted using cyan on both the initial and post-attack distribution plots.

The visualization component integrates with the (*-p/ - -plot*) command line option for both *detect.py* and *attack.py*.

3.3.3.3 Experimental Results

All experiments were done using the *DBLP* dataset. Specifically, the version of mined by the Stanford Network Analysis Project (SNAP) [61] was used for all experiments discussed below. The *DBLP* dataset is a citation dataset of major computer science publications. It consists of bibliographic information about all included publications and authors. As of January 2019, the dataset contains over 4.4 million publications and over 2.2 million authors. Still, the framework supplies multiple datasets, which can be specified when running attacks or detection. The provided datasets are *mock0* and *DBLP*.

The robustness of the Embedding model against evasion attacks was evaluated using the following setup. Of the 135 detected anomalies, 11 nodes were uniformly sampled to represent different nodes with varying distances to the anomaly threshold. For each of the sampled nodes, all three attacks were run. Each attack was run for a maximum of 18 steps until being stopped. If a score of below 2.5 was reached, it was stopped immediately. The step size was set to 100, meaning the Embedding model would run for 100 iterations between each attack step. This number was chosen as a trade-off between attack time and model *convergence*, since the rate of descent of the model loss decreased strongly around iteration 100. After each attack step, the recomputed anomaly score of the evasive node as well as the inserted edge was printed and saved as a checkpoint.

As Table shows, all attacks were able to successfully decrease the anomaly score in only a few steps to evade detection for the anomaly threshold of 16.0 they had been detected with. In fact, the highest post-attack anomaly score of 6.21 was less than half the threshold. However, due to the distribution of the underlying anomaly scores, an anomaly score of 6.21 would still be within the 97th percentile of anomalousness, making it feasible to still be detected when lowering the anomaly threshold. While this would be an improvement over the 99th percentile all anomalies started out in, it could still not be enough to achieve the stated objective. Therefore, we decided to consider an evasive attack successful if and only if the score percentile decreased beyond a threshold 90%, to ensure a high enough number of other nodes to blend in with when looking for anomalies. Under this criteria, 26 out of the 33 attacks were able to achieve evasion. Beyond that, 15 of the 33 attacks were able to achieve a final score percentage of $\leq 60\%$. About half all attacks were able to achieve evasion by inserting only three edges or fewer. Evasion Attack Results

Node	Initial	Random [Edges-C]	Heuristic [Edges-C]	Gradient [Edges-C]
69460	25.14	2.48 (59%) [2-0.08]	3.01 (46%) [3-0.09]	5.03 (91%) [5-0.08]
4256	20.15	3.68 (79%) [2-0.12]	3.04 (71%) [2-0.08]	2.73 (66%) [6-0.15]
27513	1.91	2.31 (49%) [3-0.12]	2.52 (61%) [7-0.11]	1.48 (38%) [7-0.12]
106237	18.91	3.56 (80%) [3-0.09]	3.99 (82%) [4-0.09]	5.97 (95%) [6-0.13]
92904	17.75	4.91 (92%) [3-0.09]	1.98 (50%) [4-0.09]	5.10 (93%) [2-0.08]
163051	17.00	1.60 (39%) [2-0.11]	2.69 (54%) [6-0.18]	2.81 (64%) [5-0.08]
26641	16.42	1.94 (51%) [2-0.08]	6.21 (97%) [2-0.09]	6.21 (97%) [2-0.05]
29960	16.78	1.41 (42%) [6-0.18]	1.47 (43%) [2-0.09]	2.95 (69%) [1-0.07]
170972	16.65	4.82 (91%) [4-0.19]	1.97 (49%) [5-0.11]	4.22 (88%) [5-0.09]
107629	16.22	2.20 (58%) [1-0.06]	2.03 (52%) [2-0.09]	2.03 (52%) [2-0.08]
149041	16.00	3.64 (80%) [4-0.08]	1.79 (48%) [3-0.12]	3.83 (84%) [3-0.12]

Table 3: Graph Anomaly Evasion Attack Results

3.3.4 Defending AI algorithms used in Network Intrusion Detection against evasion attacks

3.3.4.1 Introduction - adversarial attack detector for IDS

The following section is an extension of the work described in d7.2. While the description contained in d7.2 was a theoretical proposal of a solution, the following paragraphs focus on the implementation details of the first demonstration. To keep the chapter self-contained, while not repeating descriptive work, parts of d7.2 are recapitulated. The proposed solution is a novel approach to handling adversarial attacks against artificial neural networks. In this work:

- four different adversarial attack methods are implemented: Fast Gradient Sign Method, Basic Iterative Method, Carlini and Wagner Attack, and Projected Gradient Descent.
- a method of detecting those attacks is introduced
- the method does not influence the detection results of the IDS
- the solution is experimentally tested on a recent IDS benchmark dataset
- five different pattern recognition algorithms were tested in the detection pipeline, and the results are provided

The work contained here takes into consideration all the neural activations in all the layers, thanks to the fact that the architecture used in intrusion detection, with satisfactory results, uses fewer layers and fewer neurons on those layers. To the best of our knowledge, this is the first time using neural activations will be applied to detecting adversarial examples in intrusion detection.

The general consensus among the researches with regard to the defensive measures is that no fully safe system has been put forward and no truly field-proven solutions exist [31]. The methods developed to this point apply to certain kinds of attacks, but do not provide defence against all possible adversarial attacks.

	Active Max	Active Min	Idle Mean	Idle Std	Idle Max	Idle Min	Label
73473	0	0	0.00000	0.00000	0	0	BENIGN
108430	0	0	0.00000	0.00000	0	0	BENIGN
491895	0	0	0.00000	0.00000	0	0	BENIGN
153362	4682692	50625	9986655.83300	29155.74716	10000000	9927624	BENIGN
151486	0	0	0.00000	0.00000	0	0	ATTACK
296735	17929	17929	99200000.00000	0.00000	99200000	99200000	BENIGN
13835	0	0	0.00000	0.00000	0	0	BENIGN
199802	0	0	0.00000	0.00000	0	0	BENIGN
223250	14010	14010	98400000.00000	0.00000	98400000	98400000	ATTACK
169516	1008	1008	98600000.00000	0.00000	98600000	98600000	ATTACK
292934	0	0	0.00000	0.00000	0	0	ATTACK
24739	23888	23888	94400000.00000	0.00000	94400000	94400000	BENIGN
70723	0	0	0.00000	0.00000	0	0	BENIGN
94048	3003	3003	40800000.00000	47000000.00000	74100000	7540892	ATTACK
66756	0	0	0.00000	0.00000	0	0	BENIGN
121334	1242892	37559	9960084.50000	83966.80895	10000000	9900711	BENIGN
510825	0	0	0.00000	0.00000	0	0	BENIGN
196264	0	0	0.00000	0.00000	0	0	ATTACK
127627	0	0	0.00000	0.00000	0	0	ATTACK
196010	0	0	0.00000	0.00000	0	0	ATTACK
626271	0	0	0.00000	0.00000	0	0	BENIGN
22538	0	0	0.00000	0.00000	0	0	BENIGN

Figure 27: Some features and the class labels of CICIDS2017 after binarisation

Some of those solutions lead to the deterioration in ML performance [14].

3.3.4.2 Proposed Method for Evasion Attack Detection in Neural Networks

In this section, the overall approach for Evasion Attack Detection in Neural Networks will be presented. Firstly, the utilised dataset is disclosed, which is followed by the applied dataset preprocessing and the IDS training pipeline. Then, the attacks on the IDS are performed and tested. The neural activations of those attacks, as well as the activations for clear samples are gathered; finally the attack detector is trained and tested.

3.3.4.2.1 Intrusion Detection based on an Artificial Neural Network

In this work, the dataset was first cut into four parts in a stratified fashion to ensure full coverage of all kinds of attacks included in the dataset in all its sub-parts. This procedure results in the following setup:

- Dataset A - used to train the IDS classifier
- Dataset B - used to test the IDS classifier and to craft the adversarial attacks, and test them on the original IDS ANN, then to acquire the activations of neural nodes in the IDS network of benign, attack and adversarial samples to train the Adversarial Detector
- Dataset C and D - used to craft test adversarial samples and acquire the activations for the neural nodes of benign, attack and adversarial samples

Listing 3.1: Splitting the CICIDS2017 dataset into stratified subsets

```
from sklearn.model_selection import train_test_split

dgornepol, ddolnepol = train_test_split(dataset1, test_size=0.5, random_state=0
    , stratify = dataset1['_Label'])

d4train, d4adv = train_test_split(dgornepol, test_size=0.5
    , random_state=0, stratify = dgornepol['_Label'])
```

```
df_test, d_test_adv = train_test_split(ddolnepol, test_size=0.5, random_state=0,
                                       , stratify = ddolnepol['_Label'])
```

All of the sub-parts were then turned into a binary classification task, leaving all the benign samples as 'BENIGN', but changing all the names of possible attacks to simply 'ATTACK'.

The binarized dataset is fed to the architecture described above, and the training procedure results in building a model capable of binary classification.

	Precision	Recall	F1-score	Support
ATTACK	0.96	0.97	0.97	139675
BENIGN	0.99	0.99	0.99	405383
micro avg	0.98	0.98	0.98	545058
macro avg	0.97	0.98	0.98	545058
weighted avg	0.98	0.98	0.98	545058
samples avg	0.98	0.98	0.98	545058

Table 4: 'IDS ANN' trained on Dataset A and tested on Dataset B

3.3.4.2.2 Adversarial Attacks

After testing the trained IDS (the optimisation procedure of an ANN-based IDS can be found in [41]), four different adversarial attacks were crafted based on the ATTACK class of Dataset B. The algorithms used for the creation of evasion attacks were:

- Carlini and Wagner attack (CW) [11]
- Fast Gradient Sign Method (FGM) [23]
- Basic Iterative Method (BIM) [29]
- Projected Gradient Descent (PGD) [35]

Listing 3.2: Evasion Attack generation

```
attackCL2 = CarliniL2Method(classifier=classifier)
attackFGM = FastGradientMethod(classifier=classifier)
attackBIM = BasicIterativeMethod(classifier=classifier)
attackPGD = ProjectedGradientDescent(classifier=classifier)
df_adv_CL2 = attackCL2.generate(df)
print("Carlini_attacks_generated")
df_adv_FGM = attackFGM.generate(df)
print("FGM_attacks_generated")
df_adv_BIM = attackBIM.generate(df)
print("BIM_attacks_generated")
df_adv_PGD = attackPGD.generate(df)
print("PGD_samples_generated")
```

1397 samples of the 'ATTACK' class were randomly extracted from Dataset B and turned into adversarial samples with the use of those four algorithms. The IDS ANN classified those as 1353 ATTACKS and 44 BENIGNS. Using adversarial attacks we were able to force the IDS ANN to classify 1296 'ATTACK' as 'BENIGN' samples for BIM and PGD, 1324 for FGM and 59 for CW.

The abovementioned procedures introduce adversarial noise to the samples. This noise resulted in negative values in some of the features. Those negative values were supplanted by zeros.

The samples from Dataset B not used in crafting Adversarial Attacks were annotated as 'nonadversarial', the Adversarial Attacks were labelled 'adversarial'. With 5588 adversarial attack samples, a matching number of nonadversarial records was randomly picked from the unused samples of Dataset B to form the base for a balanced 'Adversarial Training Dataset' for the adversarial attack detector. Dataset D was subjected, except for the balancing, to the exact same crafting/annotation procedure to form the base for the testing dataset for the detector.

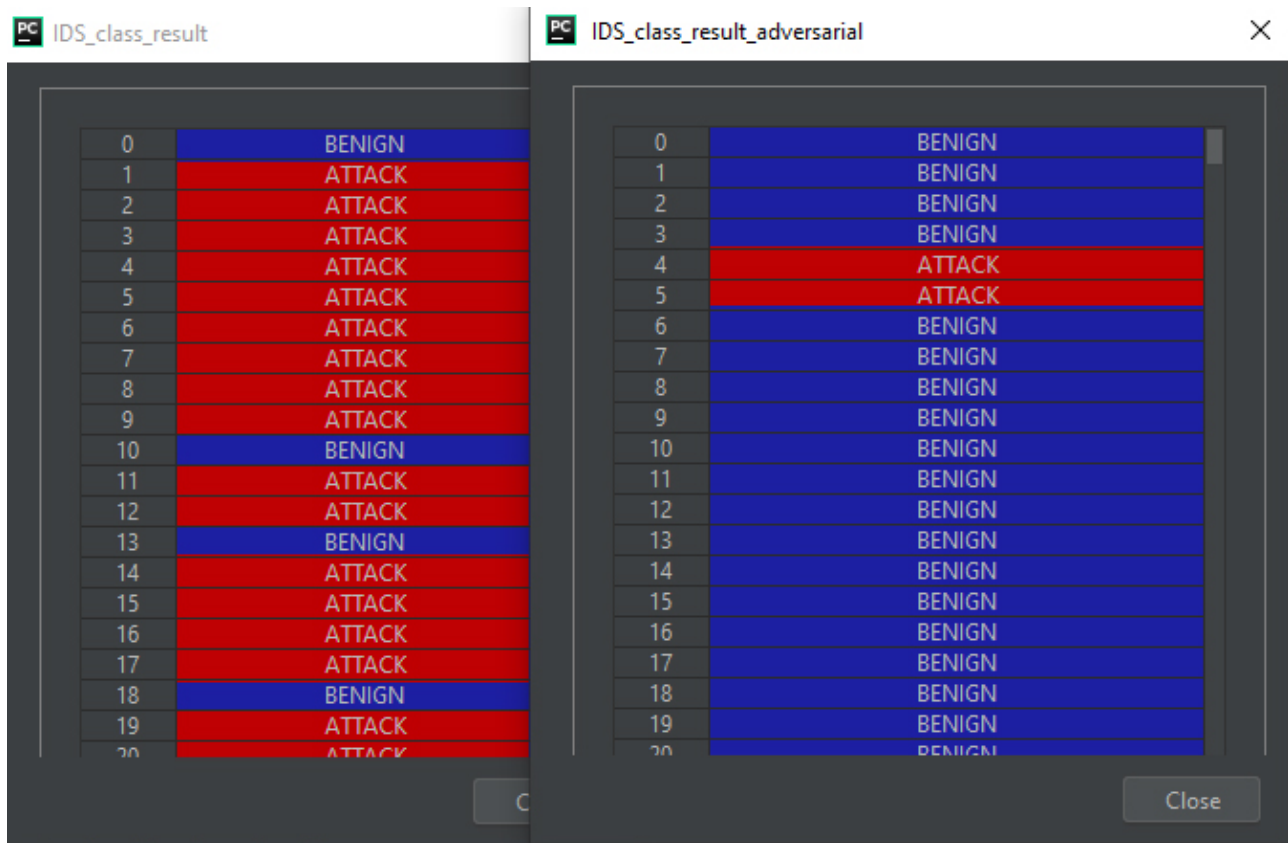


Figure 28: The IDS ANN classification of the same samples before and after evasion attacks

3.3.4.2.3 Detection Method

The training and testing activation datasets were fed to the IDS ANN and the activations for all the 102 neurons (including the softmax layer), as shown in Fig.29, were recorded and annotated as adversarial or nonadversarial, respectively.

Listing 3.3: Collecting neural activations

```
layer_outputs = [layer.output for layer in classifier.layers]
activation_model = models.Model(inputs=classifier.input, outputs=layer_outputs)
activations = activation_model.predict(x)

activations_df0 = pd.DataFrame(activations[0]) #dense_1
activations_df1 = pd.DataFrame(activations[1]) #dense_2
activations_df2 = pd.DataFrame(activations[2]) #dense_3
activations_df3 = pd.DataFrame(activations[3]) #dense_4
act_flt_frm = [activations_df0, activations_df1, activations_df2, activations_df3]
activations_flat = pd.concat(act_flt_frm, axis=1)
```

3.3.4.2.4 IDS ANN Setup

The IDS setup was as follows: an Artificial Neural Network of 3 hidden layers was compiled, with 40 neurons on the first hidden layer, 40 on the second and 20 on the third layer. The Rectified Linear Unit activation function was utilised, and the selected optimiser was ADAM. With the batch size of 100 and 10 epochs, the network achieved the accuracy of 0.9827 when trained with Dataset A and tested on Dataset B. The precision, recall and f1-score are showcased in Table 4

3.3.4.2.5 Detector Setup

The recorded activations were used to train the detector artificial neural network. The architecture of the detector is as follows: 3 hidden layers with the ReLU activation function, 51, 51 and 25 neurons respectively,

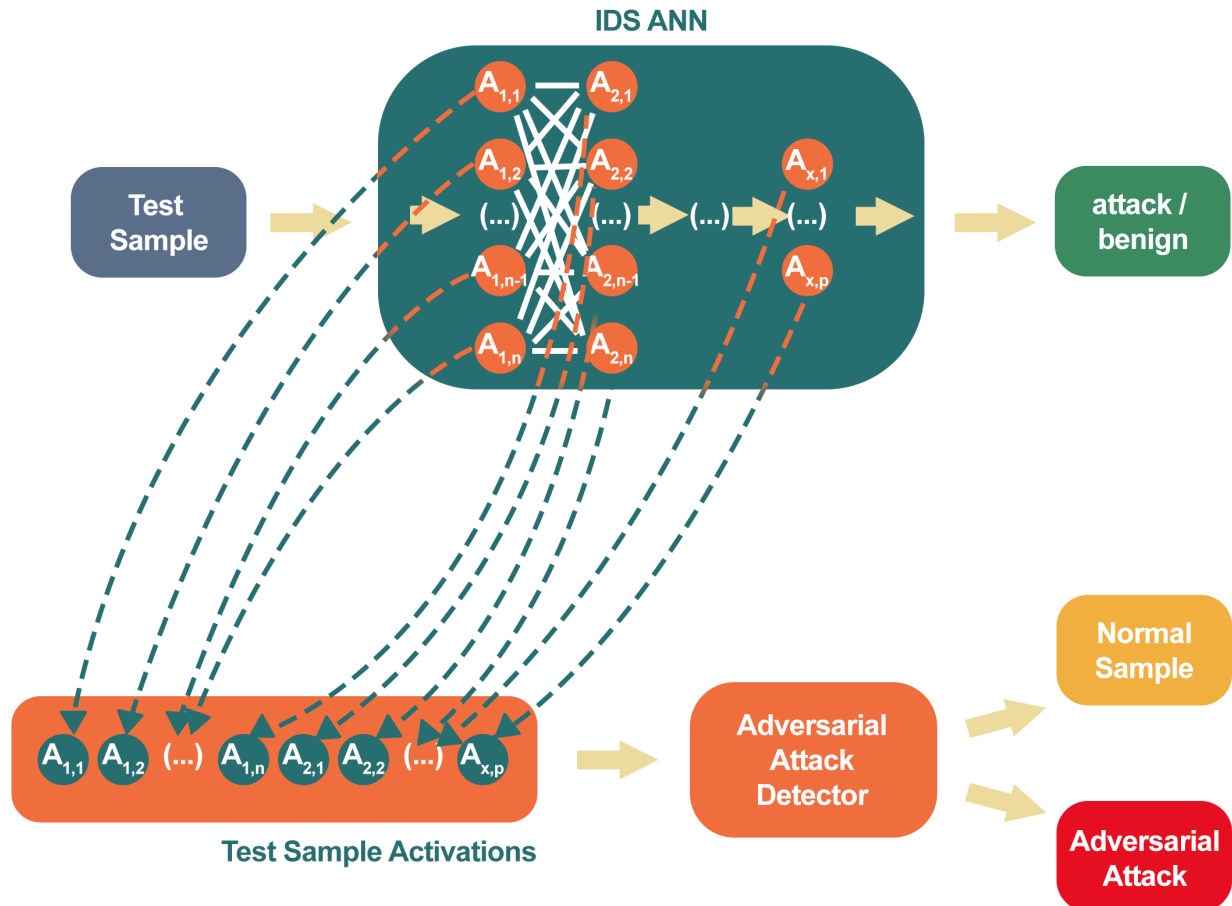


Figure 29: The acquisition of IDS ANN activations for a given test sample

and the ADAM optimiser. Using the batch size of 100 and just 10 epochs, the detector achieved the accuracy of 0.8506 on the testing set. The detailed results are assembled in Tab. 5.

Listing 3.4: Detector ANN setup

```

classifier = Sequential()
classifier.add(Dense(kernel_initializer="uniform"
    , units = 51, input_dim=102, activation="relu" ))
classifier.add(Dense(kernel_initializer="uniform"
    , units = 51, activation="relu" ))
classifier.add(Dense(kernel_initializer="uniform"
    , units = 25, activation="relu" ))
classifier.add(Dense(kernel_initializer="uniform"
    , units = 2, activation="softmax" ))
classifier.compile(optimizer="adam"
    , loss="categorical_crossentropy", metrics=['accuracy'])

classifier.fit(X_train, y_train, batch_size=5, epochs=40)

y_pred = classifier.predict(X_test)
  
```

The detector achieves high accuracy and the recall for the adversarial class signifies that it can recognise the attacks with great promise. The precision however is the evidence of a high number of false-positives.

Since in the process of creating the ANN-based adversarial attack detector a dataset of neural activations of the IDS architecture was created, the authors proceeded to test the approach using other well-established classifiers. In Tab. 5, the results of detection with Random Forest (RF) are presented. As immediately apparent, with this kind of data, RF achieves results superior to ANN, with higher recall and better precision. Notably, the accuracy of this approach exceeds 91% (91.24).

```

11035/11176 [=====>.] - ETA: 0s - loss: 0.1740 - acc: 0.8923
11176/11176 [=====] - 3s 254us/step - loss: 0.1741 - acc: 0.8920
Epoch 40/40

 5/11176 [.....] - ETA: 11s - loss: 0.0317 - acc: 1.0000
230/11176 [.....] - ETA: 2s - loss: 0.1200 - acc: 0.9261
475/11176 [>.....] - ETA: 2s - loss: 0.1461 - acc: 0.9095
680/11176 [>.....] - ETA: 2s - loss: 0.1465 - acc: 0.9103
865/11176 [=>.....] - ETA: 2s - loss: 0.1479 - acc: 0.9064
1110/11176 [=>.....] - ETA: 2s - loss: 0.1488 - acc: 0.9099
1355/11176 [==>.....] - ETA: 2s - loss: 0.1544 - acc: 0.9033
1620/11176 [==>.....] - ETA: 2s - loss: 0.1564 - acc: 0.9062
1835/11176 [==>.....] - ETA: 2s - loss: 0.1632 - acc: 0.9025
2035/11176 [==>.....] - ETA: 2s - loss: 0.1645 - acc: 0.9007
2265/11176 [==>.....] - ETA: 2s - loss: 0.1609 - acc: 0.9015
2495/11176 [==>.....] - ETA: 1s - loss: 0.1628 - acc: 0.8986
2690/11176 [==>.....] - ETA: 1s - loss: 0.1599 - acc: 0.9007
2905/11176 [==>.....] - ETA: 1s - loss: 0.1601 - acc: 0.8998
3105/11176 [==>.....] - ETA: 1s - loss: 0.1584 - acc: 0.9011
3285/11176 [==>.....] - ETA: 1s - loss: 0.1584 - acc: 0.9020
3455/11176 [==>.....] - ETA: 1s - loss: 0.1595 - acc: 0.8999
3665/11176 [==>.....] - ETA: 1s - loss: 0.1589 - acc: 0.9007
3875/11176 [==>.....] - ETA: 1s - loss: 0.1593 - acc: 0.8996

```

Figure 30: The ANN serving as the adversarial attack detector during training

	ANN			Random Forest			
	Precision	Recall	F1-score	Precision	Recall	F1-score	Support
adversarial	0.06	0.91	0.11	0.11	0.99	0.20	5588
non-adversarial	1.00	0.85	0.92	1.00	0.91	0.95	543661
macro avg	0.53	0.88	0.51	0.56	0.95	0.58	549249
weighted avg	0.99	0.85	0.91	0.99	0.91	0.95	549249

Table 5: Results of Adversarial Attack Detector over the test set activation using the ANN and RF classifiers

Listing 3.5: Random Forest and K-Nearest Neighbour classifiers used for adversarial attacks detection

```

from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=20, criterion='entropy', random_state=0)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

from sklearn.neighbors import KNeighborsClassifier
neigh = KNeighborsClassifier(n_neighbors=10)
neigh.fit(X_train, y_train)
y_pred_NN = neigh.predict(X_test)

```

Finally, the same activation dataset was utilised to train a nearest neighbour classifier. The procedure achieved results on par with the Random Forest method. The details can be found in Tab. 6

	precision	recall	F1-score	Support
adversarial	0.11	0.99	0.20	5588
nonadv	1.00	0.91	0.95	543661
macro avg	0.56	0.95	0.58	549249
weighted avg	0.99	0.91	0.95	549249

Table 6: Results of Nearest Neighbour-based Adversarial Attack Detector over the test set activation

3.3.4.3 Conclusions and future work

The major contribution of this work comes in the form of using neuron activations at test time to detect adversarial attacks performed with the use of four known evasion attack algorithms, namely: Fast Gradient Sign, Basic Iterative Method, Carlini and Wagner attack, and Projected Gradient Descent in the context of cybersecurity. The authors collected the test time neural activations of an ANN trained on a part of the CICIDS2017 dataset [49] and the neural activations of the adversarial examples crafted for this ANN. Using these activations, the authors trained and tested five different ML classifiers to detect adversarial examples, achieving the recall of 0.99 for adversarial attacks with two of the classifiers - Random Forest and the Nearest Neighbour classifier. The initial description coming from d7.2 was extended with implementation details.

The reported results are promising, and suggest the possibility of building an adversarial attack detector which does not affect the classification results of the protected model, which can drive further research in the direction of defending networks and intrusion detection systems based on machine learning algorithms. However, further lowering of the false positive rate is of utmost importance for the future development of this method.

While the topic of evasion attacks on artificial neural networks has been gaining in popularity recently, most of the research is performed on the image recognition datasets. For this reason, it is hard to compare the results of this innovative work with other defence or detection systems.

Chapter 4 Explainability

In this chapter, we introduce new applications and implementations compared to D7.2.

4.1 Introduction

Nowadays, Machine Learning models are used for various applications with already successful or promising results. Unfortunately, a common criticism is the lack of transparency associated with these algorithm decisions. This is mainly due to a greater interest in performance (measurable on specific tasks) at the expense of a complete understanding of the model. This results in a lack of knowledge of the internal working of the algorithm by the developer and the end user. The most obvious consequences are firstly a difficulty to correct the algorithm by an expert (different assumptions, removing outliers, adding new variables or diverse samples). Secondly, limiting its adoption by operational staff. There is even an urgent need for an explainable Artificial Intelligence (AI). Indeed, beyond these first reasons, the European Commission has imposed by legal means, with the General Data Protection Regulation, this transparency constraint on companies whose algorithms learn from personal data coming from European citizens. The challenge that companies are facing today is to bring AI into production. The transition from conclusive laboratory tests (Proof of Concept) to a production environment is not easy. To ensure that the model generalizes well on new data, a good human/machine interaction is highly appreciated.

There is no single definition of interpretability or explainability concerning model prediction (e.g. see the excellent introductory book [38]). Therefore, there are several ways to proceed. Assessing them objectively is a real problem because we do not have unanimous criteria. Most studies analyze the feedback from a panel of individuals, expert or not, to demonstrate the contribution of a method in terms of understanding. Methods are rarely compared directly with each other, but rather against a lack of interpretation of algorithm decisions. However, some references try to create quantitative indicators to evaluate the complexity of a model [39].

We can however separate methods into two dimensions [6]. If the method is local or global, and if its approach is model agnostic or on the contrary inherent to it. A global method aims at explaining the general behaviour of a model, whereas a local method focuses on each decision of a model. The agnostic category (also called post-hoc explanation) considers the model as a black box. On the other hand, inherent or non-agnostic methods can modify the structure of a model or the learning process to create intrinsically transparent algorithms. Naturally, the best strategy is to find a model that is both completely transparent by design and sufficient in terms of accuracy. Unfortunately, the most effective machine learning models tend to be less transparent because their degrees of complexity are high (e.g. gradient boosting trees, deep learning, kernel methods).

In this Chapter, we focus on two approaches. The first one is model agnostic and allows to provide local attribute importance in an optimal way. The second approach is a hybrid oracle explainer based on decision trees, which has been applied to Intrusion Detection Systems.

4.2 Attribute Importance based on Shapley Values

4.2.1 Shapley Values definition

In Collaborative Game Theory, Shapley Values ([48]) can distribute a reward among players in a fairly way according to their contribution to the win in a cooperative game. We note $\mathcal{M}$ a set of d players. Moreover, we note $v : P(\mathcal{M}) \rightarrow R_v$ a reward function such that $v(\emptyset) = 0$. The range R_v can be $\mathbb{R}$ or a subset of $\mathbb{R}$. $P(\mathcal{M})$ is a family of sets over $\mathcal{M}$. If $S \subset \mathcal{M}$, $v(S)$ is the amount of wealth produced by coalition S when they cooperate. The Shapley Value of a player j is a fair share of the global wealth $v(\mathcal{M})$ produced by all players together:

$$\phi_j(\mathcal{M}, v) = \sum_{S \subset \mathcal{M} \setminus \{j\}} \frac{(d - |S| - 1)! |S|!}{d!} (v(S \cup \{j\}) - v(S)),$$

with $|S| = \text{cardinal}(S)$, i.e. the number of players in coalition S . The Shapley Values are the only values which respect the four following properties:

- **Additivity:** $\phi_j(\mathcal{M}, v + w) = \phi_j(\mathcal{M}, v) + \phi_j(\mathcal{M}, w)$ for all j , with $v : P(\mathcal{M}) \rightarrow R_v$ and $w : P(\mathcal{M}) \rightarrow R_v$;
- **Null player:** if $v(S \cup \{j\}) = v(S)$ for all $S \subset \mathcal{M} \setminus \{j\}$ then $\phi_j(\mathcal{M}, v) = 0$;
- **Symmetry:** $\phi_{\Pi j}(\Pi \mathcal{M}, \Pi v) = \phi_j(\mathcal{M}, v)$ for every permutation Π on $\mathcal{M}$;
- **Efficiency:** $\sum_{j \in \mathcal{M}} \phi_j(\mathcal{M}, v) = v(\mathcal{M})$.

When there is no risk of confusion, we will simply write ϕ_j instead of $\phi_j(\mathcal{M}, v)$ in the rest of document.

4.2.2 Shapley Values as contrastive local attribute importance in Machine Learning

In this study we consider the context described in [37] but all the estimation methods presented hereafter can be extended. Let be $X^* \subset \mathbb{R}^d$ a dataset of individuals where a Machine Learning model f is trained and/or tested and d the dimension of X^* . $d > 1$ else we do not need to compute Shapley Value. We consider the attribute importance of an individual $\mathbf{x}^* = \{x_1^*, \dots, x_d^*\} \in X^*$ according to a given reference $\mathbf{r} = \{r_1, \dots, r_d\} \in X^*$. We're looking for $\phi = (\phi_j)_{j \in \{1, \dots, d\}} \in \mathbb{R}^d$ such that:

$$\sum_{j=1}^d \phi_j = f(\mathbf{x}^*) - f(\mathbf{r}),$$

where ϕ_j is the attribute contribution of feature indexed j . We loosely identify each feature by its column number. Here the set of players $\mathcal{M} = \{1, \dots, d\}$ is the feature set.

The local explanation is then contrastive because attribute relevance depends on a reference. For Machine Learning purposes, the range R_v of wealth v could be any real number, namely the output of a regression model, the maximum probability estimated or the individual likelihood in a classification task, anomaly score for outliers detection, and so on. It could also be a discrete number, for instance the predicted rank in a ranking problem, a one-loss function for binary or multi-class classification which means that the reward equals 1 if two instances are in the same class and 0 otherwise. Figure 31 gives an example. A machine learning has been trained to predict whether an individual will re-offend (giving the estimated probability associated). The importance of individual attributes give an interpretation of the difference between the probability of that individual (70% here) and the mean probability of a reference group (14%). It is relevant to choose non predicted recidivist individual references as element of comparison. We observe that the age of the individual (21 years old) has increased the probability by 46% while the absence of priors crimes has reduced it by 9%. The prediction is thus decomposed into an additive model where each attribute has a main effect.

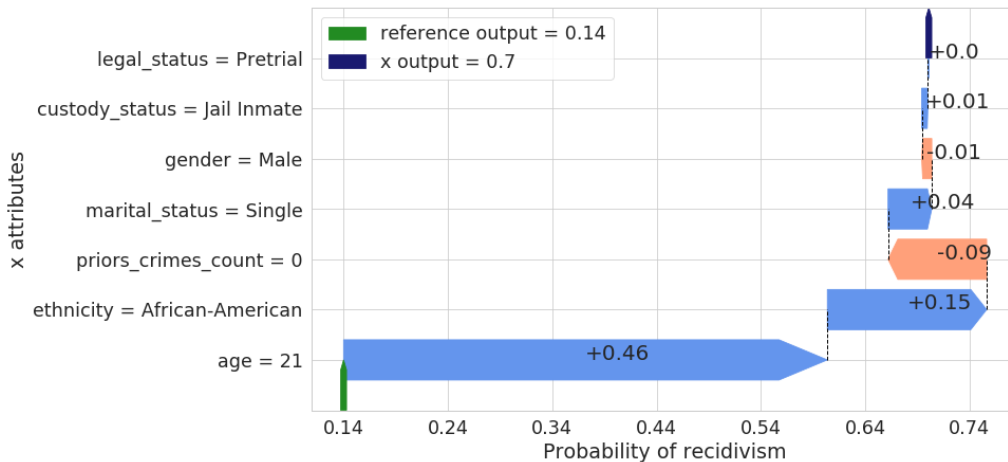


Figure 31: Illustration of how are used Attribute Importance on a particular prediction on the COMPAS dataset [17].

In Machine Learning, a common choice for the reward is $v(S) = \mathbb{E}[f(X)|X_S = \mathbf{x}_S^*]$, where $\mathbf{x}_S^* = (x_j^*)_{j \in S}$ and X_S the element of X for the coalition S .

For any $S \subset \mathcal{M}$, let's define $z(\mathbf{x}^*, \mathbf{r}, S)$ such that $z(\mathbf{x}^*, \mathbf{r}, \emptyset) = \mathbf{r}$, $z(\mathbf{x}^*, \mathbf{r}, \mathcal{M}) = \mathbf{x}^*$ and

$$z(\mathbf{x}^*, \mathbf{r}, S) = (z_1, \dots, z_d) \text{ with } z_i = \begin{cases} x_i^* & \text{if } i \in S \\ r_i & \text{if } i \notin S \end{cases}.$$

As explain in [37], each reference $\mathbf{r}$ sets a single-game with $v(S) = f(z(\mathbf{x}^*, \mathbf{r}, S)) - f(\mathbf{r})$, $v(\emptyset) = 0$ and $v(\mathcal{M}) = f(\mathbf{x}^*) - f(\mathbf{r})$. If we sample many references from a dataset with one distribution D^{pop} , we can recover the more classical definition of attribute contributions according to a base value $\mathbb{E}_{r \sim D^{pop}}[f(r)]$ like in [34], [54] and [53] by averaging all the Shapley Values of each single game obtained on individuals from the reference population.

Out of direct interest, estimating $\mathbb{E}[f(X)|X_S = \mathbf{x}_S^*]$ is not an easy task. In [28], the authors exhibit two popular ways using conditional or interventional distributions. Both techniques have their own pro and cons which lead to different definitions of what attribute relevance should be. When we use a reference $\mathbf{r}$ and create a new instance $z(\mathbf{x}^*, \mathbf{r}, S)$, we are in the second family. The main drawback is that we assume feature independence (only for the calculation of $v(S)$) which can produce out-of-distribution samples. On the other hand, the conditional approach could give non zero contribution to attributes which do not impact the reward function. It happens when an irrelevant attribute is correlated to relevant ones. Anyway, the methods that we present here are agnostic to the chosen approach, it can be conditional or interventional. Even if we use the interventional approach as an illustration, the same techniques will also work by changing the reward function or the way we estimate $v(S)$.

4.2.3 Estimation of Shapley Values by Monte Carlo

The Monte Carlo method and its variants are by far the most commonly used techniques for estimating Shapley Values. In its classical form, this approach samples random permutations between players (e.g. [54], [53], [36]). A criterion is to reduce the number of times the costly reward function is asked. The Algorithm 1 is an optimized version of the algorithm proposed by [54] which divides by two the use of the reward function v . This strategy can also be combined with stratified sampling (e.g. [26], [36]) in order to reduce the number of iterations required. Indeed, estimated variances of each player are updated online. Let $\pi(\{1, \dots, d\})$ be the set of all ordered permutations of $\{1, \dots, d\}$ in Algorithm 1.

Data: instance $\mathbf{x}^*$ and $\mathbf{r}$, the reward function v and the number of iterations T .

Result: The Shapley Values $\hat{\phi} \in \mathbb{R}^d$

initialization $\hat{\phi} = \{0, \dots, 0\}$ and $\hat{\sigma}^2 = \{0, \dots, 0\}$;

for $t=1, \dots, T$ **do**

Choose the subset resulting of an uniform permutation $O \in \pi(\{1, \dots, d\})$ of the features values ;

$v^{(1)} = v(\mathbf{r})$;

$\mathbf{b} = \mathbf{r}$;

for j in O **do**

$\mathbf{b} = \begin{cases} x_i^* & \text{if } i = j \\ b_i & \text{if } i \neq j \end{cases}, \text{ with } i \in \{1, \dots, d\}$;

$v^{(2)} = v(\mathbf{b})$;

$\phi_j = v^{(2)} - v^{(1)}$;

if $t > 1$, $\hat{\sigma}_j^2 = \frac{t-2}{t-1} \hat{\sigma}_j^2 + (\phi_j - \hat{\phi}_j)^2/t$;

$\hat{\phi}_j = \frac{t-1}{t} \hat{\phi}_j + \frac{1}{t} \phi_j$;

$v^{(1)} = v^{(2)}$;

end

end

Algorithm 1: Optimized version of Monte Carlo algorithm.

[36] demonstrate that the estimation error could be bounded with high probability regarding some assumptions. These results are still valid for the optimized version.

4.2.4 Estimation of Shapley Values using a projected stochastic gradient algorithm

In our article [50], we introduce a new approach for estimating Shapley Values based on a projected stochastic gradient algorithm. An alternative way to Monte Carlo is to use the equivalence between the initial formulation of the Shapley Values and an optimization problem. Indeed, these values are the only solution of a weighted linear regression problem with an equality constraint (see [45], [34] and [1]). The convex optimization problem is given by Equation (4.1).

$$\begin{aligned}
& \underset{\phi \in \mathbb{R}^d}{\operatorname{argmin}} && \sum_{S \in \mathcal{M}, S \neq \{\emptyset, \mathcal{M}\}} w_S [v(S) - \sum_{j \in S} \phi_j]^2 \\
& \text{subject to} && \sum_{j=1}^d \phi_j = v(\mathcal{M}),
\end{aligned} \tag{4.1}$$

where the weights $w_S = \frac{(d-1)}{\binom{d}{|S|}|S|(d-|S|)}$.

Our goal is to reduce the number of coalitions used and then the reward function evaluations while controlling the estimation error. The function we want to minimize is:

$$F(\phi) = (X\phi - Y)^T W (X\phi - Y) = \frac{1}{n} \sum_{i=1}^n n w_i (y_i - \mathbf{x}_i^T \phi)^2 = \frac{1}{n} \sum_{i=1}^n g_i(\phi),$$

where $n = 2^d - 2$ is the number of all coalitions except the full and the empty coalitions, $W = \operatorname{diag}(w_S)$ is a diagonal matrix of size $n \times n$ whose diagonal elements are the weights w_S without $S = \emptyset$ or $\mathcal{M}$. X is a binary matrix of size $n \times d$ representing all coalitions (except null and full coalitions): each element in the column j of the row i is equal to 1 if the player j is in the coalition i , 0 otherwise. $\mathbf{x}_i$ corresponds to the i -th row of X . $\mathbf{Y} = (y_i)_{i \in \{1, \dots, n\}}$ is a vector of size n with $y_i = v(S)$ where S is the i -th coalition. We assume that for all $i \in \{1, \dots, n\}$, $|y_i| < C$, with $C > 0$ a constant. In our configuration $v(S) = v(z(\mathbf{x}^*, \mathbf{r}, S)) - v(\mathbf{r})$, each coalition S is indexed by an integer i .

We denote $K_1 = \{\phi; \sum_{j=1}^d \phi_j = v(\mathcal{M})\}$ and $K_2 = \{\phi; \|\phi\| \leq D\}$ two convex sets. $D > 0$ is a constant which is required to demonstrate the theoretical performance (see [50]), but in practice it can be large enough. This allows to avoid large gradient norm, but it can be removed by using a small learning rate. The convex set K_1 ensures that the solution respects the equality constraint $\sum_{j=1}^d \phi_j = v(\mathcal{M})$. F is a μ -strongly convex function defined on a convex set:

$$K = \{\phi; \sum_{j=1}^d \phi_j = v(\mathcal{M}); \|\phi\| \leq D\} = K_1 \cap K_2.$$

This optimization problem has a unique solution ϕ^* which is the Shapley Values if D is chosen such that $\|\phi^*\| \leq D$.

We denote $\phi_t = (\phi_t^i)_{i \in \{1, \dots, d\}}$ the Shapley Values estimator at the iteration t . We also define $i_t \sim U_n(\{1, \dots, n\})$, with $U_n(\{1, \dots, n\})$ a discrete uniform distribution with support $\{1, \dots, n\}$, the coalition randomly draw for the iteration t . To find the unique minimum of F on K , the Projected Stochastic gradient algorithm at each iteration t follows the rules:

$$\begin{aligned}
i_t & \sim U_n(\{1, \dots, n\}), \\
\phi_t & = \operatorname{Proj}_K(\phi_{t-1} - \gamma_t \nabla g_{i_t}),
\end{aligned}$$

where

- γ_t is a constant or decreasing step-size (also called the learning rate). $\forall t, \gamma_t > 0$;
- Proj_K is the orthogonal projection on K ;
- $\mathbb{E}[\nabla g_{i_t} | \mathcal{F}_{t-1}]$ is the gradient of F at ϕ_{t-1} . $\mathcal{F}_{t-1}$ is the σ -field generated by $x_1, y_1, \dots, x_{t-1}, y_{t-1}$;
- $\mathbb{E}[\|\nabla g_{i_t}\|^2] \leq B^2$ (finite variance condition). $B > 0$.

For each iteration t , we need to find the orthogonal projection of $\phi_t \in \mathbb{R}^d$ onto the convex set $K_1 \cap K_2 = K$ where K_1, K_2 are convex sets. Dykstra's algorithm ([10], Algorithm 2) can be used because we know how to project onto the sets K_1 and K_2 separately. For $\phi_t \in \mathbb{R}^d$, if ϕ_t does not belong to either K_1 or K_2 :

$$\begin{aligned}
\operatorname{Proj}_{K_1}(\phi_t) & = \phi_t - \frac{\sum_j \phi_j^t - v(\mathcal{M})}{d}, \\
\operatorname{Proj}_{K_2}(\phi_t) & = \phi_t \times \frac{D}{\|\phi_t\|}.
\end{aligned}$$

The theoretical properties of convergence are established in [50].

Data: instance $\phi_t \in \mathbb{R}^d$, Proj_{K_1} and Proj_{K_2} and the maximum number of iterations L .

Result: the orthogonal projection of ϕ_t onto K

initialization: $\alpha_1 = \phi$ and $p_1 = q_1 = 0$;

```

for  $l=1, \dots, L$  do
     $\beta_l = \text{Proj}_{K_2}(\alpha_l + p_l)$ ;
     $p_{l+1} = \alpha_l + p_l - \beta_l$ ;
    if  $\text{Proj}_{K_1}(\beta_l + q_l) = \alpha_l$  then
        break;
    else
         $\alpha_l = \text{Proj}_{K_1}(\beta_l + q_l)$ ;
         $q_{l+1} = \beta_l + q_l - \alpha_l$ ;
    end
end
return  $\alpha_l$ 

```

Algorithm 2: Dykstra's algorithm 2.

4.2.5 Implementation: Python module Shapkit

Thales has developed a Python module that implements the previous approaches, which is available in Open Source ¹. The method used is:

- agnostic: no particular information on the model is needed, it works with black box algorithms. We only define a reward function (e.g. the model output);
- local: the explanation is computed at instance level. Thus, each interpretation is associated to a given prediction;
- More suitable for tabular data with meaningful features.

In Table 7, we propose the requirements of the module.

Module	Version
Python	≥ 3.6
numpy	$\geq 1.17.2$
pandas	$\geq 0.25.3$
matplotlib	$\geq 2.2.3$
seaborn	$\geq 0.9.0$
tqdm[optional]	$\geq 4.26.0$

Table 7: *shapkit* requirements

The method is a post-hoc explanation, so the user does not have to change her usual pipeline. Firstly, the user has to train the model:

```
model.fit(X_train, y_train)
```

Then, the user defines her reward function fc (e.g. simply set by the model output):

```
fc = lambda x: model.predict_proba(x)
```

Then the user selects an instance of interest x for which she need more interpretation and picks also one or several reference(s) (instance or dataset of individuals). If the number of features is not too high (said lower than 10), then the exact Shapley Values can be computed.

```
true_shap = ShapleyValues(x=x, fc=fc, ref=reference)
```

If the dimension is too high, then the users can use some approximation approaches:

- Monte Carlo algorithm:

```
mc_shap = MonteCarloShapley(x=x, fc=fc, ref=reference, n_iter=1000)
```

¹<https://github.com/ThalesGroup/shapkit>

- batch Monte Carlo algorithm: If the reward function f_c can handle data set of inputs, the batch version of Monte Carlo algorithm is more efficient as it calls the reward function only once:

```
mc_shap_batch = MonteCarloShapleyBatch(x=x, fc=fc, ref=reference, n_iter=1000)
```

- Projected Stochastic Gradient Descent algorithm:

```
sgd_est = SGDshapley(d, C=y.max())
sgd_shap = sgd_est.sgd(x=x, fc=fc, ref=reference
, n_iter=5000, step=.1, step_type="sqrt")
```

4.2.6 Experimental evaluation: Adult income use case

4.2.6.1 Adult income use case

The data set used comes from the repository [4]. It is also known as the “Census Income” data set. The prediction task is to predict if a person makes over 50000\$ a year. Thus the target variable is named “income” and worth either $> 50000\$$ or $\leq 50000\$$. The features used to predict the target are continuous or categorical. Hereafter is a description of feature types and categories:

For all the experiments concerning that data set, we will remove the feature “education-num” and filter individuals of race “White” and “Black”. The first feature only encodes “education” and provides no more information.

4.2.6.2 Use of Shapley Values as local attribute importance

In this use case, f is a machine learning model which outputs the probability that a given individual makes over 50000\$ a year. Assuming that the algorithm is quite effective, however it only returns a probability score without any details on how it has made its choice. We would like to know how each attribute (characteristic of the individual) influences the model output. Furthermore, contributions explain the difference between the individual prediction and the mean prediction for all references. These references are defined by the user. As an example, in that case, references are selected into another predicted class. That is to say, if we want to interpret the result of an individual classified into the high income class, we pick as references other individuals predicted as earning less than 50000\$ a year.

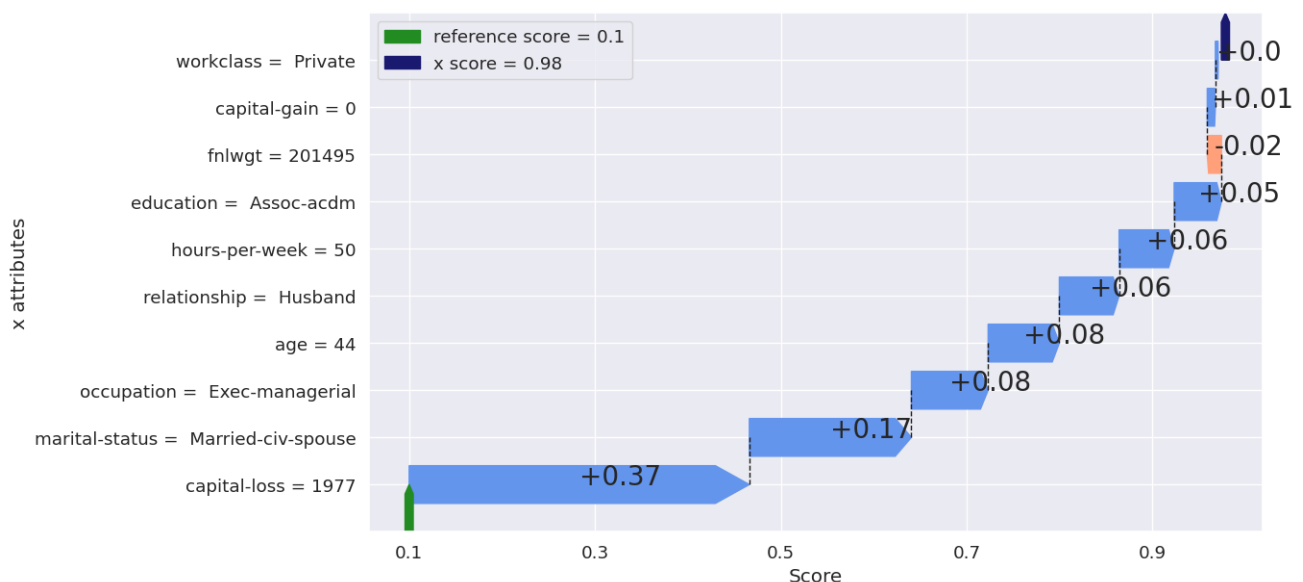


Figure 32: Attribute Importance

The Figure 32 displays the kind of interpretation associated to a given prediction for the individual x . We want to understand the model decision associated to an individual x . As an example, here the selected individual has a probability of 98% to earn more than 50000\$ a year (the blue tick at top right). Attribute importance are computed with respect to one or several references. On this example, we chose only predicted low income class individuals as good elements of comparison. The mean probability for that group of references is about 1% (green tick at the bottom left). Finally, the gap between our individual prediction

Features	Description
age	continuous
workclass	Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked
education	Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool
education-num	continuous
marital-status	Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse
occupation	Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces
relationship	Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried
race	White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black
sex	Female, Male
capital-gain	continuous
capital-loss	continuous
hours-per-week	continuous
native-country	United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad-Tobago, Peru, Hong, Holand-Netherlands

Table 8: Feature description of the Adult Income data set

and the mean reference prediction is split by the attribute importance. The sum of all contributions equals that difference. Now, we can analyze that the capital-loss, the "married-civ-spouse" status and the "exec-managerial" occupation of the individual x increase respectively by 37%, 17% and 8% the estimated probability.

The algorithms used to compute the Shapley Values comes from the open source library available here: <https://github.com/ThalesGroup/shapkit>.

4.3 xAI in intrusion detection

The work over the past few months has led to designing an explainability enhancing system, which is flexible, easy to understand and effective. Based on the earlier discoveries in [55], the idea of explaining Artificial Neural Networks (ANN) using the Hybrid Oracle-Explainer approach based on comprehensible decision trees has been further expanded. The theoretical framework created there has been turned into a full-fledged

product. It will be capable of being utilised with a variety of models, datasets and parameters, expanding its use even beyond Intrusion Detection Systems (IDS), if needed. Here, accompanying the first presentation of the prototype, the theory behind it has been presented, followed by the discussion on the design, implementation and the current performance of the system.

4.3.1 Theoretical foundation

This section presents a short recap of the previous work in [55], in order to provide the context for the current development. It discusses:

- The three principles that have provided the foundation for the chosen design,
- A general overview of the explanation generation process,
- Crucial algorithms.

In [55], the three fundamental rules were indicated that a good explainability enhancing system for IDS should follow:

1. In the context of IDS, the accuracy and reliability of a system are the top priority.
2. One phenomenon can have more than one explanation [44].
3. The delivered explanation should be simple and help to develop trust [43].

Those considerations have ultimately led to the choice of the surrogate-based approach that utilises comprehensible decision trees to generate explanations. Since separate models were used to explain decisions, the performance was not affected.

Finally, thanks to the inherently explainable nature of decision trees paired with accessible visualisation library [58], we could deliver explanations that are easy to understand for any human operator.

Figure 33. is a schematic representation of a general process that takes place when an explanation for a decision is requested. This process remains at the heart of the developed solution and is basically left intact,

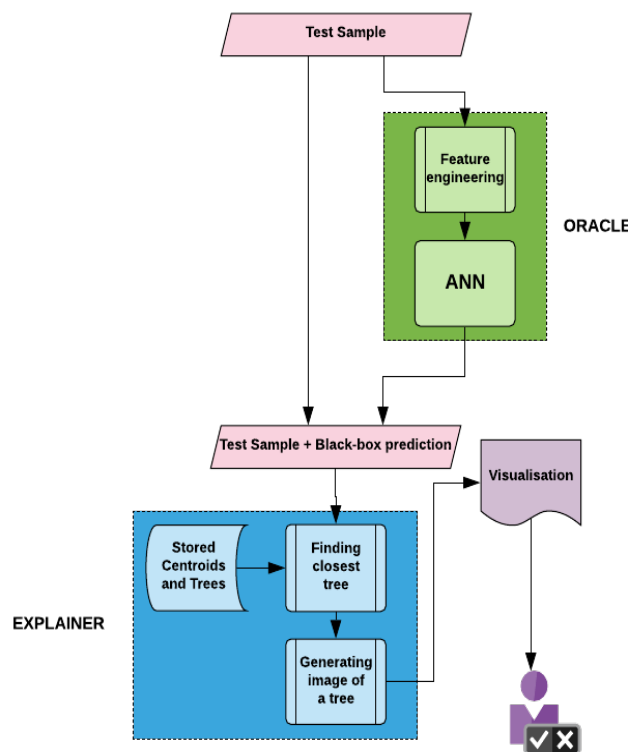


Figure 33: Overview of the explanation generation process

being adapted to work with variety of users, models and datasets.

The chosen sample is first being transformed to the form used by the opaque classifier during training. In this case, the role of the black-box machine learning algorithm is fulfilled by a Feed Forward Artificial Neural

Network (ANN). Then, after obtaining a prediction, the sample in its original form, along with the Oracle output, is being passed to the Explainer module. There, it is compared with the saved centroid of each cluster made during the training process in order to find n closest in terms of Euclidean norm. Following that, starting with the closest centroid, decision tree trained on the according cluster is being retrieved. If its prediction matches that of the classifier (Oracle), the search stops and the local explainer is returned. Otherwise, the algorithm continues until it finds a supporting tree or runs out of centroids. In that case, the tree linked to the closest centroid is returned. Finally, visualisation of the decision tree is being drawn, together with a highlighted path leading to the prediction made by the chosen explainer.

The authors of [8] have developed the procedures for generating comprehensible decision trees and finding the most appropriate tree for the investigated sample. Both procedures proved fundamental to the explainer module of the discussed solution.

The prepared training set and test set are imported.

No additional transformations are performed, so clusters are generated directly on the training set. The number and size of the clusters is controlled by the parameter k , which indicates the level of representativity. The higher its value, the bigger the clusters, and therefore, there are fewer of them there. To compute the clusters, the method uses a microaggregation heuristic named the Mean Distance to Average Vector (MDAV). A detailed description is available in [19], while the algorithm can be found in [8]. Then, for each cluster, a centroid is calculated, that will be further used in the explanation provision process, together with the trained decision trees.

4.3.2 Implementation

4.3.2.1 Technologies

The solution is a web application based on the REST API. Since the main focus of the application is around artificial intelligence (AI), the Python language in version 3.7.8 was the natural choice. There exists a rich ecosystem of libraries and frameworks supporting anything AI-related, from simple data transformations up to ready models and solutions. Additionally, some of the most popular frameworks for backend, such as Django or Flask, have been written in Python. In Table 9, the main modules that have been included in the solution are listed.

For the sake of this work, TensorFlow has been used, which is a high performance, open source library, provided by the developers and engineers of the Google Brain team. It serves as a capable support for machine and deep learning, as it is currently implemented in an array of scientific and industry applications [2]. Keras, which used to operate on top of TensorFlow, is now its integral part. It brings an astounding speed of experimentation along with incomparable user experience. This is attained via modular, expandable design. Keras was brought into existence more as an interface than an autonomous library. Keras received full support in the TensorFlow library, and makes possible intuitive coding of both machine and deep learning procedures [15].

Dtreeviz is a visualisation library dedicated to decisions trees, which allows one to easily generate good-looking and user-friendly diagrams [57]. The aforementioned Flask was selected over Django due to its lightweight approach conjoined with the flexibility it offers. It provided the necessary tools to develop an effective REST API for the application, without enforcing additional complexity and bloated overhead. It can also be easily customised and expanded with the import of extra modules for specific tasks; this is why it proved to be the best framework for the needs of the project.

As for the user interface, the selected technology was React JS, version 10.0.0. It is one of the main modern frontend technologies, competing with Angular or Vue. One of its main advantages is the reusability of the components, and the focus on quick development. In addition, it is stable, thanks to the one-direction data flow.

4.3.2.2 Clusters, centroids and trees generation

In Listing 4.1, the code used to generate new clusters, centroids and trees has been presented. It is a convenient wrap up for low-level methods, developed by the authors of [8].

It has been utilised in the discussed project, and here it can be seen for example in line 29, where MDAV is used to create clusters and centroids.

To use the methods from this class, it is necessary to have the dataset used during model training. Listing 4.2. shows the step by step usage of the TreeExplainer class. First, necessary data and transformations are

loaded. Then, for the specified k values, clusters and centroids are generated and saved. Clusters made in this step are then used to train explanation trees with a desired depth.

The whole procedure and implementation is simple and straightforward. For now, it is performed "manually", i.e. trees and clusters are prepared beforehand. One of the goals is to automate the whole process and fully integrate it with the developed API.

Listing 4.1: Cluster, centroids and trees generation

```
class TreeExplainer:

    def __init__(self, train_set_x, train_set_y, label_encoder = None):
        """
        :param train_set_x: train set values used to train model
                           for which explanations are to be generated
        :param train_set_y: train set targets used to train model
                           for which explanations are to be generated
        :param label_encoder: in case of categorical data, label encoder used
                              during training process is needed
        """
        self.train_set_x=train_set_x
        self.train_set_y=train_set_y
        if label_encoder is not None:
            self.train_set_y = label_encoder.transform(self.train_set_y)
        pass

    def generate_clusters_and_centroid(self, representativity=None):
        """
        Generates clusters and centroids for given array of representativity values.
        :param representativity: array with representativity values. Default is [0.2].
        :return: array of clusters and array of their respective centroids
                 for each representativity value.
        For example clusters=[[for representativity = 0.2 ],[for
representativity = 0.5]]
        , centroids=[[for representativity = 0.2 ],[for representativity = 0.5]]
        """
        if representativity is None:
            representativity = [0.2]

        clusters = []
        centroids_of_clusters = []
        K = [int(len(self.train_set_x) * r) for r in representativity]
        for k in K:
```

Module	Version
tensorflow	2.3.1
pandas	1.1.4
scikit-learn	0.23.2
numpy	1.19.0
dtreeviz	1.1.2
joblib	0.17.0
scipy	1.5.4
Flask	1.1.2

Table 9: Main Python modules used in the system

```

        clustering , centroids = mdav(self.train_set_x , self.train_set_y , k)
        clusters.append(clustering)
        centroids_of_clusters.append(centroids)

    return clusters , centroids_of_clusters

def generate_trees(self , clusters , tree_depths=None):
    """
    :param clusters: clusters for specified representativity
                    created with method "generate_clusters_and_centroid"
    :param tree_depths: Array with values specifying depth of
                        a tree , for example [2,3]
    :return: Array with trees for given centroids with depth variants ,
            for example [[trees_depth_2],[trees_depth_3]]
    """
    if tree_depths is None:
        tree_depths = [3]

    tree_explanations = []
    for tree_depth in tree_depths:
        explanations = gen_explanations(clusters[0] , tree_depth)
        tree_explanations.append(explanations)

    return tree_explanations

```

Listing 4.2: Use of TreeExplainer Class

```

# 1) Load train set used to fit your model.
# It should be separated to values and targets and be ndarrays.
# Some transformations here can be case specific.

train_set_x = pd.read_csv(Path("Datasets/train_set_x.csv")
                          .resolve())
                          .to_numpy()
train_set_y = pd.read_csv(Path("Datasets/train_set_y.csv")
                          .resolve(), encoding="UTF-8").T.squeeze()
                          .to_numpy(dtype='str')

# 2) In case of categorical classification
# label encoder used during training will be necessary

label_encoder = load(Path("TransformationTools/label_encoder.joblib").resolve())

# 3) Generate clusters and centroids , then save them.

treeExplainer = TreeExplainer(train_set_x , train_set_y , label_encoder)
clusters , centroids = treeExplainer.generate_clusters_and_centroid(
    representativity=[0.005])

dump(clusters , Path('Clusters_and_centroids/clusters_K_0_005.data').resolve())
dump(centroids , Path('Clusters_and_centroids/centroids_K_0_005.data').resolve())

# 4) Generate Trees with desired depth and save them
depths = [3,4,5]
trees = treeExplainer.generate_trees(clusters , depths)

for tree , depth in zip(trees , depths):
    dump(tree , 'Trees/explanations_K_0_005_depth'+str(depth)+' .data')

```

4.3.2.3 Finding an explanation and generating visualisation

Listing 4.3. presents what the core of the system is; i.e the mechanism of finding the best decision tree for the investigated sample. It is packed within the Explainer class and the implementation of the whole process is quite explicit. It consists in the user picking the sample of their choice; the process will be discussed in the following section.

Then, it is passed through the API which calls the method *"generate_visualisation_for_chosen_sample"* from an instance of the Explainer class. There, the necessary resources are loaded and together with the sample and its classification are passed to the implementation of the algorithm, which is the modified version of [8]; protection from the case where one class tree is found has been added.

Next, after all the necessary components, such as a matching decision tree, centroid and cluster number have been chosen, they are all passed to the dtreeviz[57] that is responsible for generating an image of a tree. This is then sent back to the user.

Listing 4.3: Explainer module implementation

```
class Explainer(object):
```

```

    def make_path_based_on_config(self):
        '''
        This function is a temporary solution necessary for
        finding appropriate resources
        :return: Path modifier to the appropriate resources
        '''
        path_fragment = []
        if config.parameter_k == 0.005:
            path_fragment.append('0.005')
        else:
            path_fragment.append('0.2')

        path_fragment.append('_depth' + str(config.tree_depth_parameter))
        return path_fragment

    def find_best_tree(self, sample, prediction):
        '''
        Finds n closest matching trees.
        If no matching tree is found (i.e. one which has the same prediction),
        the closest one is being picked
        :param predicted_sample: sample for which decision tree is to be found
        :param prediction:
        :return: explanation_ext_prediction == prediction of found decision tree
                ret_exp == found decision tree
                ret_cen == centroid
                        of the cluster based on witch chosen decision tree was
                        trained
                ret_cluster_number == cluster position
                        in the collection (cluster based on which decision tree was
                        trained)
        '''
        explanation_ext_prediction = []
        explanation_ext_prediction = []

        p, q, ret_exp, ret_cen, ret_cluster_number =
            pre_explanations_ext(self.saved_trees,
                                self.centroids_centers[0],
                                sample.reshape(1, -1),
                                [prediction],
                                3)
        explanation_ext_prediction.append(q)

```

```

    return explanation_ext_prediction , ret_exp , ret_cen , ret_cluster_number

def generate_visualization_for_chosen_sample(self
    , sample, prediction , data_manager):
    '''
    :param sample: sample chosen by user to explain
    :param prediction: sample classification done by Oracle
    :param data_manager: instance of a class responsible for managing data
                        (loading, transforming etc.)
    :return: Visualization generated by dtreeviz module in .svg format
    '''

    # Collecting necessary resources
    path_modifier = self.make_path_based_on_config()
    # This variable is to simply build a path for a specific file needed
    temp_path = 'Clusters/clusters_K_' + path_modifier[0] +
    '.data'
    self.clusters = load(Path(temp_path).resolve())
    temp_path = 'Clusters/centroids_K_' + path_modifier[0] +
    '.data'
    self.centroids_centers = load(Path(temp_path).resolve())
    temp_path =
        'Models/explanations_K_' + path_modifier[0] + path_modifier[1] +
        '.data'
    self.saved_trees = load(Path(temp_path).resolve())

    # Utilization of the method "find_best_tree"
    explanation_ext_prediction , ret_exp , ret_cen , ret_cluster_number =
        self.find_best_tree(sample, prediction)

    # Generating visualisation with dtreeviz
    viz = dtreeviz(ret_exp[0],
                    self.clusters[0][ret_cluster_number[0]][0],
                    self.clusters[0][ret_cluster_number[0]][1],
                    target_name=data_manager.get_target_column_name(),
                    orientation='LR',
                    fancy=False,
                    feature_names=data_manager.get_columns_name(),
                    class_names=list(data_manager.get_target_groups()),
                    X=sample)

    return viz.svg()

```

Listing 4.4: Guided Explanation Search

```

def pre_explanations_ext(explanations , centroids , X, T, n):
    predictions_first = []
    predictions_guided = []
    ret_exp = []
    ret_cen = []
    ret_cluster_numbers = []
    for sample, truth in zip(X, T):
        is_there_non_one_class_classifier = False
        secure_mins = []
        end = n
        mins = np.array([dist(sample, c) for c in centroids]).argsort()

        #Protection from picking one class classifier
        while is_there_non_one_class_classifier is not True:
            # select the n closest classifiers
            start = end - n

```

```

checked_mins = mins[start:end]
end = end + end

for m in checked_mins:
    if len(explanations[m].classes_) > 1:
        secure_mins.append(m)

if len(secure_mins) > 0:
    is_there_non_one_class_classifier = True

for m in secure_mins:
    exp = explanations[m]
    exp_pred = [int(exp.predict([sample]))]
    if exp_pred[0] == truth:
        break
    predictions_first.append(int(explanations[mins[0]].predict([sample])[0]))
    predictions_guided.append(exp_pred[0])
    ret_exp.append(exp)
    ret_cen.append(centroids[m])
    ret_cluster_numbers.append(m)
return predictions_first, predictions_guided
, ret_exp, ret_cen, ret_cluster_numbers

```

4.3.3 Presentation of the web application

In this section, the current interface of the web application has been presented.

Figure 34. shows the main hub of the application. There is a table containing the samples with predictions (in red). From here, by clicking the button on the right side of each sample, the user can request the explanation of a decision. Then, depending on the configuration (representativity parameter and maximal tree depth), a chart is generated. Examples of such are presented in Figures 35 and 36.

First, the example presents how the tree may look when the representativity was set to 0.005 and the maximal depth limited to 3. Generally speaking, with the low representativity, the value trees have the tendency to be rather minimal. It is because there are more small cluster then; thus, the probability of having representants from many classes is limited, which originates in a higher count of shallow trees.

xAI SAFAIR

Web setup?

Explainer Tool

Settings

Templates

Help

Network Frames

#ID	Mean Packet Length	Std Packet Length	↓ Variance Packet Length	Label	Action
1	1016.51	3507.29	11351605.68	DoS Hulk	 Explain Prediction
2	855.8	1777.31	3562922.17	DoS Hulk	 Explain Prediction
3	779.73	1700.08	3408011.73	DDoS	 Explain Prediction
4	813.12	1637.57	3058808.09	DoS Hulk	 Explain Prediction
5	740.33	1258.57	1802289.02	BENIGN	 Explain Prediction

Rows per page: 51-5 of 100<>




Figure 34: Sample selection

On the other hand, as it is clearly shown in Figure 36, an increased representativity value leads to bigger, much more complex trees. On the chart, it was set to 0.2 with the max tree depth set to 5. Although this tree

is still readable, as the path leading to the explainer is highlighted in orange, some level of comprehensibility is lost. In the end, the selection of parameters is a fine balancing act, depending on the user's needs. It is also reinforced by the fact, that those values have a direct impact on the explainer's fidelity and performance, which is further shown in the next section.

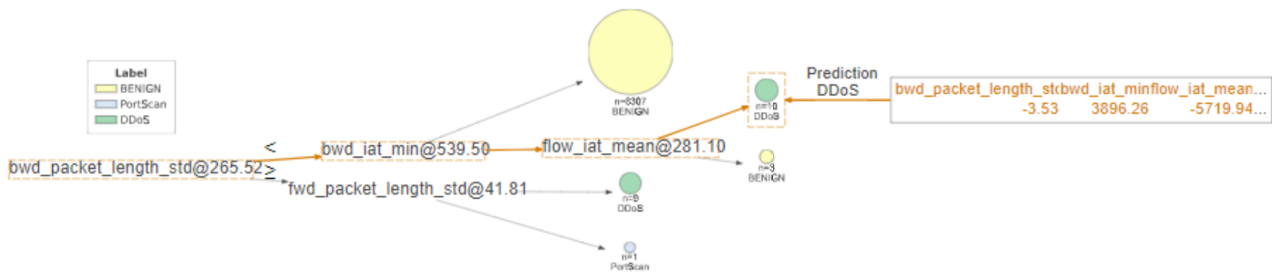


Figure 35: Example of an explanatory tree chart (representativity=0.005, max tree depth=3)

4.3.4 Evaluation

Evaluation tests were performed using 2 explainers, each made using different k values (where k can be think of as a representativity parameter). They are all presented in Table 10.

The quality of the explainer relies heavily on the value of the variable k. The more clusters there are, and therefore, the more explainers, the better the accuracy. Decision trees trained on clusters use the default configuration delivered by the scikit-learn package; only the maximal depth of a tree was limited to 4. The algorithm searches for the matching explainer from the 3 closest centroids.

k	Clusters	Samples in each cluster	Achieved accuracy	Referring score table
0.2	5	253 202	95%	Table 11
0.005	200	6 330	99%	Table 12

Table 10: Tested Variants of Explainers

Class	Precision	Recall	Support
Benign	98%	98%	567 807
DDoS	82%	76%	32 296
DoS GoldenEye	53%	19%	2542
DoS Hulk	81%	91%	57 335
DoS Slowhttptest	23%	19%	1406
DoS Slowloris	0%	0%	1490
FTP-Patator	15%	35%	2016
PortScan	99%	99%	39 614
SSH-Patator	0%	0%	1418

Table 11: Explainer Scores with Sample Support for k=0.2



Figure 36: Example of an explanatory tree chart (representativity=0.2, max tree depth=5)

Class	Precision	Recall	Support
Benign	99%	99%	567 807
DDoS	99%	99%	32 296
DoS GoldenEye	93%	87%	2542
DoS Hulk	96%	98%	57 335
DoS Slowhttptest	93%	94%	1406
DoS Slowloris	58%	66%	1490
FTP-Patator	91%	93%	2016
PortScan	99%	99%	39 614
SSH-Patator	94%	97%	1418

Table 12: Explainer Scores with Sample Support for $k=0.005$

4.3.5 Summary

Since the previous work package, the concept of the Hybrid Oracle-Explainer Approach using comprehensible decision trees has further been developed. A basis for an accessible web application that can be widely used in a variety of other projects as a separate solution has been created. On top of being flexible and simple to use, the solution also offers good practical results.

Chapter 5 Fairness

Technical work about fairness metrics, implementation and application to real use case is new in this deliverable compare to D7.2. On this topic, SAFAIR contribute at three levels: fairness metrics, algorithms that achieve fairness and application of these two things on real data.

5.1 Introduction

From the US law regulation, fairness is often understood through two distinct notions: **disparate treatment** and **disparate impact**. The fairness of a decision making process is judged by these terms. At first, is the process based on subjective individual elements renders a disparate treatment ? On the other hand, the outcomes should not exhibit any differences between persons whose sensitive attributes are not alike. As an example, we wish that machine learning model outputs and errors could be similar for two sub-populations. In the same way, two similar individuals should receive closed model decisions. Machine Learning models tend to reproduce and amplify biases. These biases can come from the data: there are known biases such as selection bias when sampling is poor, historical biases, when a population is disadvantaged, and so on. But biases that can also be due to algorithms: some recommendation algorithms lock people in bubbles instead of offering them new possibilities or when data are unbalanced. If a data set already exhibits biases, then there is a fewer opportunity that future observations will contradict former predictions. Indeed, new data collection could be guided by past decisions of the machine learning model. Moreover, labels are often created by humans and a model will tend to reproduce those bias to increase its performance. At group-level discrimination, a minority group could be penalized because of a small sample size or features less informative for that particular group characteristics. The consequence is a disparate result between this group and the majority group.

Remove sensitive features from the training data set can be not enough. Indeed, these features can be correlated with others and then the discriminant information is still present. The model finds new associations between features if it improves the loss function. Even if no sensitive feature is easily identified as a potential risk for fairness, information can still be spread or hidden through features and hard to recognize. One popular solution is to keep them during the learning phase of the model as a constraint.

Finally, Fairness is a very challenging task because we can not detect automatically all kind of biases. Then, if we have been able to identify any discrimination, controlling that biases could decrease model performance (task metrics, speed, memory consumption, etc.) and thus dealing with a trade-off between Fairness and a sufficient performing model.

5.2 Measures

There is no consensus about a unique mathematical formulation of fairness. In this section, we give some classical measures for a binary classification task and a binary sensitive feature. However, it can be extended to different task and with several sensitive features. For the remaining of that study, we note a binary class prediction $\hat{Y} \in \{0, 1\}$ and a binary sensitive attribute $Z \in \{0, 1\}$. Fairness metrics can be divided into two groups: **group fairness** and **individual fairness**.

5.2.1 Group fairness

5.2.1.1 Demographic Parity

Demographic Parity is also known as Independence or Statistical Parity. It guarantees that mean acceptance rates from different groups are the same, i.e. the model outputs disparity according to a sensitive attribute. The exact formulation is that $\hat{Y}$ is independent of the sensitive feature Z , thus $\mathcal{P}(Y = y|Z = 0) = \mathcal{P}(Y = y|Z = 1)$, $\forall y \in \{0, 1\}$. In practice, equality is relaxed and we can use the popular $p\%$ -rule metric ([62]):

$$\min \left(\frac{P(\hat{Y} = 1|Z = 1)}{P(\hat{Y} = 1|Z = 0)}, \frac{P(\hat{Y} = 1|Z = 0)}{P(\hat{Y} = 1|Z = 1)} \right) \geq \frac{p}{100} \in [0, 1].$$

The higher is p , the better is the fairness treatment of sensitive feature Z .

5.2.1.2 Equal of Opportunity

Equal of Opportunity ([63]) ensures that $\hat{Y}$ is independent of Z conditional on Y . It measures the model errors according to a sensitive attribute and is also called Positive Rate Parity. We can compute the equal opportunity score for the positive class in the same way as the previous $p\%$ -rule:

$$\min \left(\frac{P(\hat{Y} = 1|Z = 1, Y = 1)}{P(\hat{Y} = 1|Z = 0, Y = 1)}, \frac{P(\hat{Y} = 1|Z = 0, Y = 1)}{P(\hat{Y} = 1|Z = 1, Y = 1)} \right) \geq \frac{p}{100} \in [0, 1].$$

This constraint tends to reduce errors uniformly in all groups if we consider also the negative group ($Y = 0$).

5.2.1.3 Predictive Rate Parity

[63] defined the Predictive Rate Parity. Y should be independent of Z conditional of $\hat{Y}$. This is equivalent to satisfying both $\mathcal{P}(Y = 1|\hat{Y} = 1, Z = 0) = \mathcal{P}(Y = 1|\hat{Y} = 1, Z = 1)$ and $\mathcal{P}(Y = 0|\hat{Y} = 0, Z = 0) = \mathcal{P}(Y = 0|\hat{Y} = 0, Z = 1)$. We will consider as an example the positive predictive parity which guarantees equal chance of success $Y = 1$ given acceptance $\hat{Y} = 1$ for all sensitive groups. A score associated to these parity is defined hereafter:

$$\min \left(\frac{P(Y = 1|Z = 1, \hat{Y} = 1)}{P(Y = 1|Z = 0, \hat{Y} = 1)}, \frac{P(Y = 1|Z = 0, \hat{Y} = 1)}{P(Y = 1|Z = 1, \hat{Y} = 1)} \right) \geq \frac{p}{100} \in [0, 1].$$

This score is a group-conditioned calibration which measures the label distribution knowing model prediction according to an attribute.

5.2.1.4 Differential Fairness

[21] proposes the Differential Fairness which states that a mechanism $M(x)$ is ε -differentially fair in a framework (A, Θ) , where A is the ensemble of attributes to protect, if for all $\theta \in \Theta$ with $x \sim \theta$ and $y \in \text{Range}(M)$,

$$\exp(-\varepsilon) \leq \frac{P_{M,\theta}(M(x) = y|\mathbf{s}_i, \theta)}{P_{M,\theta}(M(x) = y|\mathbf{s}_j, \theta)} \leq \exp(\varepsilon),$$

for all $(\mathbf{s}_i, \mathbf{s}_j) \in A \times A$, where $P(\mathbf{s}_i|\theta) > 0, P(\mathbf{s}_j|\theta) > 0$.

5.2.2 Individual Fairness

With the previous criteria, we capture discrimination against an individual as it belongs to a group. Individual-level discrimination measure how the model handle one individual comparing the most similar individuals. The consistency is given by $1 - \frac{1}{n} \sum_{i=1}^n \sum_{j \in \text{Knn}(i)} |\hat{Y}_i - \hat{Y}_j| \in [0, 1]$, where $\text{Knn}(i)$ are the K-Nearest Neighbors of i .

5.2.3 The Impossibility theorem of fairness

We can demonstrate that it is impossible to fulfill the three criteria (demographic parity, equal of opportunity and predictive parity rate) simultaneously (e.g. see [46]). As an example, considering that the sensitive feature Z and the target feature Y are dependent, then either Demographic Parity holds or Predictive Rate Parity but not both:

$$Z \perp\!\!\!\perp \hat{Y} \Rightarrow Z \not\perp\!\!\!\perp Y|\hat{Y}.$$

In the experimental section, we will observe that algorithms which try to satisfy demographic parity constraint degrade the equal of opportunity and predictive parity rate constraints.

5.3 Algorithms

Several techniques have been proposed to achieve Fairness objective. Some are based on regularization methods: the direct and indirect information which rely on the sensitive attribute(s) are penalized (see e.g. [42]). Other techniques will use new representations that hide attribute, e.g. by learning a latent space using deep learning models (see e.g. [32]). Finally, a fairness constraint could be introduced by modifying either the inputs or the outputs of the algorithms (see e.g. [65]).

5.3.1 Fair Adversarial Networks

We consider the adversarial network proposed by [33]. They use this architecture in context where they want to introduce independence between the outputs of a classifier and some nuisance parameters. Our objective is to use this architecture to render independent the outputs of a classifier regarding given sensitive features. The main idea is to constraint the optimization objective via a zero-sum game which is classically used for training Generative Adversarial Networks (GANs, see [22]). The architecture is made of two neural networks. The first one is the classifier trained to perform a specific classification task. Another neural network, called the adversarial network, will be use to constraint the parameters of the previous network. The overview of the architecture is given by Figure 37.

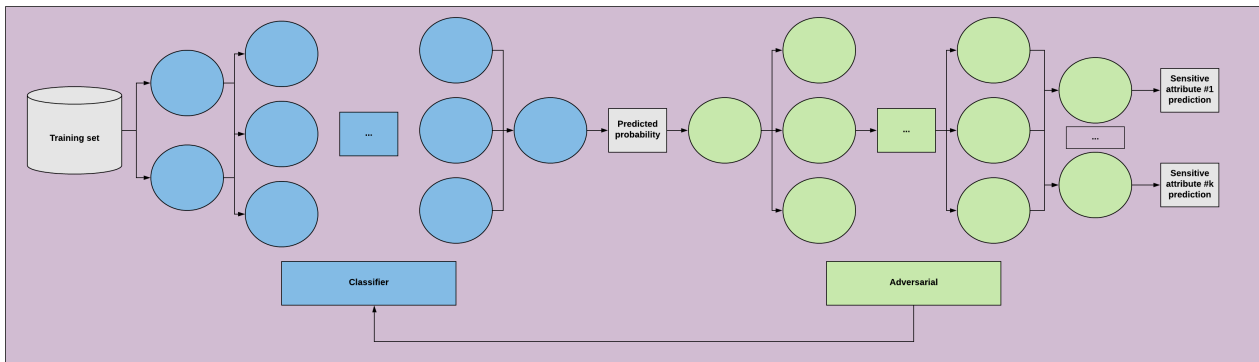


Figure 37: Overview of adversarial network architecture of [33]

The classifier outputs predictions $\hat{Y}$ based on the input X , the data set in which sensitive features Z are removed. The adversarial network try to predict sensitive attributes $\hat{Z}$ from the predictions $\hat{Y}$. We note prediction losses of the target and sensitive features by $Loss_y(\theta_{clf})$ and $Loss_Z(\theta_{clf}, \theta_{adv})$ where θ_{clf} and θ_{adv} are classifier and adversarial network parameters respectively.

The objective of the classifier can be formulated as:

$$\min_{\theta_{clf}} [Loss_y(\theta_{clf}) - \lambda Loss_Z(\theta_{clf}, \theta_{adv})].$$

The model should be good for predicting the right targets while ensuring that sensitive attributes will not be derived from them. Increasing the penalization term λ tends to produce a fairer classifier but often decreases performance. In the meantime, the adversarial network is designed to predict sensitive attributes based on the predictions from the classifier. Its own objective is defined by the optimization problem:

$$\min_{\theta_{adv}} Loss_Z(\theta_{clf}, \theta_{adv}).$$

Given θ_{clf} , the adversarial model is not affected by the performance of the classifier.

The procedure for training a fair classifier is summarized bellow:

- 1) Pre-train the classifier (update θ_{clf})
- 2) Pre-train the adversarial network based on the predictions from the pre-trained classifier (update θ_{adv})
- 3) During T iterations, do:
 - first train the adversarial for a single epoch while keeping the classifier fixed (update θ_{adv})
 - then train the classifier on a single sampled mini batch while keeping the adversarial fixed (update θ_{clf}).

The adversarial training starts at step 3. The zero sum game will render the classifier fair and accurate as much as possible (depending on the penalization term λ). The overall objective addressed is:

$$\theta_{\text{clf}}, \theta_{\text{adv}} = \arg \min_{\theta_{\text{clf}}} \max_{\theta_{\text{adv}}} [Loss_Y(\theta_{\text{clf}}) - \lambda Loss_Z(\theta_{\text{clf}}, \theta_{\text{adv}})].$$

5.3.2 Fair Random Forest

[42] propose an enhancement of the classical Random Forest algorithm (see [25]) to ensure that the model is fair regarding some selected features. At each node of the trees, split are chosen based on a criterion which takes into account the target and also the protected attribute(s). Indeed, the algorithm chooses the best split according to the information gain $G(T, a)$ where T is the set of training examples and a is an attribute to split on. The authors note $T_{1,\dots,k}$ the set of points in the k branches of a tree. Then, the information gain $G(T, a)$ can be measured by:

$$G(T, a) = I(T) - \sum_{T_i \in \text{splits}(a)} \frac{|T_i|}{|T|} I(T_i),$$

where $|T|$ is the number of elements in set T , and $I(T)$ is an impurity measure associated to set T . If we consider a classification task, then the Gini impurity could be used:

$$I_{\text{Gini}}(T) = 1 - \sum_{T_i \in \text{splits}(\text{label})} \left(\frac{|T_i|}{|T|} \right)^2.$$

By doing so, no constraint is added to preserve fairness property of the model. That is why the authors propose to measure the impurity score also for the sensitive feature Z . We will denote I^l and I^Z the impurity scores of the class label and the sensitive attribute, and the gains G^l and G^Z respectively. Then, Fair Forest algorithm selects at each node the best split with respect to the fair gain measure G_{fair} :

$$G_{\text{fair}}(T, a) = G^l(T, a) - G^Z(T, a).$$

In order to compare impurity scores on a similar scale range, a factor $\frac{1}{1 - |\text{splits}(a)|}$ is applied. This new criterion discourages the selection of any feature correlated with both the sensitive feature and the target. The information which is directly or indirectly linked with a protected attribute is automatically penalized. The algorithm tries to find a good trade off between accuracy and fairness.

In the study, the authors show how to handle continuous features as well. We extend the work done in that paper by introducing a penalization λ to control more finely the trade-off between accuracy and fairness. The information gain considered is now expressed by the following formula:

$$G_{\text{fair}}(T, a) = G^l(T, a) - \lambda G^Z(T, a).$$

5.4 Experimental evaluation: Adult income use case

We will test the two algorithms described here-above and analyze their performance, both in terms of prediction and fairness, on the adult income use case (see Section 4.2.6.1. The data set is classically divided into a train and test parts. The algorithms learn from train data set and we evaluate metrics on the test one. We carry out two experiments. The first one is about protecting one binary sensible feature, the variable “Sex”. Then, we test another approach for preserving discrimination among several binary sensible features (“Sex” and “Race”). Independently of these experiments, the baseline metrics concerning the prediction task can be set a dummy classifier whose decisions are constant towards the majority class (e.g. predicting always a less than 50000\$ income).

accuracy	ROC_AUC	PR_AUC
0.75	0.5	0.25

Table 13: Baseline metrics from a dummy classifier (always predicts the majority class with max probability)

For the rest of that section, we use the following abbreviations in Tables:

- ROC_AUC: the area under the ROC curve;
- PR_AUC: the area under the Precision-Recall curve;
- DF: Differential Fairness;
- PPR: the positive predictive rate;
- EOS: the Equal of Opportunity Score.

For both model, Fair adversarial networks and Fair Forest, we present the results obtained with different penalization values. This value is the λ term and thus manages the trade off between prediction performance and fairness. The Fair Forest model with a penalization value of 100.0 performs worst that the dummy classifier and then the fairness metrics are not relevant.

5.4.1 Fairness regarding the “Sex” feature

Tables 14 and 15 summarizes the performance and fairness metrics when we want protect the “Sex” feature.

penalization	n_iters	accuracy	ROC_AUC	PR_AUC	DF	p%-rule	PPR	EOS
0.01	100	0.85	0.91	0.78	1.19	30	98	82
0.1	100	0.85	0.91	0.78	1.14	32	97	85
1.0	100	0.85	0.90	0.77	0.82	44	86	96
10.0	100	0.84	0.88	0.74	0.42	66	77	72
100.0	100	0.82	0.87	0.73	0.29	75	82	60

Table 14: Fair Adversarial Networks - Protected feature “Sex”

penalization	n_trees	accuracy	ROC_AUC	PR_AUC	DF	p%-rule	PPR	EOS
0.01	100	0.84	0.86	0.72	1.30	27	98	77
0.1	100	0.84	0.87	0.73	1.27	28	96	80
1.0	100	0.84	0.82	0.69	0.93	39	82	89
10.0	100	0.76	0.76	0.56	0.38	68	88	60

Table 15: Fair Forest - Protected feature “Sex”

As we expect, performances tend to decrease when the penalization term increases but the differential fairness and the p%-rule metric are improved. This is coherent with the constraint that the model learned should output predictions $\hat{Y}$ less dependent on the sensitive feature “Sex” Z . An interesting phenomena is the behavior of the other fairness metrics. Indeed, the positive predictive rate and the equal opportunity score are erratic or become worse. This is an example of illustration of the impossibility theorem of fairness. Indeed, as we improve the p%-rule metric, we know that it would be impossible to achieve perfect scores for the other fairness criteria. Another way to understand that results, is to see that the added constraint reduces performances of the classifier and thus the ratio of estimated probabilities of $\hat{Y}$ given Y and the opposite decline or at least become unstable.

5.4.2 Fairness regarding the “Sex” and “Race” features

In this subsection, we are interesting on protected several binary sensitive features. That is why we will consider simultaneously the “race” and “sex” features. A direct method for doing this could be to extend the previous work on single sensitive feature by adding new penalization terms and thus regularization. This approach seems to work well in practice but incorporate more hyper-parameters to control fairness properties of the model. Another possibility to manage several sensitive features, is dealing with the interaction of all those attributes. The problem is more challenging because not only the model outputs need to be independent of these sensitive features (e.g. Male/Female, Black/White) but also according to their interactions (e.g. Black Female / White Male, etc.). This can be viewed as a fairness constraint toward a new designed feature which

encodes all possible interactions. One advantage of that approach is the theoretical guarantee which states that if the model is ϵ -Differential Fairness (ϵ -DF) for a Cartesian product D (e.g. Sex $\times$ Race) then ϵ is an upper bound on ϵ -DF for any nonempty proper subset of D (e.g. Sex and Race features)(see [21]). In other words, controlling ϵ -DF of a created intersection feature assures that individual ϵ -DF are also well managed.

For each model, we present performance metrics and differential fairness of the intersection between “Sex” and “Race” sensitive features in Tables 16 and 17.

penalization	n_iters	accuracy	ROC_AUC	PR_AUC	DF
0.01	100	0.85	0.91	0.78	1.98
0.1	100	0.85	0.91	0.78	1.92
1.0	100	0.85	0.91	0.78	1.41
10.0	100	0.83	0.89	0.75	1.00
100.0	100	0.79	0.88	0.74	0.89

Table 16: Fair Adversarial Networks - Protected both “Sex” and “Race” features

penalization	n_trees	accuracy	ROC_AUC	PR_AUC	DF_inter
0.1	100	0.84	0.88	0.73	1.96
0.1	100	0.84	0.86	0.72	1.94
1.0	100	0.85	0.85	0.71	1.77
10.0	100	0.77	0.78	0.61	1.11

Table 17: Fair Forest - Protected both “Sex” and “Race” features

Once again, the models achieve to improve the inter-sectional differential fairness by increasing the penalization term and thus the fairness constraint during the learning phase.

The remaining tables of that section (18, 19, 20, 21) show results for individual sensitive features per model. We observe empirically that differential fairness metrics for the “Sex” and “Race” features are both upper bounded by those of the intersection feature of that two variables. This is what we were expected based on the theoretical property.

penalization	DF	p%-rule	PPR	EOS
0.01	1.19	30	98	82
0.1	1.14	32	98	85
1.0	0.87	42	87	99
10.0	0.51	60	86	71
100.0	0.50	61	91	66

Table 18: Fair Adversarial Networks - “Sex” Fairness metrics

By dealing only with a new designed feature (the intersection between several features), we have to manage only one penalization term while having guarantees that differential fairness and p%-rule for each sensitive feature will be improved. However, due to the impossibility theorem of fairness, there is a little chance that the other fairness criteria will respect high expectation quality.

To conclude this experimental section, it seems that the neural network classifier trained with an adversarial network and based on the algorithm described previously, is a good candidate to find an efficient machine learning model which respect fairness constraints. Moreover, the procedure is quite general and thus could be extended on other tasks than classification and with categorical (not only binary) and continuous sensitive features. Indeed, a strength of the deep learning framework is the ability to keep the architecture while modifying for instance the loss functions based on the use case tasks or the constraints that we want.

penalization	DF	p%-rule	PPR	EOS
0.01	0.89	41	90	83
0.1	0.86	42	89	84
1.0	0.64	53	82	97
10.0	0.59	55	85	94
100.0	0.76	47	97	92

Table 19: Fair Adversarial Networks - "Race" Fairness metrics

penalization	DF	p%-rule	PPR	EOS
0.01	1.27	28	95	81
0.1	1.29	28	96	79
1.0	1.29	28	99	77
10.0	0.80	45	95	86

Table 20: Fair Forest - "Sex" Fairness metrics

penalization	DF	p%-rule	PPR	EOS
0.01	0.98	38	98	83
0.1	1.02	36	98	79
1.0	0.94	39	97	86
10.0	0.80	45	98	96

Table 21: Fair Forest - "Race" Fairness metrics

Chapter 6 Summary and Conclusion

This document detailed some approaches that will be implemented in the final demonstrator of SPARTA WP7 SAFAIR program. The Chapter 2 is dedicated to the ethical and legal challenges in regards of European and International documents. The approaches that will implemented in the demonstrator cover three of the identified challenges: the security and robustness (Chapter 3), explainability (Chapter 4) and fairness (Chapter 5). About security and robustness, we implement some evasion attacks and defence mechanisms against them in ML. We propose in the demonstrator classical methods of evasion attack, as well as the method proposed by [3]. Concerning defence, we propose some classical approaches, plus one which will be submitted in a conference. Four use case are considered: two concern cybersecurity (malware detection in PDF and intrusion detection with AI), one other is about evasion attacks on graph anomaly detection and the last one is medical (healthcare images). Concerning the explainability, we detailed an approach of local explanation based on Shapley Value, which has been published [50] and whose implementation is in Python module *shapkit*. Moreover, we introduce the use of xAI for intrusion detection. Based on work of [55], we use a hybrid oracle-explainer approach based on comprehensible decision trees to explain Artificial Neural Network. Concerning fairness, we propose two approaches based of adversarial network [33] and Fair Random Forest [42], and propose several metrics that address different challenges of fairness.

We will continue all this work for the final demonstrator. One continuation of this work will be to apply the Shapley Values on a cyber security use case and to explore some global explanations approaches.



Chapter 7 List of Abbreviations

Abbreviation	Translation
ADAM	Adaptive Moment Estimation
AI	Artificial Intelligence
ALTAI	Assessment List for Trustworthy Artificial Intelligence
ANN	Artificial Neural Network
B2G	Business to Government
BIM	Basic Iterative Method
CW	Carlini and Wagner attack
CLI	Command-line Interface
CIA	Centered Initial Attack
CIPL	Centre for Information Policy Leadership
DF	Differential Fairness
DPIA	Data Protection Impact Assessment
DPO	Data Protection Officer
EC	European Commission
EOS	Equal of Opportunity Score
EPRS	European Parliamentary Research Service
EU	European Union
FGSM	Fast Gradient Sign Method attack
GAN	Generative Adversarial Network
GDPR	General Data Protection Regulation
IDS	Intrusion Detection System
Iter-FGSM	Iterative Fast Gradient Sign Method attack
OECD	Organisation for Economic Co-operation and Development
MDAV	Mean Distance to Average Vector
ML	Machine Learning
NN	Neural Network
PPR	Positive Predictive Rate
PGD	Projected Gradient Descent
ReLU	Rectified Linear Unit
RF	Random Forest
ROC	Receiver Operating Characteristic
SAFAIR	Secure and Reliable AI Systems for Citizen
SVM	Support Vector Machine
xAI	Explainable Artificial Intelligence

Chapter 8 Bibliography

- [1] Kjersti Aas, Martin Jullum, and Anders Løland. Explaining individual predictions when features are dependent: More accurate approximations to Shapley values. *arXiv:1903.10464 [cs, stat]*, February 2020. URL <http://arxiv.org/abs/1903.10464>. arXiv: 1903.10464.
- [2] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from tensorflow.org.
- [3] Boussad Addad, Jerome Kodjabachian, and Christophe Meyer. Clipping free attacks against artificial neural networks. *arXiv:1803.09468 [cs, stat]*, March 2018. URL <http://arxiv.org/abs/1803.09468>. arXiv: 1803.09468.
- [4] Adult Income data set. Adult income data set repository. <http://archive.ics.uci.edu/ml/datasets/Adult>.
- [5] Giuseppe Ateniese, Giovanni Felici, Luigi V. Mancini, Angelo Spognardi, Antonio Villani, and Domenico Vitali. Hacking Smart Machines with Smarter Ones: How to Extract Meaningful Data from Machine Learning Classifiers. *arXiv:1306.4447 [cs, stat]*, June 2013. URL <http://arxiv.org/abs/1306.4447>. arXiv: 1306.4447 version: 1.
- [6] Alejandro Barredo Arrieta, Natalia Diaz Rodriguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado González, Salvador Garcia, Sergio Gil-Lopez, Daniel Molina, V. Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. *Information Fusion*, 2019.
- [7] Battista Biggio, Igino Corona, Davide Maiorca, Blaine Nelson, Nedim Srndic, Pavel Laskov, Giorgio Giacinto, and Fabio Roli. Evasion Attacks against Machine Learning at Test Time. *arXiv:1708.06131 [cs]*, 7908:387–402, 2013. doi: 10.1007/978-3-642-40994-3_25. URL <http://arxiv.org/abs/1708.06131>. arXiv: 1708.06131.
- [8] Alberto Blanco-Justicia and Josep Domingo-Ferrer. *Machine Learning Explainability Through Comprehensible Decision Trees*, pages 15–26. 08 2019. ISBN 978-3-030-29725-1. doi: 10.1007/978-3-030-29726-8_2.
- [9] K Borg. *Real time detection and analysis of pdf files*. PhD thesis, 2013.
- [10] James P. Boyle and Richard L. Dykstra. A method for finding projections onto the intersection of convex sets in hilbert spaces. In Richard Dykstra, Tim Robertson, and Farroll T. Wright, editors, *Advances in Order Restricted Statistical Inference*, pages 28–47, New York, NY, 1986. Springer New York. ISBN 978-1-4613-9940-7.
- [11] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 39–57. IEEE, 2017.
- [12] Nicholas Carlini and David Wagner. Towards Evaluating the Robustness of Neural Networks. *arXiv:1608.04644 [cs]*, March 2017. URL <http://arxiv.org/abs/1608.04644>. arXiv: 1608.04644.
- [13] Deepayan Chakrabarti. Autopart: Parameter-free graph partitioning and outlier detection. In *European Conference on Principles of Data Mining and Knowledge Discovery*, pages 112–124. Springer, 2004.
- [14] Anirban Chakraborty, Manaar Alam, Vishal Dey, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. Adversarial attacks and defences: A survey. *CoRR*, abs/1810.00069, 2018. URL <http://arxiv.org/abs/1810.00069>.
- [15] François Chollet et al. Keras. <https://github.com/fchollet/keras>, 2015.
- [16] code. Vicomtech code repository. <https://github.com/sparta-safair/Vicomtech/blob/master>.
- [17] COMPAS data set. COMPAS data set repository. <https://www.propublica.org/datastore/dataset/compas-recidivism-risk-score-data-and-analysis>.
- [18] Cuan, Damien, Delaplace, and Valois. Malware Detection in PDF Files Using Machine Learning. *SE-CRYPT 2018*, 2018.

- [19] Josep Domingo-Ferrer and Vicenç Torra. Ordinal, continuous and heterogeneous k -anonymity through microaggregation. *Data Mining and Knowledge Discovery*, 11:195–212, 09 2005. doi: 10.1007/s10618-005-0007-5.
- [20] Goicoechea-Telleria I. Orduna-Urrutia R. Echeberria-Barrio X., Gil-Lerchundi A. Deep learning defenses against adversarial examples for dynamic risk assessment. *13th International Conference on Computational Intelligence in Security for Information Systems (CISIS 2020)*, August 2020. doi: 10.1007/978-3-030-57805-3_30. URL https://doi.org/10.1007/978-3-030-57805-3_30.
- [21] James Foulds, Rashidul Islam, Kamrun Naher Keya, and Shimei Pan. An intersectional definition of fairness, 2018.
- [22] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014.
- [23] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples, 2014.
- [24] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and Harnessing Adversarial Examples. *arXiv:1412.6572 [cs, stat]*, March 2015. URL <http://arxiv.org/abs/1412.6572>. arXiv: 1412.6572.
- [25] Tin Kam Ho. Random decision forests. In *Proceedings of the Third International Conference on Document Analysis and Recognition (Volume 1) - Volume 1*, ICDAR '95, page 278, USA, 1995. IEEE Computer Society. ISBN 0818671289.
- [26] E. Molina J. Tejada J. Castro, D. Gómez. Improving polynomial estimation of the shapley value by stratified random sampling with optimum allocation. *Computers and Operations Research Volume 82 , Pages 180-188*, 2017.
- [27] Kittilsen. *Detecting malicious pdf documents*. PhD thesis, 2011.
- [28] I. Elizabeth Kumar, Suresh Venkatasubramanian, Carlos Scheidegger, and Sorelle Friedler. Problems with shapley-value-based explanations as feature importance measures. 2020.
- [29] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world. *arXiv preprint arXiv:1607.02533*, 2016.
- [30] Alexey Kurakin, Ian Goodfellow, and Samy Bengio. Adversarial examples in the physical world. *arXiv:1607.02533 [cs, stat]*, February 2017. URL <http://arxiv.org/abs/1607.02533>. arXiv: 1607.02533.
- [31] X. Liao, L. Ding, and Y. Wang. Secure machine learning, a brief overview. In *2011 Fifth International Conference on Secure Software Integration and Reliability Improvement - Companion*, pages 26–29, June 2011. doi: 10.1109/SSIRI-C.2011.15.
- [32] Christos Louizos, Kevin Swersky, Yujia Li, Max Welling, and Richard Zemel. The variational fair autoencoder, 2015.
- [33] Gilles Louppe, Michael Kagan, and Kyle Cranmer. Learning to pivot with adversarial networks, 2016.
- [34] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 2017.
- [35] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks, 2017.
- [36] Sasan Maleki, Long Tran-Thanh, Greg Hines, Talal Rahwan, and Alex Rogers. Bounding the estimation error of sampling-based shapley value approximation with/without stratifying. 2013.
- [37] Luke Merrick and Ankur Taly. The explanation game: Explaining machine learning models with cooperative game theory. 2019.
- [38] Christoph Molnar. Interpretable machine learning, a guide for making black box models explainable. 2019. URL <https://christophm.github.io/interpretable-ml-book/>.
- [39] Christoph Molnar, Giuseppe Casalicchio, and Bernd Bischl. Quantifying model complexity via functional decomposition for better post-hoc interpretability. 2019.
- [40] Nuria Oliver. Governance in the era of data-driven decision-making algorithms. *Women Shaping Global Economic Governance*, CEPR Press.
- [41] Marek Pawlicki, Rafal Kozik, and Michal Choraś. Artificial neural network hyperparameter optimisation for network intrusion detection. In *Intelligent Computing Theories and Application - 15th International Conference, ICIC 2019, Nanchang, China, August 3-6, 2019, Proceedings, Part I*, pages 749–760, 2019. doi: 10.1007/978-3-030-26763-6_72. URL https://doi.org/10.1007/978-3-030-26763-6_72.

- [42] Edward Raff, Jared Sylvester, and Steven Mills. Fair forests: Regularized tree induction to minimize model bias, 2017.
- [43] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1135–1144, 2016.
- [44] Ariella Richardson and Avi Rosenfeld. A survey of interpretability and explainability in human-agent systems. In *XAI Workshop on Explainable Artificial Intelligence*, pages 137–143, 2018.
- [45] Luis M. Ruiz, Federico Valenciano, and Jose M. Zarzuelo. The family of least square values for transferable utility games. *Games and Economic Behavior*, 24(1):109 – 130, 1998. ISSN 0899-8256. doi: <https://doi.org/10.1006/game.1997.0622>. URL <http://www.sciencedirect.com/science/article/pii/S0899825697906229>.
- [46] Kailash Karthik S. The impossibility theorem of machine fairness – a causal perspective, 2020.
- [47] Uri Shaham, Yutaro Yamada, and Sahand Negahban. Understanding Adversarial Training: Increasing Local Stability of Neural Nets through Robust Optimization. *Neurocomputing*, 307:195–204, September 2018. ISSN 09252312. doi: 10.1016/j.neucom.2018.04.027. URL <http://arxiv.org/abs/1511.05432>. arXiv: 1511.05432.
- [48] Lloyd S Shapley. A value for n-person games. In *Contributions to the Theory of Games*, 2.18:307–317, 1953.
- [49] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *ICISSP*, pages 108–116, 2018.
- [50] Vincent Thouvenot Simon Grah. A projected stochastic gradient algorithm for estimating shapley value applied in attribute importance. *Machine Learning and Knowledge Extraction - 4th IFIP TC 5, TC 12, WG 8.4, WG 8.9, WG 12.9 International Cross-Domain Conference, CD-MAKE 2020, Dublin, Ireland, August 25–28, 2020, Proceedings*, August 2020. doi: 10.1007/978-3-030-57321-8. URL <http://dx.doi.org/10.1007/978-3-030-57321-8>.
- [51] SPARTA. Deliverable D7.1, AI systems threat analysis mechanisms and tools, July 2020.
- [52] SPARTA. Deliverable D7.2, Preliminary description of AI systems security mechanisms and tools, July 2020.
- [53] Erik Strumbelj and Igor Kononenko. An efficient explanation of individual classifications using game theory. *J. Mach. Learn. Res.*, volume 11, pages 1-18, 2010.
- [54] Erik Strumbelj and Igor Kononenko. Explaining prediction models and individual predictions with feature contributions. *Knowledge and information systems* 41.3, 647-665, 2014.
- [55] M. Szczepański, M. Choraś, M. Pawlicki, and R. Kozik. Achieving explainability of intrusion detection system by hybrid oracle-explainer approach. In *2020 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2020. doi: 10.1109/IJCNN48605.2020.9207199.
- [56] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv:1312.6199 [cs]*, February 2014. URL <http://arxiv.org/abs/1312.6199>. arXiv: 1312.6199.
- [57] P. Grover T. Parr. "explained.ai". URL <https://explained.ai/decision-treeviz/index.html>.
- [58] P. Grover T. Parr. Github - parrr/dtreeviz: A python library for decision tree visualization and model interpretation. <https://github.com/parrr/dtreeviz>, 2020. (Accessed on 07/Dec/2020).
- [59] Torres and De Los Santos. Malicious pdf documents detection using machine learning techniques. 2018.
- [60] Andreas Tsamados, Nikita Aggarwal, Josh Cows, Jessica Morley, Huw Roberts, Mariarosaria Taddeo, and Luciano Floridi. The Ethics of Algorithms: Key Problems and Solutions. SSRN Scholarly Paper ID 3662302, Social Science Research Network, Rochester, NY, July 2020. URL <https://papers.ssrn.com/abstract=3662302>.
- [61] Jaewon Yang and Jure Leskovec. Defining and evaluating network communities based on ground-truth. *Knowledge and Information Systems*, 42(1):181–213, 2015.
- [62] Muhammad Bilal Zafar, Isabel Valera, Manuel Gomez Rodriguez, and Krishna P. Gummadi. Fairness constraints: Mechanisms for fair classification, 2015.
- [63] Muhammad Bilal Zafar, Isabel Valera, Manuel Gomez Rodriguez, and Krishna P. Gummadi. Fairness beyond disparate treatment and disparate impact. *Proceedings of the 26th International Conference on World Wide Web*, Apr 2017. doi: 10.1145/3038912.3052660. URL <http://dx.doi.org/10.1145/3038912.3052660>.



- [64] Haichao Zhang and Jianyu Wang. Defense Against Adversarial Attacks Using Feature Scattering-based Adversarial Training. *arXiv:1907.10764 [cs]*, November 2019. URL <http://arxiv.org/abs/1907.10764>. arXiv: 1907.10764.
- [65] Lu Zhang, Yongkai Wu, and Xintao Wu. Achieving non-discrimination in prediction, 2017.